Bezpieczeństwo w sieciach bezprzewodowych

Dr inż. Piotr Remlein

remlein@et.put.poznan.pl

Kryptografia, pojęcia podstawowe

Kryptografia (cryptography) z języka greckiego ***Krypto*** – ukryty, tajny, ***graph*** – pismo słowo.
Pierwotnie oznaczała tajne pismo.
Zajmuje się utajnionym zapisem informacji, czyli
szyfrowaniem (*encryption*) (kodowaniem), przekształceniem tekstu z postaci jawnej na niezrozumiałą (tekst zaszyfrowany, tajny).
Szyfr – metoda utajnionego zapisywania w taki sposób, by **tekst jawny** (*plaintext*) (otwarty *cleartext*) był przekształcany w **tekst zaszyfrowany** (**kryptogram, szyfrogram** *(ciphertext*)
Deszyfrowanie (*decryption*)– proces odwrotny do szyfrowania, umożliwia przekształcenie tekstu zaszyfrowanego do postaci jawnej
Szyfrowanie i deszyfrowanie w nowoczesnej kryptografii są sterowane przez klucz lub klucze kryptograficzne. Czasem deszyfrowanie nie jest potrzebne tzw. szyfrowanie jednokierunkowe, np. przy hasłach dostępu.

Kryptoanaliza – zajmuje się metodami łamania szyfrów.

Kryptologia – dziedzina obejmująca kryptografię i kryptoanalizę.

Steganografia – dział kryptografii - ukrywanie informacji (steganos w języku greckim ukryty)

Steganografia

- Ukrywanie informacji w tekście
  - Modyfikacja położenia linii
  - Modyfikacja położenia wyrazów
  - Użycie znaków interpunkcyjnych
  - Użycie właściwości znaków (np. wysokości)
  - Użycie synonimów

Steganografia

- Ukrywanie informacji w obrazie

Reprezentacja na 48 bitach (8 bitów zmienionych)



Steganografia

- Ukrywanie informacji w dźwięku

- **Kryptosystem** (cryptosystem) to algorytm wraz ze wszystkimi możliwymi tekstami jawnymi, szyfrogramami i kluczami.
- **Algorytm kryptograficzny**, inaczej szyfr (cipher) jest funkcją matematyczną użytą do szyfrowania i deszyfrowania.
- **Algorytm ograniczony** to taki, którego istota musi być trzymana w tajemnicy, by nie został złamany.

Zapis matematyczny:

$$E_K(M)=C$$

$$D_K(C)=M$$

Właściwości funkcji szyfrującej i deszyfrującej

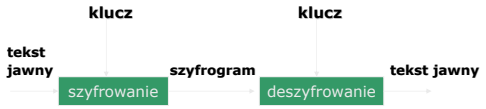
$$D_K(E_K(M))=M$$

Niektóre algorytmy stosują różne klucze szyfrujące i deszyfrujące

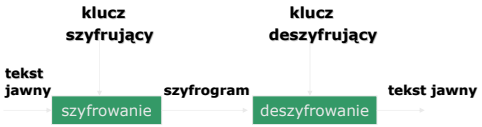
$$E_{K1}(M)=C$$
$$D_{K2}(C)=M$$
$$D_{K2}(E_{K1}(M))=M$$

Bezpieczeństwo systemów kryptograficznych z kluczami jest oparte na kluczach nie na szczegółach algorytmu. Algorytmy mogą być publikowane, wyroby stosujące te algorytmy mogą być stosowane na szeroką skalę itp.

Szyfrowanie i deszyfrowanie z jednym kluczem



Szyfrowanie i deszyfrowanie z dwoma różnymi kluczami



Algorytmy symetryczne (konwencjonalne, z kluczem tajnym, z jednym kluczem, z kluczem pojedynczym)

Klucz szyfrujący jest wyznaczany z klucza deszyfrującego i odwrotnie (często oba są takie same), wymagają uzgodnienia klucza między nadawcą i odbiorcą zanim rozpoczną transmisję.

Klucz musi być utrzymany w tajemnicy, od jego bezpieczeństwa zależy siła zabezpieczenia transmisji.

Algorytmy (szyfry) strumieniowe (potokowe) – przetwarzają bity operacja X-OR (RC4)

Algorytmy (szyfry) blokowe – przetwarzają bloki bitów (DES).

Algorytmy (szyfry) znakowe – stosują podstawienie znakowe (szyfr Cezara)

Algorytmy realizowane komputerowo zwykle operują na blokach 64 bitowych.

Wcześniej (przed erą komputerów) działały na tekście jawnym znak po znaku.

Każda para procesu komunikacji potrzebuje tajnego klucza na wyłączność. Liczba kluczy K rośnie wraz z liczbą uczestników komunikacji N w postępie geometrycznym ***K=0,5*N*(N-1)***

Jeszcze raz *zasada Kerckhoffsa*

Bezpieczeństwo systemu szyfrowania ma zależeć jedynie od ukrycia klucza, a nie od ukrywania algorytmów.

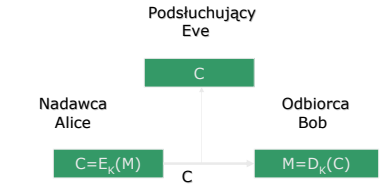
Dlaczego?

Zastąpienie algorytmu innym jest trudne, bo algorytmy są „wbudowane" w sprzęt i oprogramowanie, ich wymiana jest skomplikowana i kosztowna.

Utrzymanie algorytmów w tajemnicy przy wielu użytkownikach w systemie jest prawie nie wykonalne, słabym ogniwem może być jeden z użytkowników.

Algorytmy niejawne są zwykle bardzo słabe.

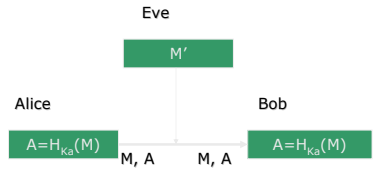
Szyfrowanie z kluczem symetrycznym



Alice i Bob uzgadniają tajny klucz K, Alice musi **bezpiecznie przekazać kopie klucza** Bobowi. (Słaby punkt?)

Potwierdzenie tożsamości z użyciem klucza symetrycznego

Skąd Bob wie, że wiadomość pochodzi od Alice?



H jest funkcją MAC (message authentication code), Ka klucz identyfikacji, Alice wysyła Bobowi wiadomość M i kod A uwiarygodniający wiadomość.
Bob oblicza poprawną wartość A korzystając z klucza Ka i sprawdza, czy nadesłana wartość A jest taka sama.

Algorytmy z kluczem publicznym (algorytmy asymetryczne)

Klucz stosowany do szyfrowania jest inny niż klucz stosowany do deszyfrowania. Klucz deszyfrujący nie może być wyznaczony z klucza szyfrującego.

Klucz szyfrujący – klucz publiczny

Klucz deszyfrujący – klucz prywatny (tajny)

Przy podpisie cyfrowym sytuacja jest odwrotna, nadawca szyfruje swoim kluczem prywatnym, odbiorca weryfikuje wiadomość kluczem publicznym nadawcy (deszyfrujemy kluczem publicznym).

Zaleta: każdy uczestnik komunikacji potrzebuje tylko swojego klucza prywatnego

Wada: duża złożoność operacji (wykorzystywane funkcje potęgowania i wykładnicza) (alg. RSA)

Szyfrowanie z kluczem publicznym



Algorytm generujący parę kluczy, algorytm szyfrujący i deszyfrujący muszą zapewnić, by odebrana wiadomość była wiadomością oryginalną, czyli by warunek **$M=D_{KPrywatnyBob}(E_{KPublicznyBob}(M))$** był spełniony dla wszystkich możliwych M.

Podpis cyfrowy

Odpowiednik kodów potwierdzenia tożsamości



Alice generuje klucze i udostępnia klucz publiczny, by przy jego znajomości inni użytkownicy mogli sprawdzić tożsamość nadawcy

Czy system taki jest naprawdę bezpieczny?

Infrastruktura klucza publicznego PKI
(public key infrastructure)

Istnieją instytucje certyfikujące, w których można sprawdzić autentyczność klucza publicznego, tzn. sprawdzić zależność między otrzymanym kluczem publicznym a jego nadawcą.
Jeśli, nie został zdeponowany klucz publiczny, A i B muszą podjąć decyzję czy kontynuują „korespondencję".

Istnieje możliwość sprawdzenia autentyczności klucza poprzez certyfikowanego partnera komunikacji. (PGP)

Zarządzanie kluczami – później !

© Piotr Remlein 2004

Kryptoanaliza – podstawowe założenie – bezpieczeństwo algorytmu kryptograficznego jest oparte wyłącznie na kluczu.
(Tzn.) zakłada się, że znany jest algorytm szyfrujący i jego implementacja.

TYPY METOD ŁAMANIA SZYFRÓW
Łamanie z szyfrogramami *(ciphertext-only attack)*
Łamanie ze znanym tekstem jawnym *(known-plaintext attack)*
Łamanie z wybranym tekstem jawnym *(chosen-plaintext attack)*
Łamanie z adaptacyjnie wybranym tekstem jawnym *(adaptive-chosen-plaintext attack)*
Łamanie z wybranym szyfrogramem *(chosen-ciphertext attack)*
Łamanie z wybranym kluczem *(chosen-key attack)*
Łamanie z gumową pałką *(rubber-hose cryptoanalysis)*

© Piotr Remlein 2004

Kategorie łamania szyfrów

1. ***Całkowite złamanie szyfru***
2. ***Ogólne wnioskowanie***
3. ***Lokalne wnioskowanie***
4. ***Częściowe wnioskowanie***

Szyfr bezwarunkowo bezpieczny *– (z kluczem jednorazowym)*
Szyfr obliczeniowo bezpieczny (silny)

© Piotr Remlein 2004

Miary złożoności metod łamania

1. **Złożoność danych**
2. **Złożoność przetwarzania**
3. **Wymagania pamięci**

© Piotr Remlein 2004

Klasyczne techniki szyfrowania

Szyfry podstawieniowe
Każdy znak w tekście jawnym jest zastępowany innym znakiem

Prosty *szyfr podstawieniowy*
Homofoniczny *szyfr podstawieniowy (A to 5, 14, lub 28 B to 7, 11 lub 22)*
Poligramowy *szyfr podstawieniowy (ABA to ERT, ACC to ZKK)*
Wieloalfabetowe *szyfry podstawieniowe zastosowanie wielu różnych prostych szyfrów podstawieniowych np. zamiana szyfru następuje ze zmianą pozycji znaku w tekście jawnym.*

© Piotr Remlein 2004

Szyfr Cezara

a b c d e f g h i j k l m n o p q r s t u v w x y z
d e f g h i j k l m n o p q r s t u v w x y z a b c

Program ROT13 *dostarczany z systemem UNIX A – N, B – O itd. każda litera przesunięta jest o 13 pozycji. Dwukrotne zaszyfrowanie programem ROT13 powoduje uzyskanie tekstu jawnego.*
Stosowany do zabezpieczania plików, Używany w sieciach Usenet do ukrywania obraźliwych tekstów, ukrycia rozwiązań zagadek itp.

© Piotr Remlein 2004

Szyfr Playfair

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

balloon – ba lx lo on
ar – RM
mu – CM
hs – BP
ea - IM

© Piotr Remlein 2004

Łamanie szyfru podstawieniowego

Metoda brutalna – wygenerowanie wszystkich możliwych kluczy

Skuteczność ataku zwiększa się jeśli znamy język tekstu jawnego, użyty algorytm szyfrowania

Analiza częstości liter
dwuznaków, …

(metodami komputerowymi łatwe do złamania)

© Piotr Remlein 2004

Szyfry przestawieniowe - transpozycyjne

Opierają się na permutacji liter tekstu jawnego
Wszystkie znaki tekstu jawnego pojawiają się w szyfrogramie, lecz w innej kolejności
Zatem łatwy do złamania metodą analizy częstości (metodami komputerowymi łatwe do złamania)

Najprostszy szyfr tzw.: ***Technika płotu***
Tekst jawny zapisywany jest jako ciąg kolumn, a następnie odczytywany jest jako ciąg wierszy
np.: tekst "spotkanie wieczorem na kolacji" zakodowany techniką płotu o głębokości 2
sokneicoenklci
ptaiwezrmaoaj

sokneicoenklciptaiwezrmaoaj

© Piotr Remlein 2004

Szyfry przestawieniowe - transpozycyjne

Bardziej skomplikowany system polega na zapisaniu komunikatu w prostokącie a następnie odczytanie kolumna po kolumnie, lecz ze zmianą kolejności kolumn

kluczem do algorytmu jest kolejność kolumn

klucz: 3 4 2 1 5 6 7
dane: s p o t k a n
i e j u t r o
n a k o l a c
j i a b c d e

tekst zaszyfrowany:
ojkatuobpeaisinjktlcaradnoce

© Piotr Remlein 2004

Szyfry przestawieniowe - transpozycyjne

Czysty szyfr transpozycyjny jest łatwy do złamania, bo częstość występowania liter jest w szyfrogramie taka sama jak w tekście jawnym. (częstość dwuznaków, sekwencji trójznakowych)

Szyfr transpozycyjny można uczynić znacznie bezpieczniejszym poprzez stosowanie kilku etapów transpozycji.

W rezultacie uzyskiwana jest permutacja bardziej skomplikowana, trudniejsza do rekonstrukcji.

ojkatuobpeaisinjktlcaradnoce
klucz: 4 3 1 2 5 6 7
dane: o j k a t u o
b p e a i s i
n j k t l c a
r a d n o c e
tekst zaszyfrowany: kekd aatn jpja obnr tilo uscc oiae

© Piotr Remlein 2004

Szyfry przestawieniowe - transpozycyjne

W celu zobrazowania podwójnej transpozycji litery tekst jawnego zostały ponumerowane od 1 do 28
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28

po pierwszej permutacji mamy:
3 10 17 24 4 11 18 25 2 9 16 23 1 8 15 22 5 12 19 26 6 13 20 27 7 14 21 28

po drugiej permutacji otrzymujemy:
17 9 5 27 24 16 12 7 10 2 22 20 3 25 15 13 4 23 19 14 11 1 26 21 18 8 6 18

© Piotr Remlein 2004

Przykład szyfru przestawieniowego

W każdy kwadrat wpisana jest strzeżona permutacja alfabetu
Tekst jawny: dzisniczszczegolne
gosieniewydarzylo

szyfrogram: xaf (długość wierszy 13 lub 17)
dau (w zasadzie dowolna)
DQ - RL

H	Q	E	F	K
R	Y	B	O	D
N	U	G	I	S
A	P	C	M	Z
L	W	J	V	X

W	H	S	F	K
L	P	D	N	Q
E	I	U	X	Y
V	B	O	A	M
R	C	G	T	Z

© Piotr Remlein 2004

Tekst jawny dzielony jest na wiersze tej samej długości (dopełniany ustalonymi znakami wypełnienia)

Kodowane są pary liter leżące w sąsiednich wierszach
litera z wyższego rzędu wybierana jest w lewym kwadracie
litera z niższego wiersza wybierana jest z prawego kwadratu
kodowane znaki ustalają przekątną prostokąta. Druga przekątna ustala znaki w szyfrogramie

sytuacje szczególne: dwa znaki w tym samym wierszu
znaki w ostatniej kolumnie

© Piotr Remlein 2004

Szyfry przestawieniowe - transpozycyjne

Przykład Szyfr ADFGVX stosowany przez Niemców w czasie II wojny światowej szyfr przestawieniowy połączony z prostym podstawieniem
w szyfrogramie DA – tekst jawny o, VG – tekst jawny h

	A	D	F	G	V	X
A	c	o	8	x	f	4
D	m	k	3	a	z	9
F	n	w	l	0	j	d
G	5	s	i	y	h	u
V	p	1	v	b	6	r
X	e	q	7	t	2	g

© Piotr Remlein 2004

Szyfr ADFGVX

został złamany przez francuskiego kryptoanalityka George'a Painvina.

Trzeba tu jeszcze dodać, że mimo, iż w nowoczesnych systemach szyfrowania stosuje się przestawienia, to jest to nieco uciążliwe, ponieważ wymaga dużych ilości pamięci, a czasami tekstów jawnych o określonych długościach. Ze względu na to szyfry podstawieniowe są stosowane znacznie częściej od przestawieniowych.

© Piotr Remlein 2004

Szyfry przestawieniowe

Tabela 6.3. Przykład szyfrowania przestawieniowego

Tekst jawny	Tekst zaszyfrowany	Algorytm szyfrowania
ALARM␣DLA␣DYWIZJI	IJZIWYD␣ALD␣MRALA	Tekst czytany wspak
ALARM␣DLA␣DYWIZJI	LARA␣MLD␣AYDIWJZ␣I	Przestawienie dwóch sąsiednich znaków
	R␣JAMADZIL␣LYIADW	Tekst jawny pisany według figury (płot, piła), a tekst zaszyfrowany - jest odczytany wierszami
	AMAWIL␣␣JADDZRLYJ	Tekst jawny wpisany wierszami w prostokąt, a tekst zaszyfrowany jest czytany kolumnami

© Piotr Remlein 2004

Okresowe szyfry przestawieniowe.

Zamiana kolejności znaków przy zastosowaniu stałego okresu.
Zd - zbiór liczb całkowitych od 1 do d.

f :Zd -> Zd -permutacja na zbiorze Zd.

Klucz szyfru: K = (d,f). Kolejne bloki d znaków są szyfrowane przez dokonanie permutacji zgodnie z f.
Tekst jawny : M = m1…md md+1…m2d…
Tekst zaszyfrowany : C = mf(1)…mf(d)md+f(1)…md+f(d)…
PRZYKŁAD
d = 4, f jest następującą permutacją :
i : 1 2 3 4
f(i) : 2 4 1 3
M = RENA ISSA NCE
C = EARN SAIS CNE

© Piotr Remlein 2004

K r y p t o a n a l i z a szyfrów przestawieniowych

Łatwo jest rozpoznać, czy zastosowany szyfr jest szyfrem przestawienowym, ponieważ częstość wystąpień liter w tekście zaszyfrowanym jest taka sama jak częstość wystąpień w tekście jawnym.

Szyfry przestawieniowe łamane są metodą anagramową, polegającą na odtworzeniu właściwej kolejności znaków na podstawie informacji zawartej w tablicach częstości digramów i trigramów.

© Piotr Remlein 2004

Szyfry wieloalfabetowe

Stosują różne podstawienia wieloalfabetowe podczas szyfrowania jednego komunikatu.

Przykład szyfr Vigenere'a

Tekst szyfrujemy na podstawie klucza. Szyfrowanie odbywa się w sposób następujący. Każdą literę tekstu jawnego szyfrujemy korzystając z alfabetu zaczynającego się od odpowiadającej litery w kluczu. W przypadku, gdy klucz jest krótszy od szyfrowanego tekstu powtarzamy go wielokrotnie. Szyfrowanie i deszyfrowanie odbywa się na podstawie tablicy Vigenere`a.
Litera szyfrogramu znajduje się na przecięciu pary liter tekstu jawnego i klucza.

© Piotr Remlein 2004

Szyfr Vigenere'a

Tekst jawny: *algorytmyistrukturydanych*
Klucz: *vigenerevigenerevigenerev*
Tekst zaszyfrowany *vtmseckqtqyxeybxpzehnrpgc*

Jawny *abcdefghijklmnopqrstuvwxyz*
klucz *bcdefghijklmnopqrstuvwxyza*
cdefghijklmnopqrstuvwxyzab
defghijklmnopqrstuvwxyzabc
efghijklmnopqrstuvwxyzabcd
fghijklmnopqrstuvwxyzabcde
ghijklmnopqrstuvwxyzabcdef
hijklmnopqrstuvwxyzabcdefg
ijklmnopqrstuvwxyzabcdefgh
jklmnopqrstuvwxyzabcdefghi
klmnopqrstuvwxyzabcdefghij

© Piotr Remlein 2004

Szyfr Vigenere'a



© Piotr Remlein 2004

Szyfr XOR

Szyfrogram jest wynikiem binarnego sumowania modulo 2 tekstu jawnego z kluczem. Ten sam program może służyć do szyfrowania i do deszyfrowania.

M XOR K = C

C XOR K = M

Operacja ta jest częścią składową wielu rozbudowanych algorytmów kryptograficznych jak np. DES (Data Encryption Standard). Operacja ta sama w sobie stanowi również prosty algorytm szyfrowania, który nie zapewnia jednak większego bezpieczeństwa. Jednak przy spełnieniu kilku bardzo ważnych warunków może stanowić, mimo swej prostoty, algorytm niemożliwy do złamania (binarna wersja algorytmu one-time-pad).

© Piotr Remlein 2004

Szyfr XOR

Dla przypomnienia operacja XOR:
0 XOR 0 = 0
0 XOR 1 = 1
1 XOR 0 = 1
1 XOR 1 = 0

w wyniku podwójnego wykonania operacji XOR otrzymamy tekst jawny.
Zatem:
M XOR K = C
C XOR K = M
Czyli:
(M XOR K) XOR K = M
W wyniku tego mamy tylko jedną procedurę, zarówno do szyfrowania jak i do deszyfrowania.

© Piotr Remlein 2004

Szyfr XOR

Przykład:
Szyfrowanie
Tekst jawny: *0100 1010*
Klucz: *0100 1000*
Szyfrogram: *0000 0010*

Deszyfrowanie
Szyfrogram: *0000 0010*
Klucz: *0100 1000*
Tekst jawny: *0100 1010*

Poziom bezpieczeństwa:
** Dla krótkich kluczy - niski*
** Dla kluczy długich (powyżej 160-bitów) - średni*
** Przy spełnieniu określonych warunków (one-time-pad) - bezwarunkowo bezpieczny*

© Piotr Remlein 2004

Szyfr one-time-pad (jednokrotny z dodatkowymi założeniami)

cała tajemnica algorytmu tkwi w założeniach nałożonych na hasło.
Sprawdzone muszą być wszystkie 3 poniższe warunki:
- hasło musi być ciągiem losowym
***- hasło musi być jednorazowe** (stąd nazwa)*
- długość hasła musi być przynajmniej tak samo długa jak długość szyfrowanego tekstu

© Piotr Remlein 2004

Szyfr one-time-pad (jednokrotny z dodatkowymi założeniami)

Jest to jedyny bezwarunkowo bezpieczny szyfr, co zostało udowodnione matematycznie w 1949 przez Shannon'a. Algorytm ten zaproponowany został przez Gilberta Vernama z AT&T w 1917 roku.
Jeżeli chodzi o pojęcie klucza losowego to pierwszy raz wprowadził je Joseph Mauborgne.
W literaturze można spotkać informacje, że podobno gorąca linia pomiędzy Waszyngtonem a Moskwą szyfrowana była z wykorzystaniem tego algorytmu.

Opis metody: Można wyróżnić 2 wersje tego algorytmu:
-wersja binarna (szyfr Vernama)
-wersja znakowa

W wersji binarnej szyfrujemy/deszyfrujemy korzystając z algorytmu Xor.
W wersji znakowej szyfrujemy/deszyfrujemy korzystając z algorytmu Vigenere`a.

© Piotr Remlein 2004

Szyfr one-time-pad problemy ze stosowaniem

klucz musi być zawczasu uzgodniony przez osoby komunikujące się
klucz musi być wybrany w sposób losowy (technicznie nie jest to łatwe)
klucz musi być przechowywany w bezpieczny sposób
klucz musi być co najmniej tak długi jak szyfrowany tekst

Złamanie choćby jednego z tych warunków powoduje, że otrzymany szyfrogram może być już łatwy do odszyfrowania.

© Piotr Remlein 2004

• Przykład:

tekst jawny	⟹	S	Z	Y	F	R
binarnie	⟹	01010011	01011010	01011001	01000110	01010010
klucz	⟹	01110010	01010101	11011100	10110011	00101011
kryptogram	⟹	00100001	00001111	10000101	11110101	01111001

Algorytm szyfrowania Base64

- jest wykorzystywany podczas kodowania i dekodowania poczty e-mail. Należy on do podzbioru standardu MIME (Multipurpose Internet Mail Extensions).

Algorytm BASE64 bazuje na 64 elementowej tablicy znaków

0 A	17 R	34 i	51 z
1 B	18 S	35 j	52 0
2 C	19 T	36 k	53 1
3 D	20 U	37 l	54 2
4 E	21 V	38 m	55 3
5 F	22 W	39 n	56 4
6 G	23 X	40 o	57 5
7 H	24 Y	41 p	58 6
8 I	25 Z	42 q	59 7
9 J	26 a	43 r	60 8
10 K	27 b	44 s	61 9
11 L	28 c	45 t	62 +
12 M	29 d	46 u	63 /
13 N	30 e	47 v	
14 O	31 f	48 w	=
15 P	32 g	49 x	
16 Q	33 h	50 y	

Proces szyfrowania

- Pobierane są trzy znaki zbioru wejściowego
- Znaki zamieniane są na zapis binarny
- Po zamianie znaków na system dwójkowy uzyskuje się ciąg 24 bitów
- Ciąg ten dzieli się na 4 sześcio-bitowe podciągi
- Podciągi uzupełniane są do długości 8 bitów, poprzez dodanie dwóch najstarszych bitów o wartości 0
- Tak uzyskane 8 bitowe ciągi zamieniamy na znaki, wg wyżej przedstawionej tabeli

Przykład szyfrowania

Tekst jawny: ANUBIS
A - 01000001
N - 01001110
U - 01010101

Po zapisie w jeden ciąg otrzymujemy:

010000010100111001010101

- Dzielimy ciąg na 4 6-bitowe podciągi:

010000 010100 111001 010101

Uzupełniamy do 8 bitów:

00010000 00010100 00111001 00010101

W zapisie dziesiętnym otrzymujemy:

- 00010000 - 16
00010100 - 20
00111001 - 57
00010101 - 21

Odczytujemy z tabeli odpowiedniki i zapisujemy:

QU5V

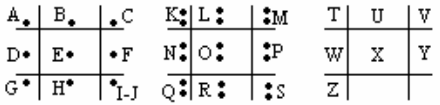
- Tą samą czynność powtarzamy dla pozostałych 3 bajtów słowa (czyli: BIS).

Odkodowanie jest operacją odwrotną (jest to kolejny szyfr symetryczny).

- Szyfr nie zapewnia bezpieczeństwa. Dlaczego jest więc wykorzystywany w poczcie E-mail? Otóż jak można zauważyć obserwując tablicę BASE64 szyfr bazuje jedynie na 64 DRUKOWALNYCH znakach - oznacza to, że nawet pliki binarne są przekształcane do postaci drukowalnej - a to gwarantuje, że dołączany do poczty zbiór nie będzie posiadał znaków zarezerwowanych dla protokołu SMTP (znaków sterujących).

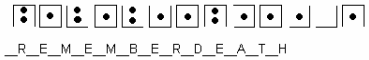
Szyfr cmentarny

- zalicza się do szyfrów podstawieniowych
- Niech A jest zbiorem liter nad alfabetem angielskim, S - zbiorem symboli przedstawionych poniżej:



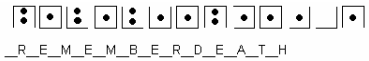
Szyfrem cmentarnym nazywamy funkcję: $F_c : A \to S$.

- Jeden z tekstów zaszyfrowanych przy użyciu szyfru cmentarnego znajduje się na cmentarzu kościoła Świętej Trójcy w Nowym Jorku, datowany na 1794 r., drugi na cmentarzu nowojorskiego kościoła św. Pawła - 1796 r. Wygrawerowany szyfrogram wzięty z płyty grobowca z pierwszego z wymienionych cmentarzy ma postać następującą:



_R_E_M_E_M_B_E_R_D_E_A_T_H





_R_E_M_E_M_B_E_R_D_E_A_T_H

Szyfr Bacona

- **Szyfr Bacona** jest szyfrem, w którym tekst zaszyfrowany zawiera pięcioliterowe ciągi złożone z liter a i b. Szyfrowanie i odszyfrowanie przebiega według schematu:

A = aaaaa
B = aaaab
C = aaaba
D = aaabb
E = aabaa
F = aabab
G = aabba
H = aabbb
I/J= abaaa
K = abaab
L = ababa
M = ababb
N = abbaa
O = abbab
P = abbba
Q = abbbb
R = baaaa
S = baaab
T = baaba
U/V= baabb
W = babaa
X = babab
Y = babba
Z = babbb

Blokowy szyfr przestawieniowy

- Alfabet 32 znaki

ABCDEFGHIJKLMNOPQRSTUVWXYZ . : , ; -

- Tekst jawny składa się z 48 znaków rozmieszczonych w 6 wierszach i 8 kolumnach

- Utworzyć tablicę o 6 wierszach i 8 kolumnach, wypełnioną losowo liczbami 0, 1, 2 i 3, w której liczba zer, jedynek, dwójek i trójek jest taka sama i wynosi 12.

- Wyciąć cztery prostokąty z papieru, o kształcie tablicy z kroku 1 i utworzyć z nich cztery szablony o numerach 0, 1, 2 i 3.
- W każdym szablonie należy wyciąć 12 otworów w miejscach komórek, zawierających liczbę, oznaczającą numer szablonu.

- Przygotować czysty kawałek papieru o wymiarze szablonu, na którym będzie zapisany kryptogram.
  Następnie do tego kawałka papieru przykładać kolejno przygotowane w kroku 2 szablony, wpisując do wyciętych w szablonach
  okienek 48 liter tekstu jawnego
  (4 razy po 12 liter).

Tablica do konstrukcji szablonów szyfrujących

0	1	1	2	3	3	0	3
1	3	2	2	3	0	0	3
3	0	3	2	0	1	0	3
2	1	0	2	1	2	3	2
1	2	1	1	0	3	1	2
2	1	0	1	0	0	2	3

Permutacja szyfrująca

1	13	14	25	37	38	2	39
15	40	26	27	41	3	4	42
43	5	44	28	6	16	7	45
29	17	8	30	18	31	46	32
19	33	20	21	9	47	22	34
35	23	10	24	11	12	36	48

Blok tekstu jawnego

A	D	I	U	N	K	T	-
L	E	C	T	U	R	E	R
A	F	O	R	Y	Z	M	-
A	P	H	O	R	I	S	M
A	K	T	O	R	K	A	-
A	C	T	R	E	S	S	.

Blok szyfrogramu

A	U	R	A	R	K	D	A
E	-	P	H	A	I	U	C
T	N	R	O	K	R	T	E
R	A	-	I	F	S	S	M
O	A	R	Y	L	S	Z	K
T	M	E	-	C	T	O	.

Przykład blokowego szyfru podstawieniowego

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
X	H	Y	U	Z	R	.	V	Q	A	:	W	T	,	B	C

Q	R	S	T	U	V	W	X	Y	Z		.	:	,	;	-
S	D	;	E	I	G	L	F	J	K	-	M	P	N	0	

X	U	Q	I	,	:	E	
W	Z	Y	E	I	D	Z	D
X	R	B	D	J	K	T	
X	C	V	B	D	Q	;	T
X	:	E	B	D	:	X	
X	Y	E	D	Z	;	;	M

Przykład szyfru produktowego

- Szyfrem produktowym jest szyfr, który stosuje po kolei dwa algorytmy szyfrujące: najpierw szyfruje się tekst jawny za pomocą algorytmu pierwszego,
  a następnie otrzymany kryptogram szyfruje się, stosując algorytm drugi. W efekcie otrzymuje się kryptogram wynikowy szyfru produktowego.

X	I	D	X	D	:	U	X
Z		C	V	X	Q	I	Y
E	,	D	B	:	D	E	Z
D	X		Q	Q	;	;	T
B	X	D	J	W	;	K	:
E	T	Z		Y	E	B	M

- Jakkolwiek składnikami przedstawionego algorytmu szyfrowania produktowego są dwa słabe szyfry, to złamanie tego szyfru produktowego, nawet dla zaawansowanego kryptoanalityka, nie będzie zadaniem trywialnym.
- Właśnie takie naprzemienne stosowanie szyfrów przestawieniowych i podstawieniowych, prowadzi, zgodnie ze wskazówkami Shannona, do syntezy odpornych do złamania szyfrów. Te nieco zmodyfikowaną zasadę stosuje wiele praktycznie używanych systemów kryptograficznych dla szyfrowania blokowego, operujących na alfabecie dwuelementowym, m. in. algorytmy DES i IDEA.

Bezpieczeństwo danych i przykłady kryptoanalizy prostych szyfrów

Podstawowe zasady kryptografii

1. Nie wolno lekceważyć przeciwnika.
2. Nie wolno opierać bezpieczeństwa systemu kryptograficznego na tajności algorytmu.
3. Nie należy poprawiać skutecznie działających systemów kryptograficznych.
4. Bezpieczeństwo danego kryptosystemu może kompetentnie ocenić tylko kryptoanalityk.
5. Należy zawsze brać pod uwagę możliwość popełnienia błędów kryptograficznych lub naruszenia dyscypliny szyfrowania.
6. Prawidłowe stosowanie systemu kryptograficznego nie zwalnia ze stosowania innych zabezpieczeń.

Sens większości powyższych zasad nie wymaga dodatkowego wyjaśniania. Zauważmy jedynie, że zasada 4. przeczy naiwnemu przekonaniu, że kryptosystem jest tym bezpieczniejszy, im bogatsza jest przestrzeń kluczy $\mathcal{K}$. W rzeczywistości, duża ilość kluczy utrudnia złamanie szyfru tylko w przypadku ataku polegającego na wyczerpującym sprawdzaniu wszystkich możliwych kluczy. Jednak w większości przypadków znane są inne — skuteczniejsze — metody ataku kryptoanalitycznego.

Błędy szyfrowania

Najczęstsze błędy popełniane podczas szyfrowania

- Równoczesna transmisja tekstu tajnego i jawnego (tzw. *kompromitacja typu: tekst jawny–kryptogram*).
- Użycie tego samego klucza do zaszyfrowania dwóch różnych tekstów jawnych (tzw. *kompromitacja typu: tekst jawny–tekst jawny*).
- Użycie dwóch różnych kluczy do zaszyfrowania tego samego tekstu jawnego (tzw. *kompromitacja typu: kryptogram–kryptogramy*).

- Powtarzanie stereotypowych zwrotów w szyfrowanym tekście.
- Używanie zbyt krótkich haseł i kluczy.
- Używanie haseł i kluczy, które łatwo odgadnąć.
- Używanie znaków interpunkcyjnych, a w szczególności znaku spacji.
- Poprawność ortograficzna i stylistyczna.

Najczęstszymi błędami popełnianymi w codziennym stosowaniu kryptografii są: ustalanie przewidywalnych kluczy i haseł, wykorzystywanie kluczy i haseł, które nie są losowe oraz zbyt mała ich długość.

Celem kryptoanalizy jest zdobycie jak najpełniejszej informacji o tekście jawnym bez znajomości tajnego klucza

R. Wobst

Typy ataku kryptoanalitycznego

- Kryptoanalityk dysponuje pewnymi danymi, które stara się wykorzystać do złamania szyfru. Znając algorytm kryptograficzny, a nie znając klucza, próbuje na podstawie tych danych znaleźć klucz (lub odczytać wiadomość bez wyznaczania klucza).

- atak siłowy (zwany metodą brute force) polegający na przejrzeniu listy wszystkich możliwych kluczy.
- Metoda ta jest niewydajna, a w przypadku współczesnych kryptosystemów, na ogół zawodna.

- Doświadczony kryptoanalityk próbuje przede wszystkim innych metod kryptoanalitycznych, dobranych pod kątem użytego algorytmu. Jeżeli nawet te metody nie przyniosą rezultatu w postaci odszyfrowania kryptogramu, to na ogół pozwalają na redukcję liczby możliwych kluczy i, w konsekwencji, umożliwią skuteczny atak siłowy.

- Atak tylko z tekstem zaszyfrowanym.
- Jest to najtrudniejsza sytuacja.

- Atak ze znanym tekstem jawnym.
- Kryptoanalityk dysponuje parą tekst jawny – tekst tajny.

- Atak z wybranym tekstem jawnym.
- Kryptoanalitykowi udało się spowodować, aby został zaszyfrowany wybrany przez niego tekst jawny i teraz dysponuje parą takich tekstów.

- Atak z wybranym tekstem tajnym.
- Kryptoanalitykowi udało się uzyskać dostęp do aparatury deszyfrującej i przy jej pomocy odszyfrować wybrany przez siebie tekst tajny.

Analiza częstości

- Analiza częstości jest jedną z najczęściej stosowanych (w wielu odmianach) metod kryptoanalitycznych. Dzięki niej można złamać w krótkim czasie szyfry monoalfabetyczne.
  Pewne jej modyfikacje są wykorzystywane również do łamania szyfrów polialfabetycznych (np. szyfru Vigenere'a z krótkim kluczem) oraz szyfrów monoalfabetycznych z homofonami.

- częstotliwość występowania spacji jest co najmniej dwukrotnie większa od częstotliwości występowania najczęstszej samogłoski. Podczas szyfrowania należy więc opuszczać znaki spacji, aby podział tekstu na pojedyncze słowa nie ułatwił procesu łamania szyfru.

- Najczęściej wnioskujemy 3 lub 4 znaki i dokonujemy ich zastąpienia w tekście zaszyfrowanym. Czytając taki zmodyfikowany tekst zaszyfrowany próbujemy znaleźć słowa, gdzie brakuje niewielu liter i na podstawie dedukcji odkrywamy kolejne odpowiedniki znaków.

Cele kryptoanalizy

Największym sukcesem jest znalezienie klucza.

Umożliwia bezproblemowe odczytywanie wiadomości – niestety aż do zmiany klucza ☹

Odczytanie tekstu jawnego

Jeżeli czas ataku jest akceptowalny (metoda szyfrowania słaba lub podatna na ataki) to powinno nas zadowolić.

Badane własności

- Własność komplementarności
- Kolizje kluczy
- Klucze słabe
- Klucze półsłabe
- Punkty stałe
- Efekt lawinowy
- Podatność na znane ataki kryptograficzne

Techniki

- Analiza statystyczna
- Kryptoanaliza różnicowa
- Kryptoanaliza liniowa
- Analiza różnicowa mocy
- Analiza z pomiarem czasu wykonywania operacji

Kryptoanaliza różnicowa

- Opublikowana w 1990 przez E. Bihama i A. Shamira.
- Atak typu chosen plaintext.
- DES z 16-rundami wymaga około
  2^{47} par tekst jawny-kryptogram (chosen plaintext)
  2^{55} par przy ataku typu known plaintext

E. Biham, A. Shamir.
Differential cryptanalysis of DES-like Cryptosystems.
Journal of Cryptology, Vol.4 No.1, pages 3-72, 1991

Kryptoanaliza różnicowa

Rozważane są pary danych wejściowych, oznaczymy je przez X i Y
Zakładamy **X XOR K** i **Y XOR K**

(X XOR K) XOR (Y XOR K) = X XOR Y

Kryptoanaliza liniowa

Wykorzystuje aproksymację liniową do opisania operacji zachodzących w algorytmie blokowym

Aproksymacja liniowa w ciele zero-jedynkowym ????

Aproksymacja probabilistyczna – wiemy że zależności:

$m_2 \oplus m_3 \oplus m_7 = c_5 \oplus c_7$
$m_4 \oplus m_6 \oplus m_7 = c_2 \oplus c_5$

zachodzą z prawdopodobieństwem 90%

Ogólnie w kryptoanalizie liniowej wykorzystuje się fakt, że pewne zależności liniowe występują z prawdopodobieństwem **RÓŻNYM** od 50 %

Analiza różnicowa mocy

Atak stosowany głównie wobec kart mikroprocesorowych

Wykorzystujemy wahania poboru mocy w zależności od wykonywanych operacji

SPA – Simple Power Attack

DPA – Differential Power Analysis – statystyczna ocena wpływu zmiany jednego bitu na kształt strumienia danych + analiza np. promieniowania elektromagnetycznego

Odkrywca SPA – Kocher 1995
Odkrywca DPA – Kocher, Jaff, June

Analiza z pomiarem czasu wykonywania operacji

W przypadku pewnych operacji czas wykonania zależy od danych wejściowych – deszyfrowanie RSA wymaga obliczenia $R = c^d \bmod n$

Atak przeciwko implementacji nie algorytmowi !!!

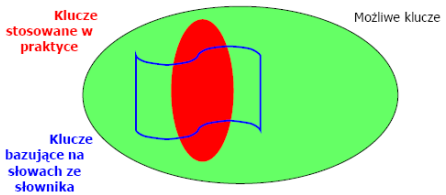
Postępowanie kryptoanalityka

- Informacje o tekście jawnym
- Przestrzeń możliwych kluczy
- Poszukiwanie niedozwolonych znaków lub bigramów
- Atak słownikowy
- Analiza statystyczna

- Nie istnieje ogólna definicja jak kryptoanalityk powinien postępować

- Struktura wiadomości –
  Jakiego rodzaju może być tekst jawny ?
  - Tekst naturalny
    - W jakim języku ?
    - Jak zapisany ?
  - Plik maszynowy
    - Jaki format zapisu ?
    - Dodatkowe cechy charakterystyczne dla formy zapisu
    - Plik skompresowany – znowu specyficzne właściwości

- Jaka jest przestrzeń możliwych kluczy
  - Jeżeli hasło ma 6 znaków to przestrzeń kluczy jest wielkości ok. 308'915'776 (26^6)



Kryptoanaliza szyfrów przestawieniowych

Tekst jawny:
PRZEKAZPIENIEDZY TAKUZGODNILISMYJ UTRO

Szyfr:
PZEKNYRPDUIJZIZZ LUEEYGITKNJOSRAI ADMO

P	R	Z	E	K	A
Z	P	I	E	N	I
E	D	Z	Y	J	A
K	U	Z	G	O	D
N	I	L	I	S	M
Y	J	U	T	R	O

Łamanie z tekstem jawnym

Załóżmy, że tekst jawny zawiera słowo PIENIEDZY.

Gdyby długość bloku, czyli liczba kolumn była równa 3 to w szyfrogramie musiałby istnieć ciąg II (w tabeli oba te znaki leżałyby jeden nad drugim).

PIE
NIE
DZY

Tak nie jest, więc 3 nie jest długością bloku.

Poprzez kolejne takie próby wyznaczamy rzeczywistą długość bloku, czyli w tym przykładzie 6 (ponieważ w krytpogramie istnieją ciągi: PD, IZ, EY)

PZEKNYRPDUIJZIZZLUEEYGITKNJOSR AIADMO

Po znalezieniu długości można wnioskować o ewentualnych przestawieniach (transformacjach) kolumn, tu takowych nie ma.

Łamanie ze znajomością wyłącznie szyfrogramu

Każdy język ma określoną strukturę, występujące po sobie litery zależą od siebie w określony sposób.

Dostatecznie długie kryptogramy można przetestować pod kątem występowania po sobie określonych par liter (rozrzuconych po operacji szyfrowania po całym szyfrogramie)

Łamanie ze znajomością wyłącznie szyfrogramu

Długość grupy znajdujemy przez wyznaczenie koincydencji znaków.

Cóż to takiego?

(William Friedman w 1920 roku)

Indeks koincydencji określa prawdopodobieństwo wystąpienia w szyfrogramie dwóch jednakowych liter, można go wyliczyć ze wzoru:

$$IC = \frac{\sum_{a \in A} F_a (F_a - 1)}{n(n-1)}$$

- F z indeksem a jest liczbą wystąpień litery a alfabetu A w tekście o długości n

Inna definicja IC

IC= liczba zgadzających się znaków/ liczba wszystkich znaków w tekście

Np.:

Oto_pierwsz**y**_tekst IC=2/18

A_tutaj mam**y**_drugi

Pierwszy_teks**t** Oto_pierwszy_**t** IC=1/14

Obliczamy IC dla różnych wartości przesunięć szyfrogramu.

Wartość IC dla tekstu przesuniętego o wielokrotność okresu T klucza będzie wyraźnie większa od IC obliczonych dla innych przesunięć

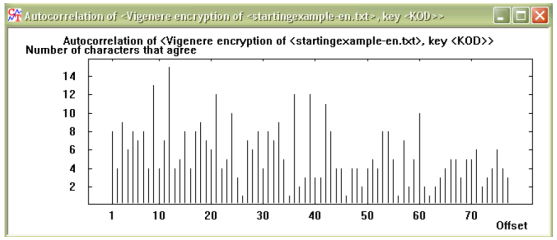
IC jest takie samo dla szyfrogramu jak i tekstu jawnego

- Zgodnie ze wzorem obliczamy liczbę wystąpień wszystkich znaków, zliczamy długość tekstu i podstawiamy do wzoru, otrzymując wartość indeksu koincydencji (IC). Następnie korzystając z tabeli określamy długość zastosowanego hasła.

Długość	IC
1	0,066
2	0,052
3	0,047
4	0,045
5	0,044
10	0,041
bardzo duże	0,038

- Przykładowo, gdy otrzymamy wartość indeksu zbliżoną do 0,06 możemy przypuszczać, że długość hasła (ilość zastosowanych alfabetów) wynosi 1 czyli zastosowano prosty szyfr monoalfabetyczny.
- W literaturze możemy znaleźć również inną postać tej metody. Polega ona na zliczaniu ilości znaków pokrywających się w obu tekstach. Teksty przesuwamy względem siebie o kolejne pozycje i zliczamy liczbę pokrywających się znaków. Po przesunięciu tekstów o wielokrotną długość klucza, uzyskujemy największą liczbę pokrywających się znaków. Dla dłuższych tekstów ilość pokrywających się znaków powinna wynosić w przypadku języka angielskiego około 6,5-6,9%.

- wykres przedstawiający ilości pokrywających się znaków w stosunku do przesunięcia tekstów względem siebie. Dwa zdania z pewnego artykułu zostały zaszyfrowane algorytmem Vigenera z kluczem KOD.
- wartości największe są dla przesunięcia będącego wielokrotnością liczby 3.



Test Kasiskiego

sugeruje długość klucza na podstawie analizy powtórzeń znaków w kryptogramie.

W każdym języku mamy często powtarzające się powtórzenia kilku znaków, np. –enie, -anie, ówki.

Jeśli dwa takie same ciągi znaków znajdują się w tekście jawnym w odstępie wynoszącym wielokrotność okresu klucza, to na odpowiednich miejscach w kryptogramie uzyskujemy identyczne grupy znaków.

- Chcąc skorzystać z metody Kasiskiego w pierwszej kolejności wyszukujemy w tekście zaszyfrowanym powtarzające się ciągi znaków a następnie analizujemy odstępy między takimi samymi ciągami.
- Przykładowo w tekście znaleźliśmy 7 wystąpień trójki *fad* a odstępy między nimi wynoszą odpowiednio 24, 12, 20, 16, 35, 44 i 92. Możemy zauważyć, że sześć z siedmiu wartości, reprezentujących długość odstępu, podzielnych jest przez 4 -- zatem możemy przyjąć, że długość klucza wynosi 4 lub 2. Wartość 35 mogła być związana z przypadkowym zaszyfrowaniem innego tekstu z innymi przesunięciami i w ten sposób otrzymaliśmy ten sam tekst zaszyfrowany.

wg metody Kasiskiego

szukamy powtórzeń bloków co najmniej trzech liter w szyfrogramie i obliczamy odstępy między nimi.

Jeśli np. otrzymamy odstępy: 24, 54, 18, 29, 66, to możemy przypuszczać, że okres klucza wynosi prawdopodobnie 6 (ewentualnie 2 lub 3). Wartość 29 można uznać za błąd w pomiarach.

Kryptoanaliza szyfrów monoalfabetycznych

Jedno podstawienie litera tekstu jawnego – litera szyfrogramu.

Rozkład częstości występowania liter zostaje zachowany

alfabet jawny	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
0	A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
2	C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
3	D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
4	E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
5	F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
6	G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
7	H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
8	I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
9	J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
10	K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
11	L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
12	M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
13	N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
14	O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
15	P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
16	Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
17	R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
18	S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
19	T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
20	U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
21	V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
22	W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
23	X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
24	Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
25	Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

- Znajdując powtarzające się kilkuliterowe fragmenty w tekście zaszyfrowanym możemy obliczyć odległości między nimi. Odległości te są wielokrotnościami długości klucza. Obliczając ich wspólne dzielniki możemy odgadnąć jaką długość ma klucz.

Kryptoanaliza szyfrów homofonicznych

litera tekstu jawnego – zastępowana jest jedną z kilku możliwych liter

Szyfry te mogą być znacznie trudniejsze do złamania niż zwykłe szyfry podstawieniowe, zwłaszcza gdy liczba homofonów przydzielona danej literze jest proporcjonalna do częstości jej występowania. Rozkład częstości liter w szyfrogramie może być niemal jednostajny.

Ten sposób szyfrowania nie zaciera jednak regularności struktury języka.

- Siła tego algorytmu polega na tym, że do kodowania kolejnych liter wybieramy różne alfabety szyfrowe
- dzięki temu nawet jeśli jakaś litera występuje w tekście jawnym często, to jest ona kodowana na różne litery i nie możemy już zastosować klasycznego podejścia z użyciem tablicy częstości.
- Jeśli w tekście jawnym występują pary powtarzających się liter (takie jak **ss**, **ee**), to obydwie litery są kodowane inaczej i też nie możemy tego wykorzystać.
- Trudno zaatakować ten szyfr stosując sprawdzenie wszystkich możliwości, gdyż liczba możliwych kluczy wynosi 26^*k*, gdzie *k* jest długością klucza.

- Znając długość klucza możemy zastosować metodę częstości występowania poszczególnych liter dla znaków szyfrogramu powstałych poprzez przesunięcie o taką samą ilość miejsc
- przykładowo dla hasła o długości 3 te same przesunięcie będą miały znaki 1,4,7,...; 2,5,8,...; 3,6,9,....

Szyfr Vigenere'a

klucz	A	G	E	N	T	A	G	E	N	T	A	G	E	N	T	A
tekst jawny	i	a	m	b	o	n	d	j	a	m	e	s	b	o	n	d
tekst zaszyfrowany	I	G	Q	O	H	N	J	N	N	F	E	Y	F	B	G	D

Szyfry polialfabetyczne rozkład częstości występowania znaków ukrywają za pomocą wielu podstawień.

Kryptoanaliza szyfru Vigenere'a

- To samo słowo szyfrowane jest przy pomocy innego fragmentu klucza

(bond) → NTAG

(bond) → ENTA

Jednak gdy występuje bardzo często, to zwiększa się prawdopodobieństwo zakodowania kilka razy przy pomocy tego samego fragmentu klucza, a tym samym uzyskamy powtarzające się ciągi znaków w szyfrogramie.

Regularności w tekście zaszyfrowanym właśnie poszukuje kryptoanalityk.

- Zakładając długość klucza - k

Budujemy tablicę częstości liter występujących w tekście zaszyfrowanym na tych samych pozycjach modulo-k i porównujemy ze znaną tablicą częstości liter próbując odgadnąć kolejną literę klucza.

Nie zawsze otrzymana tabela częstości liter musi być całkowicie zgodna ze znaną statystyką.

- Można dokonać wstępnej analizy częściowo odczytanego tekstu poszukując najczęstszych związków liter np. the
- Czasem należy wykonać wiele prób zanim znajdzie się właściwą literę klucza.
- Można podzielić odszyfrowany tekst na fragmenty, tak by stanowiły one słowa należy dysponować zbiorem słów

Poszukiwanie negatywnego wzorca

```
  poka zuje slow oprz ekaz srod kowj utro
  srnd cxmh vorz rsuc hndc vurg nrzm xwur
  prze kazs rodk ow
*  prz ekaz srod kow
    pr zeka zsro dkow
*    p rzek azsr odko w
       prze kazs rodk ow
        prz ekaz srod kow
         pr zeka zsro dkow
*          p rzek azsr odko w
             prze kazs rodk ow
              prz ekaz srod kow
*              pr zeka zsro dkow
*               p rzek azsr odko w
*                  prze kazs rodk ow
                    prz ekaz srod kow
*                    pr zeka zsro dkow
```

Poszukiwanie negatywnego wzorca

Teoretycznie w 46% wypadków nie pokryją się dwie takie same litery.

U nas w 9 na 19 przypadków nastąpiło pokrycie – 47%, czyli statystyka nie zawodzi ☺☺☺

Dobry algorytm szyfrujący

Czy taki istnieje ???

1. Mieszanie i rozpraszanie (*confusion and diffusion*)
2. Odporny na atak brute-force (długość klucza)
3. *„Podobne”* teksty nie powinny generować *„podobnych”* kryptogramów
4. Kryptogram powinien nie odróżniać się od ciągu losowego
5. Brak cykli dla stałych, cyklicznych tekstów jawnych
6. Efekt lawinowy dla alg. Blokowych – 50% prawd. Zmian
7. Odporny na atak znanym lub wybranym tekstem jawnym
8. Odporny na kryptoanalizę liniową i różnicową
9. Łatwo wyznaczalne, o ile istnieją, słabe klucze
10. Odporny na ataki algebraiczne – brak zależności liniowej **M** <-> **C**

Szyfry kaskadowe

Szyfrem kaskadowym nazywamy szyfr, który jest złożeniem funkcji szyfrujących. W stosowanych w praktyce szyfrach kaskadowych jako funkcje składowe najczęściej stosowane są podstawienia za grupy bitów i permutacje bitów.

Nowoczesna koncepcja szyfru kaskadowego oparta jest na pojęciu tzw. przekształcenia mieszającego zaproponowanego przez Shannona w roku 1949.

© Piotr Remlein 2004

- ***Pierwszą praktyczną realizacją szyfru kaskadowego opartego na podstawieniach i permutacjach została zaproponowana przez Horsta Feistela w pierwszej połowie lat siedemdziesiątych. (Istniały także inne, wcześniejsze realizacje, jak np. szyfr ADFGVX) jednak propozycja Feistela była pierwszą wykorzystującą - obecnie bardzo szeroko stosowane - podstawienia i permutacje.)***
- ***Propozycja ta nosi nazwę sieci podstawieniowo-permutacyjnej (SPP).***

Szyfry kaskadowe

Od czasu pojawienia się propozycji Feistela powstało wiele innych szyfrów kaskadowych zawierających podstawienia i permutacje. Spośród najbardziej znanych wymienić można algorytmy DES, Lucifer, LOKI, FEAL.

© Piotr Remlein 2004

- Należy podkreślić, że wszystkie te szyfry wywodzą się z algorytmu zaproponowanego przez Feistela i rezultaty badań przeprowadzonych na algorytmie Feistela w bardzo wielu przypadkach dotyczą również innych szyfrów z podstawieniami i permutacjami.

Sieć podstawieniowo-permutacyjna.

Na Rys. przedstawiona jest N-bitowa sieć podstawieniowo permutacyjna składająca się z R rund tzw. skrzynek podstawieniowych połączonych permutacjami bitów. Skrzynki podstawieniowe są różnowartościowymi odwzorowaniami a permutacje bitów należą do specjalnego zbioru permutacji, dla których żadne dwa wyjścia skrzynki podstawieniowej nie są połączone z jedną skrzynką podstawieniową następnej rundy. Tekst jawny i zaszyfrowany są N-bitowymi ciągami (blokami).

N - liczba bitów wejścia (wyjścia) sieci podstawieniowo-permutacyjnej

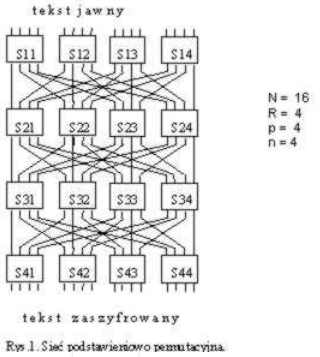
R - liczba iteracji (rund) sieci,

n - liczba skrzynek podstawieniowych w jednej iteracji

p - liczba wejść (wyjść) skrzynki podstawieniowej

© Piotr Remlein 2004

Sieć podstawieniowo-permutacyjna.



Rys.1. Sieć podstawieniowo permutacyjna.

© Piotr Remlein 2004

Sieć podstawieniowo-permutacyjna.

W sieciach podstwieniowo-permutacyjnej stosuje się kluczowanie podstawień.

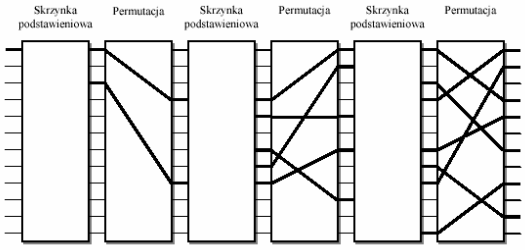
Najczęściej spotykane jest kluczowanie XOR, co oznacza, że bity klucza są dodawane bitowo modulo 2 z bitami sieci przed wejściem do skrzynek podstawieniowych.

Chociaż wykorzystywane niezależnie skrzynki podstawieniowe i permutacje bitów nie dają kryptograficznie silnych przekształceń, to jednak ich kombinacja daje szyfr zapewniający wysoki poziom bezpieczeństwa.

Dzieje się tak dlatego, iż dla takiej konstrukcji wszystkie bity wyjściowe są złożonymi funkcjami wszystkich bitów wejściowych. W związku z tym nawet niewielka zmiana na wejściu szyfru powoduje lawinę zmian w kolejnych iteracjach, aż do uzyskania stosunkowo dużej liczby zmian na wyjściu szyfru

© Piotr Remlein 2004

Rozprzestrzenianie się zmian w sieci Feistel'a



© Piotr Remlein 2004

Sieć podstawieniowo-permutacyjna.

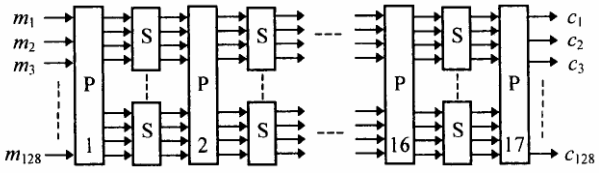
w praktyce wszystkie szyfry kaskadowe konstruowane są na bazie pomysłu Horsta Feistela polegającego na nadaniu algorytmowi takiej struktury, aby przy jego pomocy możliwe było zarówno szyfrowanie, jak i deszyfrowanie (ten sam algorytm).

© Piotr Remlein 2004

Algorytm Lucifer

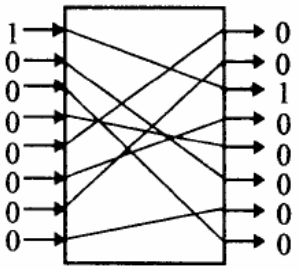
- Został zaprojektowany na początku lat 70-tych XX wieku w firmie IBM. Szyfr składa się z wykonywanych na przemian podstawień i permutacji. Urządzenia realizujące te operacje nazwano skrzynkami podstawień S (ang. S-box) i skrzynkami permutacji P.

Schemat algorytmu Lucifer

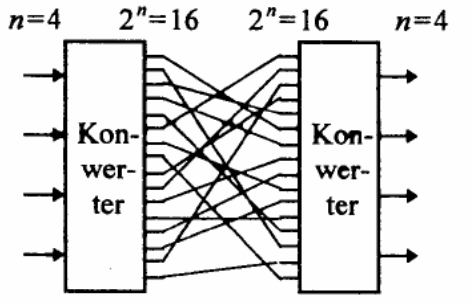


Skrzynka permutacji ma 128 wejść i tyle samo wyjść i służy do zmiany kolejności bitów.

Uproszczona skrzynka permutacji o 8 wejściach



Skrzynka podstawień zawiera dwa układy, zamieniające liczbę n-bitową w liczbę 2^n -bitową i na odwrót,



- Istnieje możliwość zmiany połączeń między tymi układami. Układy wykonują przekształcenia nieliniowe, wskutek czego liczby jedynek i zer są rożne na wejściu i wyjściu skrzynki podstawień.

- W algorytmie Lucifer ustalono dwa rodzaje połączeń, oznaczone przez 0 i 1, które można wybierać kluczem zewnętrznym. Całkowita liczba skrzynek S w układzie wynosi 512. Skrzynkami steruje się za pomocą 128-bitowego klucza. Do zmiany ciągu klucza (128-bitowego) na ciąg 512-bitowy służy specjalny algorytm.
- Metody zastosowane w algorytmie Lucifer posłużyły do stworzenia algorytmu DES, który na wiele lat wyznaczył standard szyfrowania.

Algorytm Data Encryption Standard (DES)

W algorytmie DES dane szyfrowane są w 64-bitowych blokach, z wykorzystaniem
56-bitowego klucza (hasła).

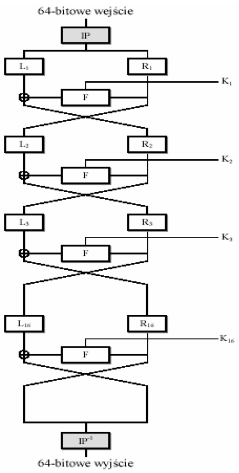
Ogólnie można przyjąć, że szyfrowanie tekstu jawnego (otwartego) przebiega w trzech etapach.

Pierwszy etap polega na przejściu 64-bitowego bloku tekstu jawnego M przez wstępną permutację IP, która przestawia bity tworząc permutowane dane wejściowe M0 = IP(M) dla drugiego etapu.

Drugi etap obejmuje 16 iteracji tej samej funkcji F, w skład której wchodzą podstawienia i permutacje.

Wynik 16 iteracji funkcji F składający się z 64-bitów poddawany jest permutacji IP-1, która jest permutacją odwrotną do permutacji IP i jest to trzeci etap, po którym otrzymujemy 64-bitowy tekst zaszyfrowany.

© Piotr Remlein 2004



Algorytm Data Encryption Standard (DES)

porządek bitów w blokach wejściowych dla IP i IP-1

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64

tablica permutacji wstępnej M0 = IP(M):

Permutacja IP

58	50	42	34	26	18	10	2	60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6	64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1	59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5	63	55	47	39	31	23	15	7

© Piotr Remlein 2004

permutacja IP-1

40 8 48 16 56 24 64 32 39 7 47 15 55 23 63 31
38 6 46 14 54 22 62 30 37 5 45 13 53 21 61 29
36 4 44 12 52 20 60 28 35 3 43 11 51 19 59 27
34 2 42 10 50 18 58 26 33 1 41 9 49 17 57 25

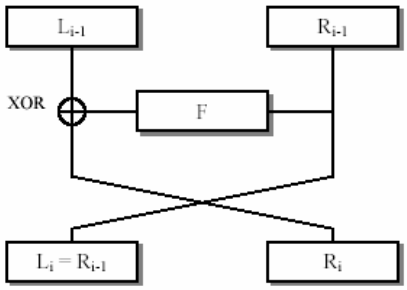
Permutacja odwrotna IP-1 przywróci oryginalną kolejność bitów. Operacja ta jest konieczna do tego, aby algorytm mógł być wykorzystywany zarówno do szyfrowania, jak i deszyfrowania.

Pomiędzy permutacjami IP i IP-1 w algorytmie wykonywanych jest 16 iteracji funkcji F łączącej podstawienia i permutacje.

© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

Pojedyncza iteracja



© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

Pojedyncza iteracja

Proces przetwarzania podczas każdej iteracji rozpoczyna się od podzielenia 64-bitowego bloku wejściowego na dwie połówki: lewą Li-1 i prawą Ri-1. Jeżeli przyjmiemy, że Li i Ri będą odpowiednio określały lewą i prawą połowę rezultatu i-tej iteracji Xi = LiRi, to

możemy zapisać:

Li = Ri-1

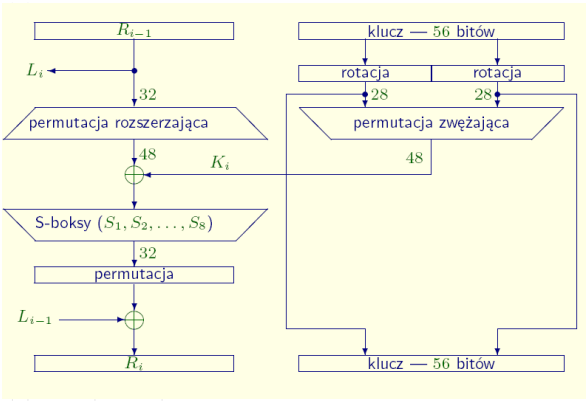
Ri = Li-1 XOR F(Ri-1, Ki)

gdzie Ki 48-bitowy klucz iteracyjny, wyprowadzony z 56-bitowego klucza głównego w algorytmie generowania kluczy iteracyjnych

(algorytm ten przedstawiony jest w dalszej części),

natomiast F jest funkcją szyfrującą występującą w każdej iteracji.

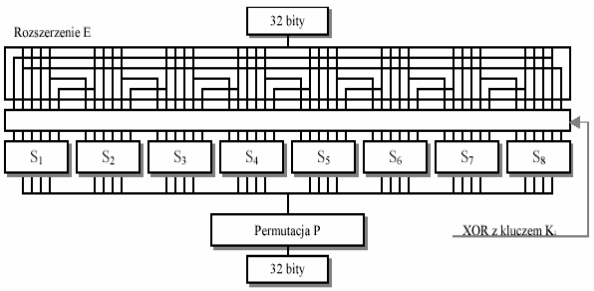
© Piotr Remlein 2004



© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

Funkcja F



© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

Funkcja F

Wejściem do funkcji F jest prawa 32-bitowa połówka Ri-1 = r1, r2, r3, ..., r32 , gdzie ri jest i-tym bitem Ri-1. Jest ona rozszerzana do 48-bitowego bloku zgodnie z tablicą wyboru bitów E.

Rozszerzenie E

r32 r1 r2 r3 r4 r5 r4 r5 r6 r7 r8 r9
r8 r9 r10 r11 r12 r13 r12 r13 r14 r15 r16 r17
r16 r17 r18 r19 r20 r21 r20 r21 r22 r23 r24 r25
r24 r25 r26 r27 r28 r29 r28 r29 r30 r31 r32 r1

Rozszerzenie to zapiszemy :

E(Ri-1) = r32, r1, r2, r3, r4, r5, r4, ..., r32, r1

Tablica ta używana jest prawie identycznie jak tablice permutacji. Różnica polega na tym, że niektóre bity Ri-1 wybierane są więcej niż raz.

© Piotr Remlein 2004

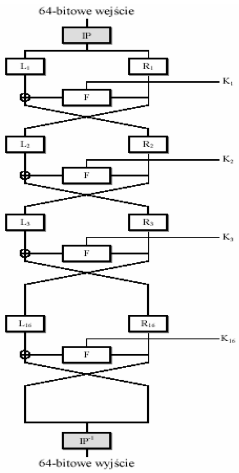
Algorytm Data Encryption Standard (DES)
Funkcja F

W następnym kroku wynik rozszerzenia E(Ri-1) poddawany jest operacji XOR z kluczem iteracyjnym Ki, co możemy zapisać E(Ri-1) XOR Ki, a jego wynik dzielony jest na osiem 6-bitowych bloków B1, B2, ..., B8 :

E(Ri-1) XOR Ki = B1, B2, B3, B4, B5, B6, B7, B8

Każdy blok Bj jest wejściem do skrzynek podstawieniowych Sj. Każda skrzynka Sj przekształca 6-bitowe wejście (blok Bj = b1, b2, b3, b4, b5, b6) w blok 4-bitowy zgodnie z tablicami podstawień

© Piotr Remlein 2004



Algorytm Data Encryption Standard (DES)
skrzynki podstawieniowe S

wiersz	kolumna															
S_1		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)
skrzynki podstawieniowe S

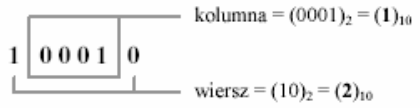
Podstawienia wykonywane są według zasady:

· wiersz określany jest za pomocą liczby całkowitej utworzonej ze skrajnych bitów bloku wejściowego Bj, tj. z b1 i b6

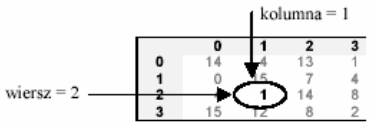
· kolumnę określa liczba całkowita utworzona ze środkowych 4 bitów, tj. z b2, b3, b4, b5.

© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)
Przykładowo – niech wejście do skrzynki S1 będzie liczbą binarną (100010)



zatem liczbą odczytaną z tablicy dot. skrzynki S1 będzie liczba całkowita równa **1**



© Piotr Remlein 2004

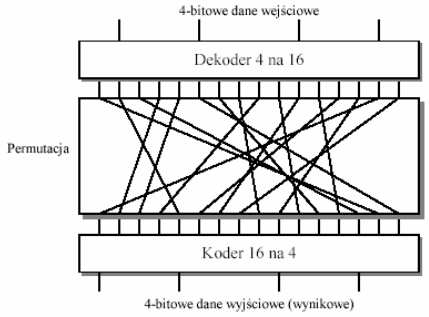
Algorytm Data Encryption Standard (DES)
Wartość podstawienia Sj(Bj) = s1, s2, s3, s4, jest więc 4-bitową liczbą całkowitą wyznaczoną z tablicy podstawień.

Strukturę wewnętrzną skrzynki S można przedstawić jako sieć podstawieniowo-permutacyjną

© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

Element skrzynki S (wiersz 0 skrzynki S1)



© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

Osiem 4-bitowych bloków wyjściowych z S-bloków są ze sobą łączone w 32-bitowy blok wynikowy, który poddawany jest permutacji P

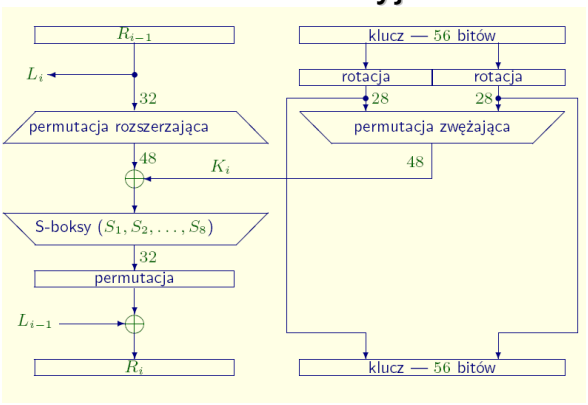
16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Ostateczny wynik wykonania funkcji F(Ri-1, Ki) poddawany jest operacji XOR z Li-1 tworząc 32-bitowy blok wejściowy dla prawej strony Ri następnej iteracji

Ri = Li-1 XOR F(Ri-1, Ki).

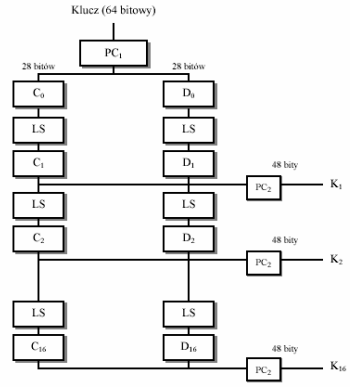
© Piotr Remlein 2004

Klucze iteracyjne



Algorytm Data Encryption Standard (DES)

klucze iteracyjne



© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

klucze iteracyjne

48 -bitowy **klucz iteracyjny** tworzony jest z 56-bitowego klucza głównego w algorytmie generowania kluczy iteracyjnych.

Wejściem jest 64-bitowy klucz podstawowy K. Permutacja PC1 odrzuca z K bity parzystości (k8, k16, k24, k32, k40, k48, k56, k64), a pozostałe bity przenosi zgodnie z porządkiem opisanym w tab.

Wynik PC1(K) dzielony jest na dwie 28-bitowe połówki: lewą C0 oraz prawą D0. Przy

każdej iteracji 28-bitowe bloki Ci i Di oddzielnie podlegają cyklicznemu przesunięciu

w lewo (rotacji w lewo) o liczbę pozycji podaną w tab.

© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

klucze iteracyjne

Permutacja klucza PC1

k_{57}	k_{49}	k_{41}	k_{33}	k_{25}	k_{17}	k_9	k_1	k_{58}	k_{50}	k_{42}	k_{34}	k_{26}	k_{18}
k_{10}	k_2	k_{59}	k_{51}	k_{43}	k_{35}	k_{27}	k_{19}	k_{11}	k_3	k_{60}	k_{52}	k_{44}	k_{36}
k_{63}	k_{55}	k_{47}	k_{39}	k_{31}	k_{23}	k_{15}	k_7	k_{62}	k_{54}	k_{46}	k_{38}	k_{30}	k_{22}
k_{14}	k_6	k_{61}	k_{53}	k_{45}	k_{37}	k_{29}	k_{21}	k_{13}	k_5	k_{28}	k_{20}	k_{12}	k_4

Przesunięcia w lewo LS

iteracja	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
liczba pozycji	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

Zatem przyjmując, że Ci i Di są odpowiednio lewą i prawą połówką używaną do generowania klucza iteracyjnego Ki, możemy zapisać :

Ci = LSi(Ci-1)

Di = LSi(Di-1)

dla i = 1, 2, 3, ..., 16, gdzie LSi jest cyklicznym przesunięciem w lewo (rotacją w lewo).

Połączone bloki CiDi są permutowane w PC2.

14	17	11	24	1	5	3	28	15	6	21	10
23	19	12	4	26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40	51	45	33	48
44	49	39	56	34	53	46	42	50	36	29	32

© Piotr Remlein 2004

Algorytm Data Encryption Standard (DES)

W wyniku takiego przekształcenia

Ki= PC2(CiDi)

otrzymywany jest iteracyjny klucz Ki .

Szyfrowanie i deszyfrowanie realizowane jest przy użyciu tego samego algorytmu, przy czym podczas deszyfrowania używana jest odwrotna kolejność kluczy iteracyjnych (K16, K15, ..., K2, K1).

W algorytmie DES każdy bit tekstu zaszyfrowanego (szyfrogramu) jest złożoną funkcją wszystkich bitów tekstu jawnego i wszystkich bitów klucza. Powoduje to, że zmiana nawet jednego bitu w przetwarzanym bloku powoduje lawinę zmian na wyjściu. Własność ta jest głównym czynnikiem wpływającym na bezpieczeństwo szyfru.

© Piotr Remlein 2004

Tryby pracy podstawieniowo-permutacyjnych szyfrów kaskadowych

Algorytmy blokowe, w tym wykorzystujące sieci podstawieniowo-permutacyjne, mogą pracować w różnych trybach.

Do najpopularniejszych trybów pracy, które wykorzystywane są również przez algorytm DES, zalicza się:

· **ECB** – Electronic Codebook mode
· **CBC** – Cipher Block Chaining mode
· **CFB** – Cipher Feedback mode
· **OFB** – Output Feedback mode

© Piotr Remlein 2004

- ECB
  - Tryb elektronicznej książki kodowej (electronic codebook).
- CBC
  - Tryb wiązania bloków zaszyfrowanych (cipher block chaining).
- CFB
  - Tryb sprzężenia zwrotnego szyfrogramu (cipher feedback).
- OFB
  - Tryb sprzężenia zwrotnego wyjściowego (output feedback).

ECB - Electronic Codebook tryb elektronicznej książki kodowej)

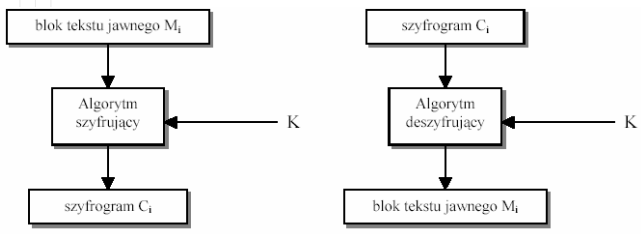
W trybie tym każdy blok tekstu jawnego przekształcany jest bezpośrednio w blok szyfrogramu.

Przy takiej realizacji możliwe jest niezależne szyfrowanie każdego bloku tekstu jawnego. Jednak w przypadku ustalonego klucza każdy 64-bitowy blok tekstu jawnego pojawiający się więcej niż raz będzie szyfrowany zawsze do takiej samej postaci.

Własność ta jest poważną wadą, którą wyeliminowano w trybie CBC.

© Piotr Remlein 2004

ECB - Electronic Codebook tryb elektronicznej książki kodowej)



© Piotr Remlein 2004

Bankowe standardy rekomendują tryby ECB i CBC **do szyfrowania**

tryby CBC i n-bitowe CFB do **uwierzytelniania**

- **Zalety ECB**
- **-** szybkość przetwarzania taka sama jak szyfru blokowego
- **-** przetwarzanie może być zrównoleglone
- **wada -** bloki są szyfrowane niezależnie, co ułatwia kryptoanalizę

- **zaleta** — utrata lub uszkodzenie pojedynczych bloków nie ma wpływu na możliwość deszyfrowania pozostałych;
- **wada** — możliwa jest modyfikacja kryptogramu bez znajomości klucza
- - charakterystyczne fragmenty tekstu jawnego nie są ukrywane
- **Zastosowanie**
- **-** szyfrowania baz danych
- **-** szyfrowanie systemów plików,

CBC - Cipher Block Chaining (tryb wiązania bloków zaszyfrowanych)

W trybie tym pierwszy blok tekstu jawnego M1 poddawany jest operacji XOR z pewny wektorem początkowym I, który podobnie jak klucz musi być znany zarówno nadawcy, jak i odbiorcy wiadomości.

Rezultat tej operacji jest następnie szyfrowany, otrzymujemy pierwszy zaszyfrowany 64-bitowy blok C1.

W kolejnym kroku blok C1 poddawany jest operacji XOR z kolejnym blokiem tekstu jawnego, a rezultat jest szyfrowany. Proces ten można zapisać jako:

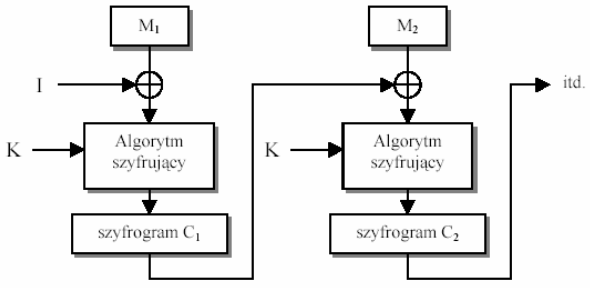
$$Ci = EK(Mi \text{ XOR } Ci\text{-}1) \quad i = 1,2, \ldots$$

gdzie E oznacza algorytm szyfrujący, K klucz (hasło), natomiast C0 = I.

Dzięki takiej konstrukcji każdy blok szyfrogramu silnie zależy od poprzednich bloków tego szyfrogramu – na wektorze I kończąc. Wadą takiego rozwiązania jest to, iż nie możliwe jest szyfrowanie ciągów bitów krótszych niż długość bloku.

© Piotr Remlein 2004

CBC - Cipher Block Chaining (tryb wiązania bloków zaszyfrowanych)



© Piotr Remlein 2004

Tekst jawny jest przed zaszyfrowaniem sumowany mod 2 z poprzednim blokiem szyfrogramu.

Szyfrowanie:

$$C_i = E_k (P_i \oplus C_{i-1})$$

Deszyfrowanie:

$$P_i = C_{i-1} \oplus D_K (C_i)$$

- **Wektor początkowy**
- - Może być dobierany losowo
- Nie musi być utajniany może być przesyłany z szyfrogramem

- **Szyfrowanie może rozpocząć się po odebraniu całego bloku danych**

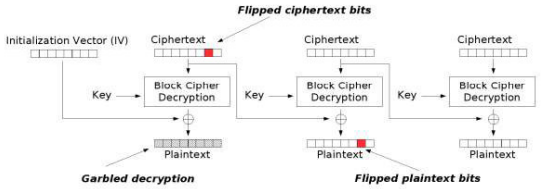
- **Zalety:**
- – takie same bloki tekstu jawnego dają różne kryptogramy
- – losowy IV powoduje, że ponowne zaszyfrowanie tego samego tekstu daje inny kryptogram
- – przekłamanie w jednym bloku kryptogramu prowadzą do błędów w dwóch blokach tekstu jawnego

- **Wady:**
- – nie można usunąć żadnego bloku kryptogramu
- – nie można dodać nowego bloku
- – zmiana w podziale na bloki powoduje lawinowy błąd rozszyfrowania nie nadaje się do szyfrowania baz danych;
- nieodporny na zakłócenia (dodatkowy bit lub utrata jednego bitu psują dalszy przekaz)

Tryby pracy

propagacja błędów w ECB i CBC

- ECB: błąd w jednym bloku przenosi się na jeden blok
- CBC: błąd w jednym bloku przenosi się na dwa bloki



Modification attack on CBC

CFB - Cipher Feedback (tryb szyfrowania ze sprzężeniem zwrotnym)

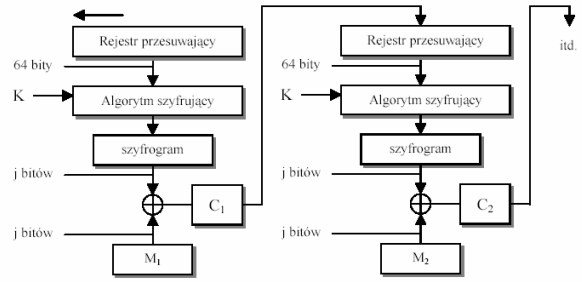
W tym trybie możliwe jest szyfrowanie ciągów bitów o długości mniejszej niż długość bloku szyfru. Dzieje się tak dlatego, gdyż tryb CFB powoduje, że algorytm działa jak szyfr strumieniowy, co zwalnia nas w takim przypadku z dopełniania tekstu jawnego do pełnych bloków.

Przy takiej metodzie szyfrowanie może przebiegać w czasie rzeczywistym (nawet pojedyncze znaki mogą być szyfrowane i natychmiast przekazywane).

© Piotr Remlein 2004

- **W trybie CFB stosowany jest rejestr przesuwny, w którym wpisany jest wektor początkowy I. Ten początkowy wektor jest wejściem dla funkcji szyfrującej. Następnie j bitów skrajnych z lewej strony rezultatu funkcji jest poddawanych operacji XOR z pierwszym blokiem j bitów tekstu jawnego M1, co daje nam pierwszy blok zaszyfrowany C1. Blok C1 jest następnie podawany na wejście rejestru przesuwnego (skrajnie po prawej), a w rejestrze wykonywana jest operacja przesunięcia w lewo o j bitów. Szyfrowanie następnych j-bitowych bloków przebiega według tego samego schematu.**

CFB - Cipher Feedback (tryb szyfrowania ze sprzężeniem zwrotnym)



© Piotr Remlein 2004

- Działanie CFB
- 1. na początku rejestr przesuwający zawiera losowy ciąg 64 bitów
- 2. zawartość rejestru przesuwającego jest szyfrowana za pomocą klucza K np. algorytmem DES
- 3. 8 pierwszych bitów kryptogramu jest dodawane modulo 2 z 8 bitami reprezentującymi literę wiadomości (Mi) dając kryptogram Ci przesyłany do odbiorcy
- 4. Ci jednocześnie przesyłane jest do rejestru przesuwającego zajmując ostatnie 8 bitów i przesuwając pozostałe bity o 8 pozycji w lewo; przesunięcie to nie jest cykliczne, tzn. pierwszych 8 bitów jest usuwanych
- 5. przy deszyfrowaniu rola wejścia i wyjścia zostaje zamieniona

- CFB dzięki rejestrowi przesuwającemu umożliwia szyfrowanie pojedynczych bajtów/bitów
- Na początku rejestr przesuwający jest losowany IV
- CFB tworzy **szyfr strumieniowy**

zalety

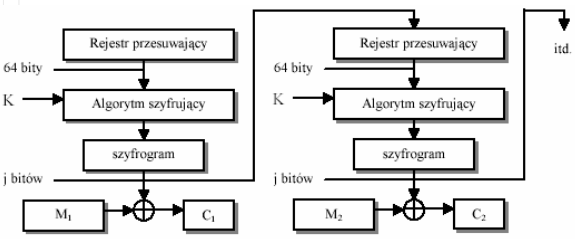
- Charakterystyczne fragmenty tekstu jawnego są ukrywane
- Więcej niż jedna wiadomość może być szyfrowana tym samym kluczem (przy założeniu, że będzie stosowany inny wektor IV)
- Błędy synchronizacji dla całych bloków są odtwarzalne; tryb 1-bitowego CFB może odtworzyć swoje działanie po wstawieniu lub utracie 1 bitu.

wady

- Jest możliwe dokonanie pewnych obliczeń wstępnych, zanim blok pojawi się, a poprzedni blok szyfrogramu może być zaszyfrowany
- Błąd w szyfrogramie wpływa na jeden bit odpowiadającego mu tekstu jawnego i cały następny blok tekstu jawnego

OFB – Output Feedback (tryb sprzężenia zwrotnego wyjściowego)

Tryb ten jest bardzo podobny do trybu CFB. Różnica polega na tym, że w OFB na skrajnie prawe pozycje rejestru przesuwnego podawany jest j-bitowy blok pochodzący z wyjścia funkcji szyfrującej, a nie z bloku zaszyfrowanego.



© Piotr Remlein 2004

Zalety

Podobne do poprzedniego trybu

Błąd w szyfrogramie wpływa na odpowiadający mu bit tekstu jawnego

Wady

Bardzo łatwa możliwość manipulacji tekstem jawnym, dowolna zmiana w szyfrogramie bezpośrednio wpływa na tekst jawny

Błędy synchronizacji nie są odtwarzalne

Tryby pracy

zastosowania

- Tryb elektronicznej książki kodowej (ECB)
  – bazy danych
- Tryb wiązania bloków zaszyfrowanych (CBC)
  – szyfrowanie plików
- Tryb sprzężenia zwrotnego szyfrogramu (CFB)
  – szyfrowanie strumieni znaków
- Tryb sprzężenia zwrotnego wyjściowego (OFB)
  – w systemach z transmisją synchroniczną o dużej szybkości, bez tolerancji na propagację błędu

Deszyfrowanie

- **Algorytm deszyfrowania za pomocą DES jest identyczny jak algorytm szyfrowania. W procesie deszyfrowania, używamy jednak w odwrotnej kolejności kluczy cyklu *Ki*, a mianowicie klucz *K16* używany podczas szyfrowania, podczas deszyfrowania jest kluczem *K1*.**

- **Permutacja *IP* odwraca działanie *IP-1*. Operacje wykonywane w pierwszym cyklu, odwracają operacje szyfrowania cyklu 16, operacje wykonywane w cyklu 2, odwracają operacje szyfrowania cyklu 15, itd...**

- **Liczne ataki na DES, pokazały, że DES jest zbyt słaby aby w dalszym ciągu stanowił standard szyfrowania. Przez wiele lat, nie było jednak następcy, szyfru który gwarantowałby znacznie większe bezpieczeństwo, szyfru którego bezpieczeństwo zostało potwierdzone.**

- **Próby modyfikacji DES, w celu zwiększenia jego siły kryptograficznej oraz eliminacji wad, nie przynosiły określonych rezultatów, zaś niektóre z nich powodowały nawet osłabienie szyfru.**

Komentarze i obserwacje dotyczące DES

- **Permutacja Inicjująca IP - Załóżmy, że 64 -bitowy blok danych jest reprezentowany przez 8 znaków ASCII (b0b1b2b3...b63). Bity parzystości zajmują więc pozycje o numerach 0,8,16,24,32,40,48,56 lub 7,15,23,31,39,47,55. Zatem po permutacji, będą one zajmowały pozycje odpowiednio 39,38,37,36,35,34,33,32 lub 31,30,29,28,27,26,25,24. Można więc zauważyć, że bity parzystości łączone są w jeden bajt, oraz to, że tworzą one pierwszy bajt bloku R lub ostatni bajt bloku L .**

S-boxy

- **Dokonując dekompozycji S -boxa S4 widać, że tylko pierwsza z jego funkcji jest nieliniowa, pozostałe trzy są liniowe i można je uzyskać z pierwszej poprzez negację bitów wejściowych oraz negację 2 i 3 wyjścia sterowaną zmienną X6. Zmniejsza to liczbę nieliniowych funkcji w DES z 32 do 29.**

- **Każdy z S-boxów charakteryzuje się również tym, że dla niektórych kombinacji wejściowych, negacja dwóch bitów, lub odpowiednia modyfikacja niektórych bitów wejściowych nie ma wpływu na wartość wyjściową S-boxa.**

- **Funkcja F nie jest funkcją typu 1 na 1 jest zatem możliwe, że dla dwóch różnych wartości 32 bitów R , na wyjściu funkcji F otrzymamy jednakową wartość.**

Własność komplementarności

- **Funkcja szyfrująca powinna być losową funkcją zarówno klucza jak i tekstu jawnego. Nie jest tak w przypadku DES. Jeśli Xj' stanowi dopełnienie bitowe bloku Xj, zaś K' - dopełnienie bitowe klucza wówczas:**
- **Cj = DES (Xj; K)**
- **Cj'= DES (Xj`; K`)**

- **Jeśli zatem kryptoanalityk ma do dyspozycji pary (X1 ; C1) oraz (X1' ; C2), wówczas dla jakiegokolwiek nieznanego klucza K można stwierdzić że: C2= DES (K; X1') oraz C2'=DES(K';X1) . Wniosek ten pozwala w przypadku poszukiwania klucza za pomocą ataku wybranym tekstem jawnym zredukować liczbę obliczeń o połowę - z 255 do 254.**

Słabe klucze

- **Jeśli obydwa bloki C i D w algorytmie rozszerzenia klucza będą zawierały same jedynki lub same zera, wówczas, generowany klucz cyklu będzie stały i jednakowy dla każdego cyklu. Zatem zamiana kolejności kluczy nie ma znaczenia, co pozwala stwierdzić, że proces szyfrowania i deszyfrowania stanowi jednakową iterację. Klucze takie nazywane są słabymi kluczami. Dla DES istnieją 4 słabe klucze których wartości przedstawione zostały poniżej.**

Słabe klucze *(weak keys)* - HEX				C_0	D_0
0101	0101	0101	0101	$\{0\}^{28}$	$\{0\}^{28}$
FEFE	FEFE	FEFE	FEFE	$\{1\}^{28}$	$\{1\}^{28}$
1F1F	1F1F	1F1F	1F1F	$\{0\}^{28}$	$\{1\}^{28}$
E0E0	E0E0	E0E0	E0E0	$\{1\}^{28}$	$\{0\}^{28}$

- **W praktyce oznacza to iż dla takiego klucza**

  c = DES(k;x)* oraz że *x=DES(k;DES(k;x))

Pół-słabe klucze (semi-weak keys)

- **W praktyce oznacza to, że dla pary kluczy (K1 ;K2) istnieje następująca zależność: x=DES(K1; DES(x ; K2)), co oznacza anulowanie procesu szyfrowania.**
- **Dla DES istnieje 6 par pół-słabych kluczy.**

Częściowo słabe klucze *(quasi weak keys)*.

- **Na Eurocrypt 94, L.R. Knudsen[1] przedstawił kolejną grupę 256 kluczy zwaną częściowo słabymi kluczami. Klucze te dają cztery różne podklucze, z których każdy jest wykorzystywane w algorytmie czterokrotnie.**

- **L.R. Knudsen udowodnił, że istnieje grupa kluczy, różniących się tylko bitami odnoszącymi się do jednego S-boxa, przy użyciu których szyfrując dany tekst jawny uzyskamy identyczne szyfrogramy. Nie stanowią one zagrożenia w ataku DES, mogą jednak stanowić pewne zagrożenie gdy za pomocą DES realizowana jest funkcja haszująca.**
- [1] L.R. Knudsen, New Potentially Weak Keys for DES and LOKI,extended abstract, Eurocrytp 94, Springer Verlag 1998

Szybkość DES

- **DES jest szyfrem przeznaczonym głównie do implementacji sprzętowych. Na rynku pojawiło się więc wiele układów szyfrujących za pomocą DES. Obecne sprzętowe implementacje DES pozwalają na uzyskanie prędkości rzędu kilkunastu GBit/sec. Poszczególne przypadki zostały omówione w dalszej części pracy, stanowiąc swego rodzaju punkt odniesienia przy porównywaniu prędkości innych szyfrów.**

Siła kryptograficzna i bezpieczeństwo DES

- **Atak na DES polegający na wyczerpującym poszukiwaniu klucza ma złożoność 2^56. Jest to jednak najgorszy z możliwych ataków na DES. Biorąc pod uwagę możliwości obliczeniowe dzisiejszych komputerów, złożoność ta nie stanowi większej przeszkody.**

Wykład 5

Podwójny algorytm DES

Mimo złożonej operacji szyfrowania DES tekst zaszyfrowany jest narażony na kryptoanalizę (łamanie szyfru). Z tego powodu dla poprawienia bezpieczeństwa szyfru stosuje się wielokrotne szyfrowanie przy użyciu algorytmu DES z użyciem wielu kluczy.

Najprostszym przykładem jest szyfrowanie wykonywane w dwóch etapach, przy użyciu dwóch kluczy. Metodę tą nazywa się **podwójnym DES-em**

Podwójny algorytm DES

Jeżeli przyjmiemy, że proces szyfrowania przy użyciu algorytmu DES opisuje zależność

$Y = EK(M)$,

gdzie poszczególne składniki oznaczają :

M - tekst jawny,

Y - tekst zaszyfrowany,

E - algorytm szyfrujący,

K – klucz szyfrujący,

to proces podwójnego szyfrowania z dwoma kluczami możemy zapisać następująco:

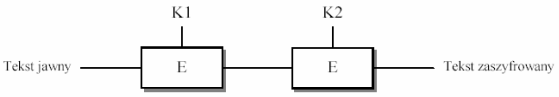
$Y = EK2(EK1(M))$

gdzie M, Y i E j.w., natomiast K1 i K2 oznaczają odpowiednio

K1 – klucz szyfrujący K1,

K2 – klucz szyfrujący K2.

Podwójny algorytm DES



Podwójny algorytm DES

Proces odwrotny do powyższego, tj. deszyfrowanie zapiszemy:

$M = DK1(DK2(Y))$

M, Y, K1, K2 – j.w. ,

D – algorytm deszyfrujący.

Z powyższego zapisu wynika, że w procesie deszyfrowania kolejność stosowania kluczy jest odwrotna, tj. w pierwszym etapie używany jest K2, a w drugim K1.

Zastosowanie *podwójnego DES*-a powoduje zwiększenie rozmiaru klucza z 56-bitów do 112-bitów, co teoretycznie zwiększa poziom bezpieczeństwa.

Podwójny algorytm DES

Przyjęta struktura w *podwójnym DES*-ie nie czyni jednak tej metody w pełni bezpieczną, gdyż możliwe jest przeprowadzenie skutecznej kryptoanalizy przy użyciu algorytmu nazywanego *atakiem ze spotkaniem pośrodku* (meet-in-the-middle attack).

Próby łamania *podwójnego DES*-a z kluczem 112-bitowym ze znanym tekstem jawnym powiodą się przy liczbie prób wynoszącej zaledwie 2^56, a nie jak by można się spodziewać 2^112.

Z tego właśnie względu w praktyce wykorzystuje się powszechnie akceptowane potrójne szyfrowanie przy użyciu algorytmu DES. **Potrójny DES** w naturalny sposób zabezpiecza nas przed atakami ze spotkaniem pośrodku.

Potrójny algorytm DES

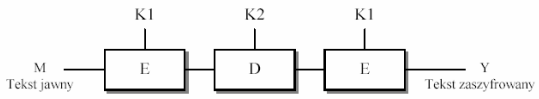
Szyfrowanie przy użyciu potrójnego algorytmu DES przebiega w trzech etapach przy użyciu trzech kluczy. Zatem do całkowitego przebadania klucza, którego rozmiar wynosi aż 168-bitów, należy przeprowadzić średnio 2^112 prób. Daje to bezpieczną metodę szyfrowania, która jest praktycznie nie do złamania, nawet przez komputer o zaawansowanej technologii.

- Używanie jednak tak dużego klucza jest poważną wadą, dlatego też opracowano alternatywną metodę *potrójnego DES*-a wykorzystującego dwa klucze.
- Proces szyfrowania przebiega wtedy według schematu

Y = EK1(DK2(EK1(M))) ,

który zaproponowany został przez W.Tuchmana.

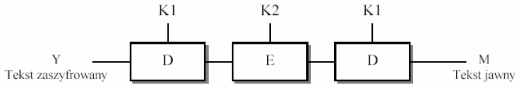
Potrójny algorytm DES



Potrójny algorytm DES

Deszyfrowanie możemy zapisać tak :

M = DK1(EK2(DK1(Y)))



zapewnieniem bezpieczeństwa dla szyfrogramów, ma jeszcze jedną zaletę. Przy użyciu tej metody możliwe jest deszyfrowanie wiadomości, które zostały zaszyfrowane przy użyciu klasycznego algorytmu DES (pojedynczego) :

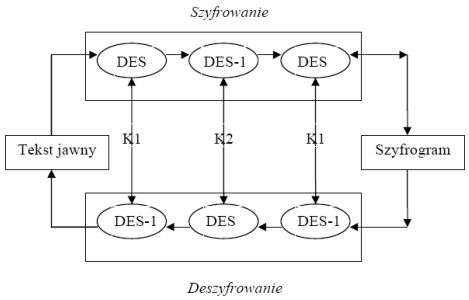
Y = EK1(DK1(EK1(M))) = EK1(M)

Modyfikacje algorytmu DES Potrójny DES

W komercyjnych rozwiązaniach wykorzystuje się trzykrotny DES. Uzyskany szyfrogram jest dużo trudniejszy do złamania metodą wyczerpującego przeszukiwania: 2^112 prób zamiast 2^56.

Co ważne potrójne DES opiera się kryptografii różnicowej

Modyfikacje algorytmu DES Potrójny DES



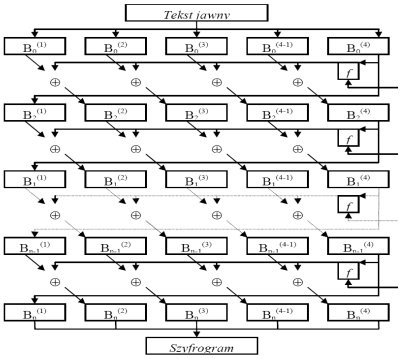
Modyfikacje algorytmu DES
Uogólniony DES

Uogólniony DES *(Generalized DES – GDES),* Tekst jawny dzielony jest na bloki o długości 32 bity i posiada niezmienioną funkcją *f.*

Funkcja *f* liczona jest tylko raz na

cykl, dla podbloku leżącego najbardziej na prawo

Modyfikacje algorytmu DES
Uogólniony DES

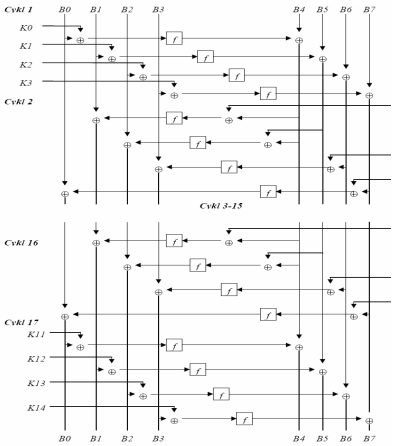


Modyfikacje algorytmu DES
NewDES

NewDES został zaprojektowany przez Roberta Scotta, w roku 1985, jako następca DES.

Blok tekstu jawnego nadal ma 64 bity, zmieniła się natomiast długość klucza, 120 bitów.

Scott zrezygnował w swojej wersji algorytmu z permutacji początkowej i końcowej, co w znacznym stopniu uprościło sam algorytm. Wszystkie operacje są wykonywane na całych bajtach, a sam algorytm nigdy nie czyta, nie zapisuje, nie permutuje żadnych pojedynczych bitów.



DESX

- Pomysł polega na tym by wykorzystać oprócz 56 bitowego klucza dwa dodatkowe 64 bitowe klucze Kp i Kc.
- Wzór opisujący kryptogram jest następujący:

C=Kc XOR DES_Kdes (P XOR Kp)

IDEA — International Data Encryption Algorithm

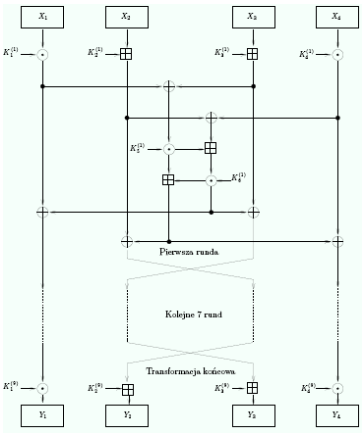
- • IDEA jest algorytmem blokowym wprowadzonym w latach 90-tych
- • IDEA używa kluczy 128 bitowych
- • IDEA jest używana w pakiecie PGP
- • IDEA jest algorytmem opatentowanym; można go używać bezpłatnie do celów niekomercyjnych
- • IDEA działa na blokach 64 bitowych i wykorzystuje 3 rożne operacje: xor (⊕), dodawanie modulo 2^16 (⊞) oraz mnożenie modulo 2^16+1 (⊙);

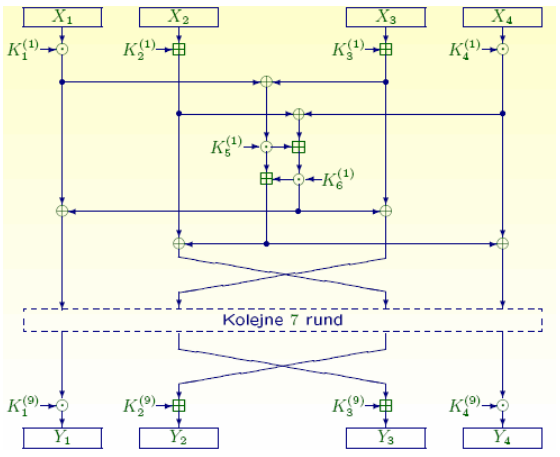
Cele stawiane przed algorytmem IDEA

- Związane ze skutecznością krytpograficzną
- Związane z łatwością realizacji algorytmu

- Długość bloków
- Długość klucza
- Mieszanie
- Rozpraszanie

- Zastosowanie trzech oddzielnych operacji utrudnia kryptoanalizę w większym stopniu niż w przypadku algorytmu DES.
- Podstawowym blokiem jest tzw. struktura mnożenia/dodawania





Szyfrowanie

- 64 bitowy blok jest dzielony na 4 bloki po 16 bitów:

X1,X2,X3,X4, które stanowią dane wejściowe dla pierwszej rundy algorytmu

- algorytm składa się z 8 rund
- w każdej rundzie wykonywane są wymienione 3

typy operacji na 16 bitowych blokach z 16 bitowymi

podkluczami (każda runda wymaga 6 podkluczy)

Szyfrowanie

- w wyniku otrzymuje się 4 bloki po 16 bitów: Y1, Y2, Y3, Y4
- pomiędzy rundami blok 2 i 3 są zamieniane
- algorytm kończy przekształcenie końcowe, które wymaga 4 podkluczy

Generowanie podkluczy

- IDEA używa klucza 128 bitowego i wymaga 8×6+4=52 podkluczy
- 128 bitowy klucz jest dzielony na bloki 16 bitowe, co daje 8 podkluczy
- na kluczu wykonuje się przesunięcie cykliczne o 25 pozycji i znowu dzieli na bloki 16 bitowe, co daje kolejne 8 podkluczy
- operację tę powtarza się tak długo aż wygeneruje się wszystkie podklucze

Generowanie podkluczy

- Deszyfrowanie algorytmem IDEA przebiega wg schematu dla szyfrowania, w którym zamiast

X1,X2,X3,X4 na wejściu podaje się bloki Y1, Y2, Y3, Y4 kryptogramu oraz klucz K (ten sam co przy szyfrowaniu)

- z klucza K generuje się podklucze $K_i^{(r)}$
- generuje się podklucze deszyfrujące K′(r) z kluczy szyfrujących wg schematu przedstawionego w tablicy

Tworzenie podkluczy deszyfrujących na podstawie podkluczy szyfrujących

Runda	$K'^{(r)}_1$	$K'^{(r)}_2$	$K'^{(r)}_3$	$K'^{(r)}_4$	$K'^{(r)}_5$	$K'^{(r)}_6$
$r=1$	$\left(K_1^{(10-r)}\right)^{-1}$	$-K_2^{(10-r)}$	$-K_3^{(10-r)}$	$\left(K_4^{(10-r)}\right)^{-1}$	$K_5^{(9-r)}$	$K_6^{(9-r)}$
$2\leq r\leq 8$	$\left(K_1^{(10-r)}\right)^{-1}$	$-K_3^{(10-r)}$	$-K_2^{(10-r)}$	$\left(K_4^{(10-r)}\right)^{-1}$	$K_5^{(9-r)}$	$K_6^{(9-r)}$
$r=9$	$\left(K_1^{(10-r)}\right)^{-1}$	$-K_2^{(10-r)}$	$-K_3^{(10-r)}$	$\left(K_4^{(10-r)}\right)^{-1}$	—	—

Założenia projektowe dla realizacji programowej

- Zastosowanie podbloków:

Operacje szyfrowania powinny przebiegać na podblokach „wygodnych” do programowania: 8, 16, 32 bitowych.

W alg. IDEA podbloki są 32-bitowe

- Proste operacje

Założenia projektowe dla realizacji sprzętowej

- Podobieństwo między szyfrowaniem i deszyfrowaniem
- Regularna struktura – w celu łatwego zrealizowania układu w technice VLSI

Cechy algorytmu IDEA

- *O ile szyfrowanie jest szybkie, o tyle wyznaczanie odwrotności w ciele modulo (2^{16} + 1) jest bardziej czasochłonne, lecz wystarczy przy znajomości klucza uczynić to raz na początku realizacji algorytmu deszyfracji.*
- *Istnieje klasa "słabych" kluczy algorytmu, tzn.takich, że są łatwe do zidentyfikowania przy pomocy ataku tekstem jawnym; jest ich 32 spośród wszystkich 2^{128}.*
- *Ze względu na przyjęte algebraiczne operacje dyfuzji i konfuzji łatwy do zaimplementowania także w trybie pracy strumieniowej (nie tylko blokowej).*
- *Krótki okres istnienia nie dał możliwości gruntownego zbadania słabych stron, niemniej jednak metody kryptoanalizy różnicowej w tym przypadku wydają się zawodzić.*

AES — Advanced Encryption Standard — Rijndael

Nowy standard przyjęty w 2001 r. w USA
• Algorytm blokowy, który zaprojektowali Joan Daemen i Vincent Rijmen
• Zarówno długość bloku jak i klucza może być wybrana jako 128, 192 lub 256 bitów
• Rijndael jest ogólnie dostępny
• Liczba rund zależy od długości bloku

AES — Advanced Encryption Standard — Rijndael

W każdej rundzie wykonywane są 4 operacje (macierzowe):
– podstawienie w S-boksie
– przesunięcie wierszy
– mieszanie kolumn
– xor z podkluczem
• Podklucze są generowane algorytmem, który zależy od rundy

AES — Advanced Encryption Standard — Rijndael

2 stycznia 1997 NIST ogłosił rozpoczęcie poszukiwań kandydata na nowy standard szyfrowania danych AES, określając minimalne wymogi następująco:
•dokumentacja szyfru musi być powszechnie dostępna,
•szyfr ma należeć do grupy blokowych szyfrów symetrycznych,
•projekt szyfru musi zakładać możliwość rozszerzenia długości klucza, w razie takiej potrzeby,

AES — Advanced Encryption Standard — Rijndael

szyfr musi być łatwy do implementacji zarówno sprzętowej jak i programowej,
nowy szyfr ma być bardziej efektywny oraz bardziej bezpieczny niż 3-DES, co implikuje następujące wymogi:
- •długość klucza określona jako 128, 192 lub 256 bitów
- •wielkość bloku równa 128, 192 lub 256 bitów

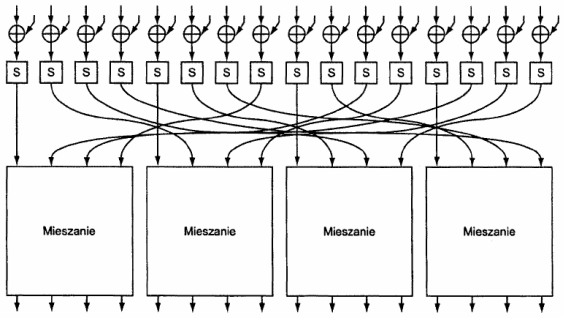
AES — Advanced Encryption Standard — Rijndael

Rozpatrując odpowiednią kandydaturę, brano również pod uwagę następujące aspekty:
•zapewniany poziom bezpieczeństwa w odniesieniu do struktury oraz złożoności,
•koszty implementacji oraz jej możliwości,
•elastyczność dotycząca nie tylko parametrów szyfru, ale i przystosowania do pełnienia innych kryptograficznych funkcji: szyfrów strumieniowych, funkcji haszujących..

AES — Advanced Encryption Standard — Rijndael

Spośród piętnastu kandydatów rundy pierwszej, do ścisłego finału zakwalifikowało się 5 szyfrów:
•Mars - szyfr zaprojektowany przez IBM,
•RC6 - RSA Labolatories,
•Rijndael - Joan Daemen, Vincent Rijmen,
•Serpent - Ross Anderson, Eli Biham, Lars Knudsen,
•Twofish - Bruce Schneier, John Kelsey, Doug Whiting, David Wagner, Chris Hall, Niels Ferguson
Wśród tych pięciu finalistów przeprowadzono serię dodatkowych badań: kryptoanaliz oceniających bezpieczeństwo, pomiarów szybkości w poszczególnych implementacjach, możliwości do zaadoptowania w nowych rozwiązaniach.

Uproszczony schemat pojedynczej iteracji



AES — Advanced Encryption Standard — Rijndael

Wszelkie transformacje Rijndaela przeprowadzane są na poziomie macierzy stanów, nazywanej *State*. Macierz ta zbudowana jest z czterech wierszy oraz z N kolumn. Wielkość tej macierzy zależy od długości bloku. Elementami macierzy są poszczególne bajty formułujące blok, oznaczane jako *S[r,c]*, gdzie *r* jest numerem wiersza, zaś *c* numerem kolumny.

AES — Advanced Encryption Standard — Rijndael

Liczba cykli NR (iteracji) wykonywanych przez szyfr zależna jest zarówno od długości klucza jak i od długości bloku. Zależności te, dla wartości uznanych jako standardowe przedstawiono w poniższej tabelce

Nr	Nb=4	Nb=6	Nb=8
Nk=4	10	12	14
Nk=6	12	12	14
Nk=8	14	14	14

AES — Advanced Encryption Standard — Rijndael

Liczbę cykli można też przedstawić za pomocą wzoru:
NR = max (NK, NB) +6,

Każdy cykl ma swój własny klucz, wygenerowany za pomocą algorytmu rozszerzenia klucza.

AES — Advanced Encryption Standard — Rijndael

Proces szyfrowania rozpoczyna się od wygenerowania listy kluczy cyklu oraz sumowania XOR pierwszego klucza cyklu *RoundKey[0]* wraz z macierzą *State.* Następnie N(R-1) razy wykonywane są kolejno cztery transformacje macierzy *State*. Ostatnia z nich stanowi sumę XOR tej macierzy i klucza cyklu o numerze określonym numerem iteracji. Po wykonaniu tego kroku wykonywana jest kolejna, ostatnia iteracja, różniąca się od pozostałych tym, że w jej trakcie macierz *State* nie podlega transformacji *MixColumn.*

AES — Advanced Encryption Standard — Rijndael

SubByte jest transformacją, podstawiająca każdemu bajtowi macierzy stanów wartość określoną przez przekształcenie algebraiczne, które jest realizowane w dwóch krokach:

wyliczenia multiplikatywnej inwersji bajtu, reprezentowanego w postaci wielomianu, w ciele GF(2^8)

dokonania przekształcenia powyższej wartości zgodnie z afiniczną transformacją w ciele GF(2), zdefiniowaną przez następujące równanie

AES — Advanced Encryption Standard — Rijndael

bi' = bi ⊕ b(i+4) mod 8 ⊕ b(i+5) mod 8 ⊕ b(i+6) mod 8 ⊕ b(i+7) mod 8 ⊕ ci (1)

dla 0 ≤ i < 8 gdzie bi jest i tym bitem bajtu b zaś ci jest i-tym bitem bajtu 01100011. Transformację tą wygodnie jest przedstawić w postaci macierzowej jako:

$$\begin{bmatrix} b_7' \\ b_6' \\ b_5' \\ b_4' \\ b_3' \\ b_2' \\ b_1' \\ b_0' \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} b_7 \\ b_6 \\ b_5 \\ b_4 \\ b_3 \\ b_2 \\ b_1 \\ b_0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

AES — Advanced Encryption Standard — Rijndael

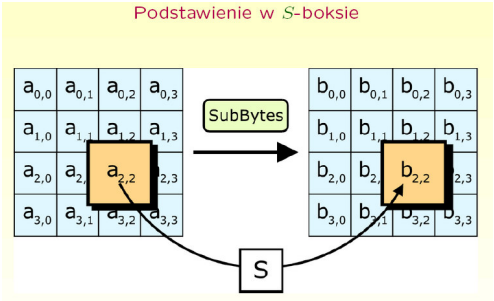
Możliwe jest zatem wyliczenie powyższych przekształceń dla każdej z wartości bajtów i stworzenie odpowiedniej tablicy podstawień, określanej jako S-box.

hex		X															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
y	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

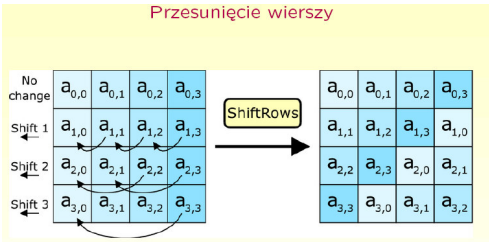
AES — Advanced Encryption Standard — Rijndael

Podstawienie odbywa się zgodnie z następującym schematem. Każdy bajt można opisać w kodzie heksadecymalnym jako {xy}HEX. Aby odczytać wartość bajtu po dokonaniu transformacji odczytujemy odpowiedni wiersz zdefiniowany przez x, oraz odpowiednia kolumnę zdefiniowaną przez y. Przykładowo B={4F}, po transformacji SubByte przyjmie wartość równą B={84}. Efekt działania funkcji SubByte przedstawia poniższy rysunek:

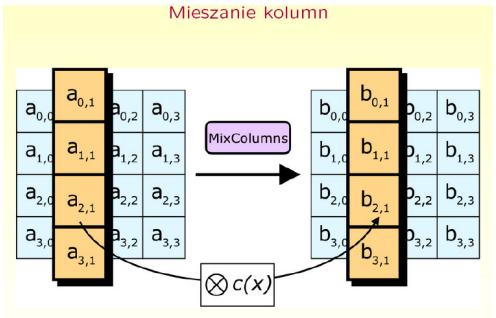
AES — Advanced Encryption Standard — Rijndael

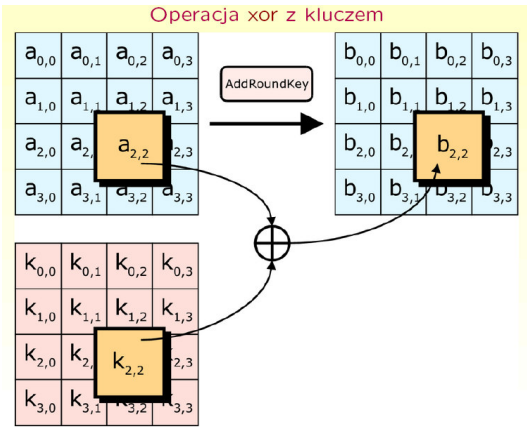


AES — Advanced Encryption Standard — Rijndael



AES — Advanced Encryption Standard — Rijndael





- Przed pierwszą rundą wykonywane jest przekształcenie AddRoundKey, a w ostatniej rundzie pomija się operację MixColumn.
- Algorytm operujący na blokach 128 bitowych wykorzystuje 10 rund, przy dłuższych blokach i kluczach wykonywane są 12 lub 14 rundy.

AES — Advanced Encryption Standard — Rijndael

Proces deszyfrowania polega na użyciu operacji odwrotnych do operacji szyfrowania, przy tych samych wartościach kluczy cyklu. Kolejność transformacji musi być również odwrotna.

RC2 (Rivers Cipher 2)

- opracowany przez Ronalda Rivesta. Stanowił tajemnicę handlową firmy RSA Data Security do 1996 roku, kiedy to został ujawniony w anonimowym liście przesłanym do jednej z grup dyskusyjnych. Algorytm ten jest mocny, jednak istnieje grupa kluczy mniej odpornych na złamanie. Dostępny jest w różnych wersjach umożliwiających stosowanie klucza o długości od 1 do 2048 bitów.

RC5

(R.Rivest 1994)

- Zmienna ilość rund, wielkość bloków i długość klucza
- Elastyczny, ale trudny do sprzętowej implementacji
- Zawiłe metody generowania dużych ilości podkluczy

RC5 założenia konstrukcyjne

- Szyfr symetryczny
- Przeznaczony do implementacji sprzętowej i programowej
- Szybki i możliwy do implementacji w procesorach o różnej długości słowa danych
- Struktura szyfru iteracyjna z różną liczbą cykli
- Możliwość pracy z kluczami o różnej długości
- Prosty i łatwy w opisie

RC5 opis parametryczny

RC5 w/r/b

w – wielkość słowa wejściowego w bitach

r – liczba cykli

b – długość klucza głównego K w bajtach

Bloki tekstu jawnego i kryptogramu mają długość 2w bitów

- Przykład zalecany przez twórców szyfru

W=32

to

r=12

b=16 bajtów

RC5 szyfrowanie

$A=A+S_0$

$B=B+S_1$

Suma w ciele GF(2^w)

$$A=(((A\ XOR\ B)\ll B)+S[2i])\ mod\ 2^w$$

$$B=(((B\ XOR\ A)\ll A)+S[2i+1])\ mod\ 2^w$$

gdzie:
i – numer rundy
2w – długość bloku
A,B – dwie połówki bloku

Schemat algorytmu RC5

RC5 deszyfrowanie

Proces deszyfrowania polega na przeprowadzeniu operacji odwrotnych do szyfrowania i w odwrotnej kolejności

B=((B – S_{2i+1}) >>>A) XOR A
A=((A – S_{2i}) >>>B) XOR B

B=B – S_1
A=A – S_0

RC5 właściwości

Prosty w implementacji zarówno sprzętowej i programowej.
Brak S-bloków pozwala zredukować wielkość potrzebnej pamięci
Stosowane operacje algebraiczne umożliwiają uzyskanie dużej szybkości na procesorach 8 i 32 bitowych oraz strukturach FPGA
Jest odporny na atak kryptoanalizy różnicowej i liniowej po 12 cyklach

RC6 właściwości

U podstaw konstrukcji leżą te same założenia jak przy RC5

Cykliczne przesunięcia danych w obrębie rejestrów roboczych
Główne różnice:
Stosowanie 4 rejestrów
Wprowadzenie dodatkowej operacji algebraicznej mnożenia w ciele GF(2^w)

RC6 opis parametryczny

RC6 w/r/b

w – wielkość słowa wejściowego w bitach

r – liczba cykli

b – długość klucza głównego K w bajtach

RC6 opis

RC6 używa rozszerzonej w stosunku do RC5 liczby operacji algebraicznych.

a + b -suma dwóch liczb w ciele skończonym ciele GF(2^w)

a - b -różnica dwóch liczb w ciele skończonym ciele GF(2^w)

a ⊕ b -suma xor - modulo 2, dwóch liczb całkowitych

a x b - mnożenie całkowite liczb w skończonym ciele GF(2^w)

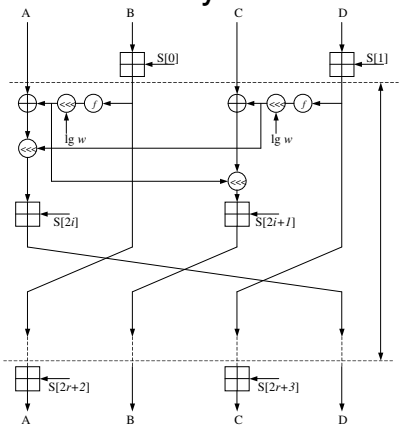
a<<<b - cykliczne przesunięcie bitów w lewo w rejestrze A, o wartość daną przez 5 mniej znaczących bitów rejestru b

a>>>b - cykliczne przesunięcie bitów w prawo w rejestrze A, o wartość daną przez 5 mniej znaczących bitów rejestru b

RC6 szyfrowanie

B = B + S[0]
D = D + S[1]
for i = 1 to r do
{
$t = (B \times (2B + 1)) <<< \lg_2(w)$
$u = (D \times (2D + 1)) <<< \lg_2(w)$
$A = ((A \oplus t) <<< u) + S[2i]$
$C = ((C \oplus u) <<< t) + S[2i + 1]$
$(A, B, C, D) = (B, C, D, A)$
}
$A = A + S[2r + 2]$
$C = C + S[2r + 3]$

RC6 szyfrowanie



Różnice RC6 w porównaniu z RC5

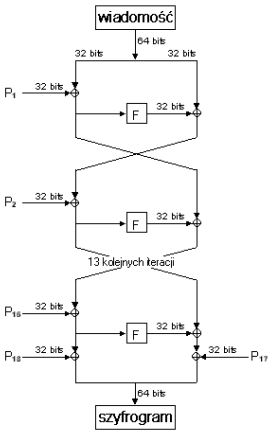
wykonanie połowy cyklu RC5

równoległe wykonywanie dwóch cykli RC5, jednego cyklu na rejestrach *A* i *B,* drugiego na rejestrach *C* i *D* z następującym wyjątkiem: zamiast zamiany zawartości *A* i *B* oraz *C* i *D*, dokonania permutacji: *(A,B,C,D)=(B,C,D,A)*

pomieszanie par rejestrów *A, B* oraz *C,D* z uwzględnieniem wartości współczynników cyklicznych rotacji.

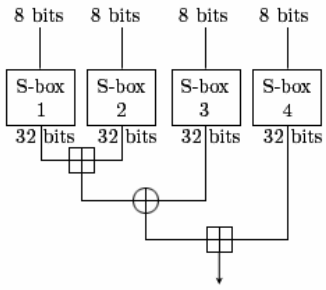
Algorytm Blowfish

- algorytm utworzony w 1993 roku przez Bruce'a Schneiera. Nie znaleziono skutecznego sposobu jego złamania. Jest używany w wielu popularnych programach kryptograficznych
- jest szyfrem blokowym operującym na 64-bitowych blokach. Długość klucza może być od 32-bitów do 448-bitów. Algorytm ma postać szyfru Feistela z 16-toma rundami i wykorzystuje duże Sbox-y zależne od klucza.



- W każdej rundzie jest wykonywana operacja na 32 bitach. Algorytm przechowuje dwie tablice podkluczy, jedną wielkości 18-stu elementów P oraz tablice S-boxów wielkości 4 razy po 256.
- S-box akceptuje 8bitów na wejściu, a na wyjściu daje 32 bity. Każde pole tablicy P jest wykorzystywane w każdej rundzie i po ostatniej rundzie każda połowa bloku danych jest XORowana przez jedno z dwóch pozostałych nie użytych pól tablicy P

- funkcja F algorytmu blowfish dzieli 32bity na wejściu na cztery części gdzie każda jest wielkości 8 bitów i wykorzystuje te części jako wejścia do SBoxów. Wyjścia są dodawane, a następnie jest wykonywana na nich operacja modulo 2^32 i xor-owania aby na wyjściu powstała wartość 32 bitowa.



Opis algorytmu

- Inicjujemy S-bloki i skrzynkę P kluczy pomocniczych ma podstawie głównego klucza użytkownika. S-bloków jako skrzynek podstawień mamy w Blowfish cztery, gdzie każda z nich posiada 256 elementów, z kolei każdy element jest wartością 32-bitową. W przypadku kluczy pomocniczy P mamy jedną skrzynkę z 18-ma elementami, również wartość każdego elementu to 32 bity.
- S-bloki: S0[0..255], S1[0..255], S2[0..255], S3[0..255]
- P: P[0..17]

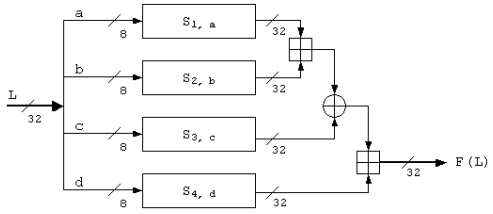
- dzielimy 64 bity tekstu jawnego "m" na xL (starsze 32 bity) i xR (młodsze 32 bity),
- wykonujemy operację sumy modulo 2 (xor) pierwszego klucza pomocniczego P1 z xL,
- wynik z punktu "c" poddajemy operacjom wynikającym w funkcji "F",
- wykonujemy operacje sumy modulo 2 wyniku z punktu "d" z xR,
- zamieniamy miejscami xL i xR,

- wracamy do punktu "c" wykonując xor z kolejnym kluczem pomocniczym Pi. Wykonujemy łącznie 16 iteracji licznika "i", tyle ile rund jest wymaganych w algorytmie Bruce Schneiera. Po 16-tym cyklu przechodzimy do następnego punktu kończąc działanie algorytmu, (UWAGA! Po 16-tym cyklu nie zamieniamy miejscami xL i xR !!! Nie wykonujemy punktu "f"),

- wykonujemy xor xR z P17 jako 17-tym kluczem pomocniczym i xor-ujemy xL z P18,
- łączymy xL i xR ze sobą w 64-ro bitowy blok otrzymując tym samym szyfrogram "c".

- Blowfish jest siecią Feistela, więc może on być w prosty sposób odwrócony poprzez Xor-owanie P17 i P18 wykonane na bloku zaszyfrowanego tekstu, a następnie wykorzystanie wszystkich pól z tablicy P w odwrotnej kolejności aby otrzymać oryginalny tekst

- Funkcja "F" wykonuje proste operacje arytmetyczne: suma modulo 2 oraz operacja dodawania modulo 2^32. Wejściem dla naszej funkcji jest 32-bitowa wartość. Zostaje ona rozłożona na cztery 8-bitowe wartości będące indeksami dla poszczególnych S-bloków. W algorytmie wykorzystujemy 4 S-bloki, każdy blok posiada 256 elementów.



Bezpieczeństwo algorytmu

- W typowych algorytmach jeśli długość klucza to k, a koszt zakodowania bloku to B, koszt ataku brute-force wynosi 2kB. W przypadku Blowfisha trzeba dla każdego klucza obliczyć S-boxy, co zajmuje tyle co zakodowanie ok. 29 bloków, a więc czas ataku brute-force wynosi około 2k + 9B (a zatem atak na 64-bitowy Blowfish zajmuje mniej więcej tyle czasu co na 73-bitowy bardziej tradycyjny szyfr). Wadą tego rozwiązania są dość duże wymagania pamięciowe – potrzebne są ponad 4 kB pamięci, co nie jest problemem dla nawet słabych komputerów, ale jest już dla np. kart chipowych.

- Nie istnieją znane ataki na Blowfisha o ilości rund większej niż 4. Są znane, dość duże jak na symetryczny szyfr blokowy, grupy słabych kluczy, czyli takich, dla których Blowfish jest słabszy niż dla typowych kluczy (większość szyfrów posiada takowe, jednak szansa na wylosowanie takiego klucza jest bardzo niska).

Skipjack

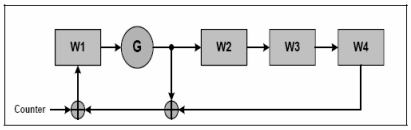
- algorytm zaproponowany przez NSA do użytku w szyfrujących układach elektronicznych Clipper i Capstone. Układy te wyposażono jednak w mechanizm umożliwiający odpowiednim służbom rozkodowanie informacji, teoretycznie po uzyskaniu zezwolenia sądowego.

- opracowywany w latach 1987-1993 przez NSA (National Security Agency) jako narzędzie do zapewniania bezpiecznej komunikacji głosowej oraz do wymiany danych z jednoczesnym umożliwieniem kontroli i podsłuchu przez uprawnione agencje rządu USA. Urządzenia działające w systemie SKIPJACK (np. telefony) mają zainstalowany odporny na ingerencje układ scalony. Dystrybucja kluczy odbywa się ręcznie.

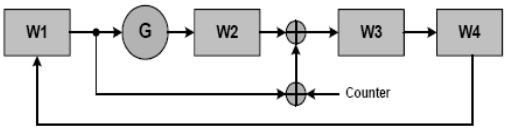
- SKIPJACK szyfruje 64-bitowe bloki w 32 etapach za pomocą 80-bitowego klucza. Zakładając, że dysponuje się hipotetyczną maszyną złożoną z 1, 2 miliarda jednodolarowych układów scalonych z zegarem 1 GHz i szyfrowaniem potokowym w jednym cyklu zegara zakres kluczy SKIPJACK wyczerpałby się po upływie roku.

Słowo 64 bitowe dzielone jest na cztery 16 bitowe części (w1, w2, w3, w4).
Są one przetwarzane 8x wg schematu A, 8x wg schematu B i ponownie 8x wg schematu A, 8x wg schematu B, w sumie 32 rundy

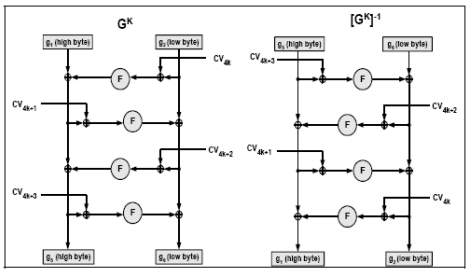
Schemat A



Schemat B



Permutacja G



SAFER

- **Jest algorytmem rozwiniętym przez J.L. Masseya (jednego z współtwórców IDEA). Twierdzi się że zapewnia bezpieczne szyfrowanie przy dość szybkiej implementacji nawet na 8 bitowych procesorach. Dostępne są dwie odmiany, jedna dla kluczy 64 bitowych, a druga dla 128 bitowych.**

Szyfry plecakowe

- szyfry Merklego - Helmana i Grahama - Shamira. Szyfry te mogą być użyte do ukrycia treści wiadomości, lecz nie do kontroli autentyczności. Wynika to z faktu, że przekształcenie deszyfrujące nie odwzorowuje całej przestrzeni wiadomości na siebie (przestrzenie *M* i *C* nie są równoważne). *Przekształcenie tajne nie może być użyte jako przekształcenie szyfrujące.*

- **Szyfr Merklego - Hellmana.**
-
- Bezpieczeństwo tego szyfru opiera się na trudności rozwiązania następującego dwójkowego zagadnienia plecakowego: mamy całkowitą liczbę dodatnią C i wektor dodatnich liczb całkowitych A = (a1, ... , an). Należy znaleźć podzbiór elementów wektora A, których suma wynosi C. Innymi słowy należy znaleźć wektor dwójkowy M = (m1, ... , m2) spełniający równanie
-
- C = A M .

- PRZYKŁAD.

  Niech n = 5, C = 14 i A = (1,10,5,22,3).
  Wówczas M = (1,1,0,0,1).

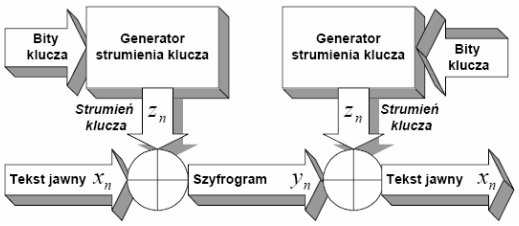
Wykład 6

Szyfry strumieniowe

Szyfry strumieniowe

- Szyfry strumieniowe stanowią klasę szyfrów z kluczem symetrycznym.
- Ich zasada działania polega na szyfrowaniu każdego znaku tekstu jawnego osobno, używając transformacji szyfrującej, która może być zależna od czasu oraz różna dla każdego ze znaków tekstu jawnego. Transformacja szyfrująca może również zależeć od wartości jednostek tekstu jawnego szyfrowanych wcześniej, co oznacza, że grupa szyfrów strumieniowych może charakteryzować się pamięcią.

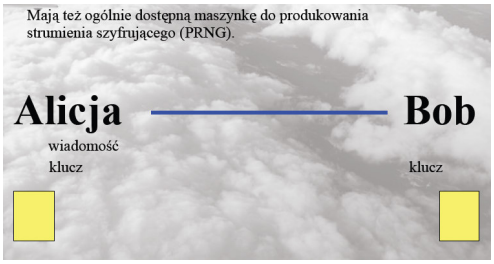
Binarny addytywny szyfr strumieniowy

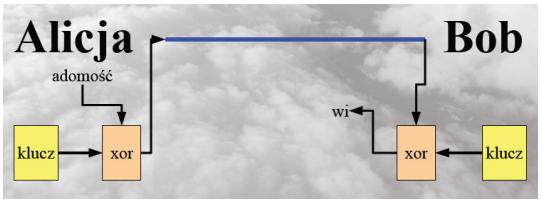


- W dokumentach NSA (National Security Agency) czasami używa się terminu “combiner-type algorithms” w odniesieniu do algorytmów, które używają jakiejś funkcji do połączenia wyjścia z pseudolosowego generatora liczb (PRNG) z niezaszyfrowaną wiadomością.

- PRNG – pseudorandom number generator. Algorytm, który generuje ciąg liczb, którego elementy są w przybliżeniu losowe.

- Generator strumienia szyfrującego to podstawa bezpieczeństwa szyfrów strumieniowych. Ważne jest aby generator produkował ciąg w dużym przybliżeniu losowy, nie mówiący nic o wykorzystywanym kluczu i o możliwie długim okresie, czyli liczbie iteracji, po której ciąg zaczyna się powtarzać.





- Szyfry strumieniowe są bardziej odpowiednie do zastosowań z ograniczoną możliwością buforowania lub tam gdzie porcje danych muszą być od razu, jak tylko się pojawią, przetwarzane.

- W szyfrach strumieniowych nie występuje propagacja błędów lub ma ona znikomy zasięg.
- Dzięki tej własności szyfry strumieniowe mogą okazać się bardziej odpowiednie od blokowych w sytuacjach transmisji o dużym prawdopodobieństwie błędów.

- Szyfry strumieniowe są używane tam, gdzie dane pojawiają się w nieznanej długości porcjach

- Szyfry blokowe w takich sytuacjach byłyby nieodpowiednie bo transmitowane byłyby niepotrzebne porcje danych – dopełnienia właściwych danych do pełnego bloku.

Wśród szyfrów strumieniowych wyróżnić można synchroniczne szyfry strumieniowe oraz asynchroniczne szyfry strumieniowe.

Synchroniczne szyfry strumieniowe

- Synchroniczne szyfry strumieniowe są szyframi, w których strumień klucza jest generowany niezależnie od zawartości tekstu jawnego oraz szyfrogramu.
- Jest potrzebna synchronizacja tzn. nadawca i odbiorca muszą być zsynchronizowani.

wada

- Podatność na ataki zmiany bitów.

Zmiana bitu w szyfrogramie powoduje zmianę bitu w rozszyfrowanej wiadomości.

- Podatność na ataki wstawiania i usuwania bitów szyfrogramu – utrata synchronizacji.

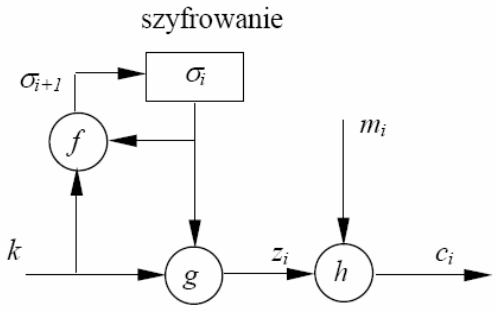
$$\sigma_{i+1} = f(\sigma_i, k),$$
$$z_i = g(\sigma_i, k),$$
$$c_i = h(z_i, m_i)$$

Synchroniczne szyfry strumieniowe

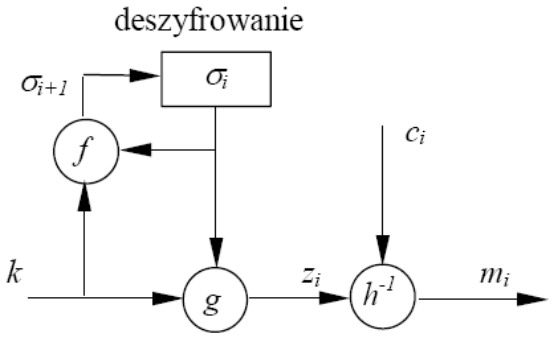
σ_i jest stanem początkowym, który może, ale nie musi zależeć od wartości klucza k, f jest funkcją opisującą stan kolejny, g jest funkcją generującą strumień klucza, zaś h jest funkcją łączącą strumień klucza z_i wraz z tekstem jawnym m_i w celu wytworzenia szyfrogramu c_i.

$$\sigma_{i+1} = f(\sigma_i, k),$$
$$z_i = g(\sigma_i, k),$$
$$c_i = h(z_i, m_i)$$

Model synchronicznego szyfru strumieniowego



Model synchronicznego szyfru strumieniowego



Jeśli transformacją h łączącą strumień klucza wraz ze strumieniem tekstu jawnego jest funkcja XOR, wówczas błędy bitowe w pewnych pozycjach szyfrogramu, będą miały wpływ tylko na odpowiednie pozycje tekstu jawnego.

Nie ma zatem w tym przypadku propagacji błędów.

Wadą tego rozwiązania jest konieczność zapewnienia pełnej synchronizacji pomiędzy nadawcą a odbiorcą. W przypadku jej braku odtworzenie tekstu jawnego stanie się dla odbiorcy niemożliwe.

Dla zminimalizowania wpływu utraty synchronizacji, stosuje się różne techniki,

np.: sekwencje inicjujące, numerację ramek itp.

Podsumowanie

Ciąg bitów klucza generowany jest niezależnie od szyfrowanej wiadomości i kryptogramu.
Musi być zachowana synchronizacja pomiędzy nadawcą i odbiorcą.
Zmiana bitu kryptogramu (przekłamanie) nie wpływa na możliwość deszyfrowania pozostałych bitów.
Dodanie lub usunięcie bitu powoduje utratę synchronizacji.
Istnieje możliwość zmiany wybranych bitów kryptogramu, a co za tym idzie zmiany deszyfrowanej wiadomości.

Asynchroniczne szyfry strumieniowe

Asynchroniczne szyfry strumieniowe, nazywane także samo-synchronizującymi szyframi strumieniowymi, należą do grupy szyfrów, w których generowany strumień klucza jest zależny od wartości klucza oraz od wartości szyfrogramów poprzednich.

Liczba szyfrogramów wpływających na wartość strumienia klucza, musi być stała w obszarze działania całego szyfru.

- Generator strumienia szyfrującego używa kilku poprzednich bitów szyfrogramu przy generowaniu kolejnych elementów ciągu.

- Pojedynczy błąd rozprzestrzenia się na kilka kolejnych elementów szyfrogramu.
- Powrót do poprawnego deszyfrowania po wstawieniu lub usunięciu bitów (czyli po stracie synchronizacji) jest automatyczny. Z ustaloną ilością niemożliwych do odzyskania bitów wiadomości.

Asynchroniczne szyfry strumieniowe

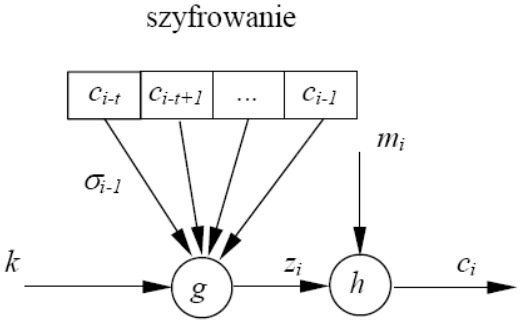
$$\sigma_i = f(c_{i-t}, c_{i-t+1}, c_{i-t+2}, \ldots, c_{i-1}),$$

$$z_i = g(\sigma_i, k),$$

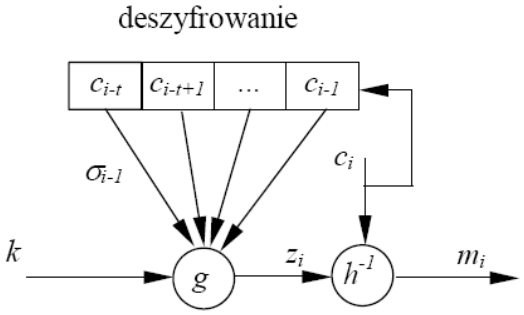
$$c_i = h(z_i, m_i)$$

σ_i jest stanem szyfru w *i-tej* chwili, f jest funkcją opisującą wpływ szyfrogramów na stan szyfru, g jest funkcją generującą strumień klucza, zaś h jest funkcją łączącą strumień klucza z_i wraz z tekstem jawnym m_i w celu wytworzenia szyfrogramu c_i.

Model asynchronicznego szyfru strumieniowego



Model asynchronicznego szyfru strumieniowego



Asynchroniczne szyfry strumieniowe

Asynchroniczne szyfry strumieniowe są szyframi z pamięcią. Stanowi ją bufor dla szyfrogramów stanów poprzednich.

Implementując tego rodzaju szyfry pojawia się problem polegający na wygenerowaniu strumienia klucza w pierwszych *t-chwilach*.

Jedną z technik rozwiązania tego problemu może być użycie dla pierwszych *t-chwil* wektora inicjalizującego *IV*.

Asynchroniczne szyfry strumieniowe

Jedną z podstawowych własności tych szyfrów jest samosynchronizacja. Szyfry te po utracie synchronizacji zdolne są do jej automatycznego przywrócenie, ponieważ proces deszyfrowania zależny jest tylko od *t* stanów poprzednich, a nie jak w przypadku szyfrów synchronicznych od wszystkich stanów.

Asynchroniczne szyfry strumieniowe

Zależność od *t* stanów poprzednich korzystnie wpływa również na propagację błędów, która jest w tym przypadku ograniczona do *t* szyfrogramów. Kolejną zaletą są dobre własności statystyczne. Ponieważ każdy bit tekstu jawnego wpływa na wartość szyfrogramu, statystyczne własności tekstu jawnego ulegają rozrzuceniu w obrębie wszystkich szyfrogramów, dlatego też szyfry te są odporniejsze na ataki bazujące na redundancji tekstu jawnego.

Podsumowanie

Utrata lub dodanie bitu w kryptogramie powoduje utratę tylko kawałka wiadomosci - samosynchronizacja.
Ograniczona propagacja błędów.
Zmiana bitu kryptogramu powoduje, że kilka innych bitów będzie deszyfrowanych błędnie — łatwiej wykryć taką zmianę.
Jednak na skutek samosynchronizacji wykrycie zmian w kryptogramie jest trudniejsze (jeśli zmiany dotyczą tylko cześci kryptogramu, to dalsza cześc jest deszyfrowana poprawnie).

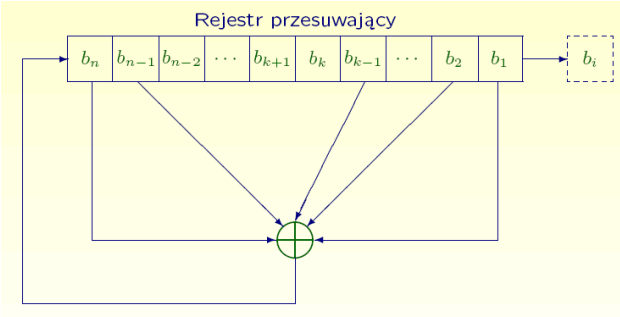
Budowa szyfrów strumieniowych

Podstawowym elementem używanym do generowania strumienia klucza są rejestry liniowe ze sprzężeniem zwrotnym LFSR. Ich zalety to:

- łatwe w implementacji,
- mogą generować sekwencje pseudolosową o stosunkowo długim okresie,
- generowane przez nie sekwencje mają bardzo dobre własności statystyczne,
- z uwagi na ich strukturę, możliwa jest ich analiza za pomocą technik algebraicznych.

Budowa szyfrów strumieniowych

Za pomocą algorytmu Berlekampa-Massey można dokonać analizy pracy rejestru LFSR, wyznaczyć wartość wielomianu definiującego sprzężenia oraz okresu. W praktyce oznacza to możliwość łatwej predykcji wartości bitów strumienia klucza. Możliwe jest jednak połączenie wyjść kilku rejestrów za pomocą określonej funkcji boolowskiej i a jej pomocą generowania strumienia klucza, użycia wyjścia jednego z rejestrów w celu taktowania innego oraz wiele innych kombinacji. Dzięki takim zabiegom drastycznie zwiększa się odporność na ataki oparte na korelacjach, oraz na opisie matematycznym pracy szyfru.

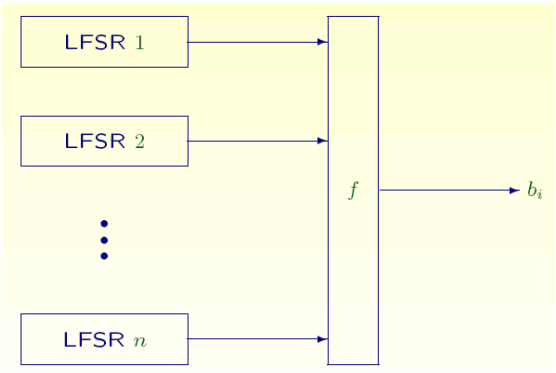


Ponieważ n-bitowy rejestr może znaleźć się w jednym z $2^n - 1$ stanów, więc teoretycznie może on generować ciąg o długości $2^n - 1$ bitów. Potem ciąg się powtarza.
(Wykluczamy ciąg samych zer, który daje niekończący się ciąg zer)

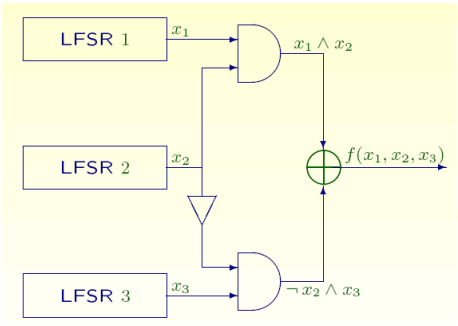
- LFSR ma słabą wartość kryptograficzną gdyż znajomość 2n kolejnych bitów ciągu pozwala na znalezienie wartości generowanych od tego miejsca.
- LFSR działa jednak bardzo szybko, zwłaszcza jeśli jest to układ hardware'owy, i stąd jest on bardzo atrakcyjny w praktycznych zastosowaniach.

Można konstruować bardziej skomplikowane układy zawierające kilka LFSR i nieliniowa funkcje f przekształcającą bity generowane przez poszczególne LFSR.

Układ kilku LFSR i nieliniowej funkcji f



Przykład: Generator Geffe



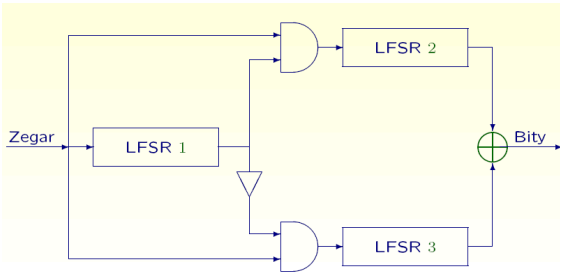
$$f(x_1, x_2, x_3) = (x_1 \wedge x_2) \oplus (\neg x_2 \wedge x_3)$$

Przykład: Generator Geffe

- Generator Geffe ma słabe własności kryptograficzne ze względu na korelacje pomiędzy generowanymi bitami i bitami LFSR 1 lub LFSR 2

Generatory sterowane zegarem

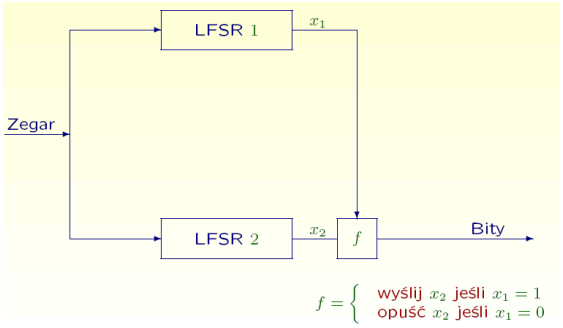
- Generator o zmiennym kroku, przemienny Stop-and-Go alternating step generator, Stop-and-Go



Generatory sterowane zegarem

- LFSR 1 jest przesuwany w każdym takcie zegara.
- Jeśli na wyjściu LFSR 1 jest 1 to LFSR 2 jest przesuwany; LFSR 3 nie jest przesuwany (poprzedni bit jest powtarzany).
- Jeśli na wyjściu LFSR 1 jest 0 to LFSR 3 jest przesuwany; LFSR 2 nie jest przesuwany (poprzedni bit jest powtarzany).
- Wyjściowe bity LFSR 2 i LFSR 3 są dodawane modulo 2 () dając kolejny bit generowanego ciągu.

Generator obcinający (shrinking generator)



$$f = \begin{cases} \text{wyślij } x_2 \text{ jeśli } x_1 = 1 \\ \text{opuść } x_2 \text{ jeśli } x_1 = 0 \end{cases}$$

Generatory, których bezpieczeństwo oparte jest na trudnościach obliczeniowych

- Generator Blum-Micali

- W generatorze tym wykorzystuje się trudność w obliczaniu logarytmu dyskretnego. Wybieramy dwie liczby pierwsze a i p oraz liczbę x_0 (zarodek), a następnie obliczamy

$$x_{i+1} = a^{x_i} \pmod{p} \text{ dla } i = 1, 2, 3, \ldots$$

- Generator Blum-Micali

- Pseudolosowy ciąg bitów tworzymy w następujący sposób:

$$k_i = \begin{cases} 1 & \text{jeżeli } x_i < (p-1)/2 \\ 0 & \text{w przeciwnym przypadku} \end{cases}$$

Generator RSA

- Generator oparty na trudności z faktoryzacją liczb.

- Wybieramy dwie liczby pierwsze p i q ($N = pq$) oraz liczbę e względnie pierwszą z $(p-1)(q-1)$. Wybieramy losową liczbę (zarodek) x_0 mniejszą od N, a następnie obliczamy

$$x_{i+1} = x_i^e \pmod{N}$$

- generowanym bitem jest najmłodszy bit x_i

Generator Blum-Blum-Shub — BBS

- Znajdujemy dwie duże liczby pierwsze p i q, takie, że $p \equiv 3 \pmod 4$ oraz $q \equiv 3 \pmod 4$; $N = pq$.
- Wybieramy losową liczbę x względnie pierwszą z N, a następnie obliczamy

$$x_0 = x^2 \pmod{N}$$

x_0 stanowi zarodek dla generatora.

- Liczymy

$$x_{i+1} = x_i^2 \pmod{N}$$

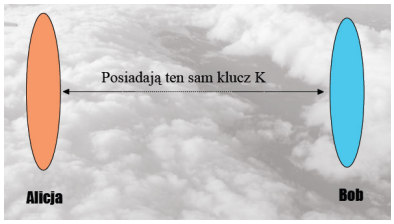
- Generowanym bitem k_i jest najmłodszy bit x_{i+1}.

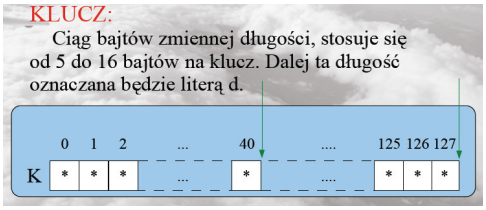
Generator RC 4

- Generator RC 4 został opracowany przez Rona Rivesta w 1987 r. Przez kilka lat był to algorytm tajny. W 1994 r. został opublikowany w Internecie program realizujący ten algorytm. Od tego czasu algorytm nie stanowi tajemnicy.
- Algorytm ten pracuje w trybie OFB (Output Feedback).
- Ciąg generowany przez RC 4 jest losowym ciągiem bajtów.

- Nazwa została opatentowana, nie można jej nielegalnie używać, sam algorytm jednak może być wykorzystywany
  - Unika się problemów z odnośnikami stosując powszechnie nazwę ARCFOUR
  - Stał się powszechnie stosowany (WEP, WPA, SSL)

RC4 założenia

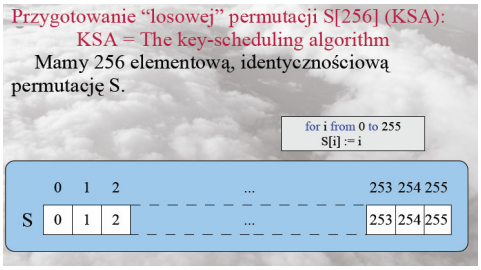


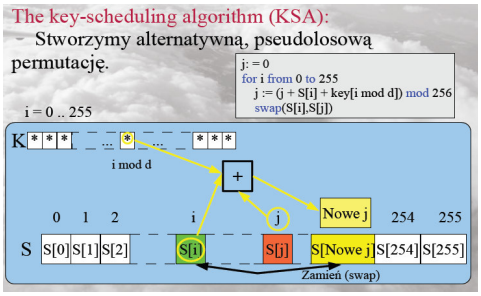


- Siła algorytmu polega na tym, że za pomocą prostej metody z wprowadzonego klucza S wytwarzany jest długi, pseudolosowy klucz wewnętrzny P. Dopiero tego klucza RC4 używa do szyfrowania otwartego tekstu. Jeżeli klucz S składa się z n bitów od S(0) do S(n-1), wówczas rozpoczyna się obliczanie:

- i, j = 0
  P[k] = k z k=0, ... ,256
  powtarzane 256 razy
  j = j + P[i] + S[i] mod 256
  zamień P[i] i P[j]
  i = i + 1 mod n
  Bit klucza K[i] niezbędny do zaszyfrowania lub odszyfrowania bitu wiadomości i oblicza się według wzoru:

  i = i + 1 mod 256
  j = j + P[i] mod 256
  zamień P[i] i P[j]
  t = P[i] + P[j] mod 256
  K[i] = P[t]

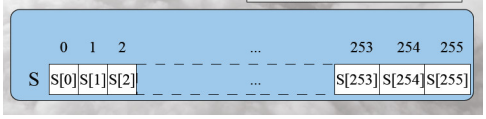




The key-scheduling algorithm (KSA):

Stworzyliśmy więc z S oraz naszego klucza, alternatywną permutację.

```
j: = 0
for i from 0 to 255
  j := (j + S[i] + key[i mod l]) mod 256
  swap(S[i],S[j])
```



Wygenerowanie za pomocą S dowolnej długości "losowego" ciągu bitów (PRGA):

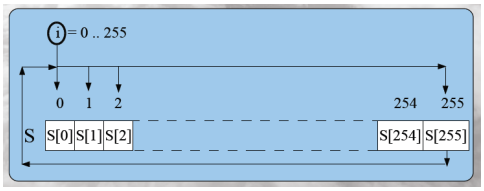
PRGA = The pseudo-random generation algorithm

Tak długo jak zachodzi potrzeba generujemy kolejne szyfrowe bajty, którymi potem będą szyfrowane kolejne bajty wiadomości.

```
i := 0
j := 0
while GeneratingOutput:
  i := (i + 1) mod 256
  j := (j + S[i]) mod 256
  swap(S[i],S[j])
  output S[(S[i] + S[j]) mod 256]
```

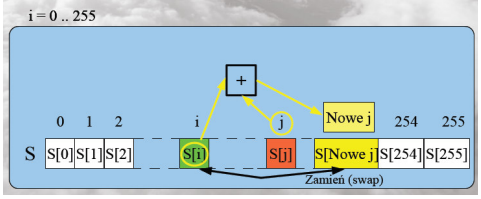
The pseudo-random generation algorithm (PRGA):

- początkowo zmienne i oraz j ustawiamy na 0
- i będzie przechodziło cyklicznie po tablicy S

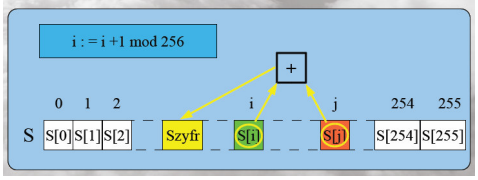


The pseudo-random generation algorithm (PRGA):

- nowe j tworzymy podobnie jak w KSA



Operacja ta jest kontynuowana tak długo jak "losowe" bajty są potrzebne do kolejnych znaków wiadomości.



Generator RC 4

- Algorytm używa dwóch wskaźników i, j przyjmujących wartości $0, 1, 2, \ldots, 255$ oraz S-boksu z wartościami $S_0, S_1, \ldots, S_{255}$, które tworzą permutację liczb $0, 1, \ldots, 255$.
- Inicjalizacja: Na początku $i = j = 0$, $S_l = l$ dla $l = 0, 1, \ldots, 255$, kolejna 256-bajtowa tablica wypełniana jest bajtami klucza, przy czym klucz jest używany wielokrotnie, aż do wypełnienia całej tablicy $K_0, K_1, \ldots, K_{255}$.

Generator RC 4

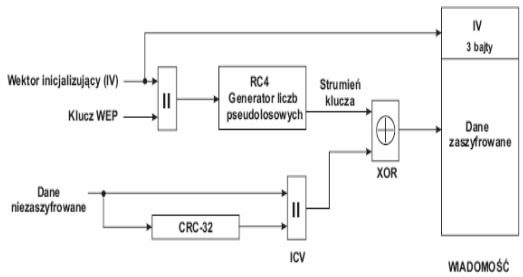
- Następnie wykonujemy:
  for $i = 0$ to 255:
  $j = (j + S_i + K_i) \pmod{256}$
  zamień S_i z S_j
- Generowanie kolejnego bajtu:
  $i = i + 1 \pmod{256}$
  $j = j + S_i \pmod{256}$
  zamień S_i z S_j
  $l = S_i + S_j \pmod{256}$
  $K = S_l$

Generator RC 4

- Otrzymany bajt K jest dodawany modulo 2 (xor) z kolejnym bajtem wiadomości dając kolejny bajt kryptogramu (przy deszyfrowaniu role tekstu jawnego i kryptogramu się zamieniają).

- Algorytm RC 4 jest używany w wielu programach komercyjnych.
- algorytm ten doskonale nadaje się do implementacji w oprogramowaniu, bardzo szybko zastosowano go w wielu produktach komercyjnych, m.in w Lotus Notesie, Oracle Secure SQL i Netscape Nawigatorze.

WEP



- nie należy używać wielokrotnie tego samego klucza
- wiadomość tym samym kluczem będzie tak samo zakodowana
- algorytm jest w istocie deterministyczny z pktu widzenia Alicji i Boba
- istnieją algorytmy, które mając GB szyfrogramu potrafią rozróżnić RC4

- Nieprawidłowe użycie RC4 w Word i Excel
- ● Szyfr ma być nie tylko dobry
- ● Musi być poprawnie użyty!!!
- – Wpadka w implementacji SSL w Netscape 1.1
- – Niedociągnięcia w obsłudze RC4 w MS Office

Rodzaj błędu w MS Word

- ● Tworzymy dokument i zapisujemy go zabezpieczając hasłem
- ● Dokonujemy modyfikacji w dokumencie i zapisujemy do innego pliku
- ● ZOSTAŁ UŻYTY TEN SAM WEKTOR INICJALIZUJĄCY
- ● Mamy dwa szyfrogramy, które niewiele się różnią
- ● Można z nich wyciągać informacje o dokumencie

- 128 bitowy klucz w RC4 już bezpieczny

– Jednak źle zarządzany jest wektor inicjalizujący

- ● Nie jest przeszkodą ponowne używanie tego samego klucza
- – Klucz i wektor inicjalizujący są hashowane i dają w wyniku klucz używany przez RC4

A5 - przykład szyfru strumieniowego

A5 jest szyfrem strumieniowym używanym w sieciach GSM do uwierzytelniania i szyfrowania konwersacji abonenta pomiędzy telefonem a stacją bazową. A5 posiada dwa warianty. Pierwszym z nich jest A5/1 - silniejsza i bezpieczniejsza wersja, drugim - A5/2 - słabsza i mniej bezpieczna wersja. Zarówno pierwszy jak i drugi używane są w Europie.

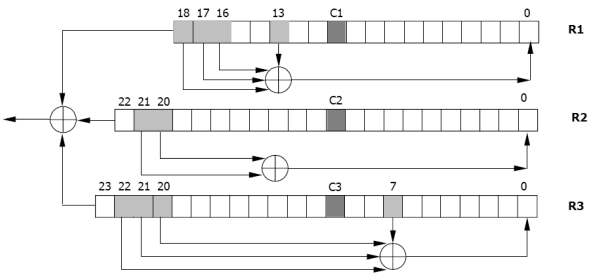
A5 - przykład szyfru strumieniowego

Rozmowa w GSM przesyłana jest jako sekwencja ramek. Każda ramka zawiera 114 bitów zarezerwowanych dla transmisji w kierunku od A do B, oraz 114 bitów zarezerwowanych dla transmisji w kierunku odwrotnym. Czas trwania ramki wynosi 4, 6ms. Numer każdej ramki -Fn jest ponad to zliczany za pomocą licznika o długości cyklu 222. Szyfrowanie rozmowy odbywa się z użyciem 64 bitowego klucza sesji - K, budowanego za pomocą funkcji haszującej z klucza głównego użytkownika i losowej wartości. Za generację klucza odpowiedzialny jest algorytm A8.

Proces szyfrowania A5

1. Dla każdej ramki 64 bitowy klucz K jest nieliniowo łączony z 22 bitową wartością licznika ramek Fn.
2. Rezultat powyższej operacji stanowi wektor inicjujący generatora pseudolosowego który generuje ciąg długości 228 bitów, który jest sumowany modulo 2 z 228 bitami tekstu jawnego w celu wytworzenia szyfrogramu.

Schemat układu szyfrującego za pomocą algorytmu A5



Proces szyfrowania A5

Układ zbudowany jest z trzech liniowych rejestrów cyklicznych. ze sprzężeniem zwrotnym.

Pierwszy rejestr - R1 ma długość 19 bitów, drugi - R2 - długość 22 bity, zaś trzeci - R3 - długość 23 bitów. Najmłodszy bit każdego z rejestrów jest oznaczony jako zero.

Proces szyfrowania A5

Każdy z trzech rejestrów tworzy generator sekwencji pseudolosowej. Oprócz tego, że rejestry są różnej wielkości, mają również różne punkty wyprowadzeń sygnałów sprzężenia zwrotnego. Rejestr R1 ma je pozycjach 13; 16;17;18, rejestr R2 w pozycjach 20 i 21, rejestr R3 w pozycjach 7,20,21,22. Punkty te zostały tak dobrane, aby okresy sekwencji pseudolosowych generowanych przez poszczególne rejestry były jak najdłuższe. Wynoszą one odpowiednio (2^19)-1, (2^22)-1 oraz (2^23)-1.

Proces szyfrowania A5

Taktowanie rejestrów odbywa się według następującej reguły start/stop:

Każdy z rejestrów posiada punkt wyprowadzenia sygnału sterującego taktowaniem Dla rejestru R1 jest to punkt C1, znajdujący się na pozycji 8, dla R2 - punkt C2 na pozycji10, oraz dla R3 - punkt C3 na pozycji 10. W każdym cyklu analizowana jest zawartość bitów C1, C2, C3. Jeżeli dwa z trzech bitów C mają wartość '1', taktowane są tylko te rejestry, w których bity C miały wartości '1'. Jeżeli dwa z trzech bitów C miały wartość '0', taktowane są tylko te rejestry, których bity C miały wartość '0'. Funkcją realizującą taktowanie jest więc funkcja większościową *(majority function)*. W każdym kroku taktowane są zatem przynajmniej dwa rejestry.

Proces szyfrowania A5

Proces generowania sekwencji pseudolosowej na podstawie klucza sesji - K , oraz wartości licznika ramki Fn realizowany jest w czterech krokach.

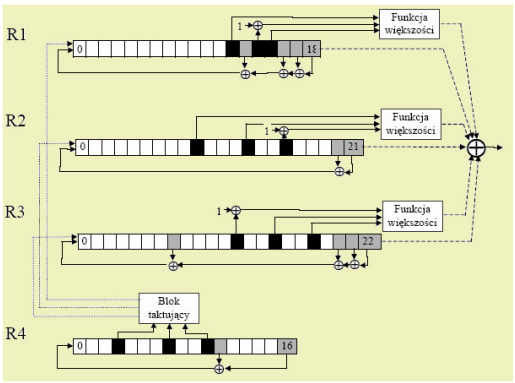
1.) Najpierw wszystkie rejestry są zerowane, a następnie taktowane są 64 cyklami zegarowymi z pominięciem reguły start/stop. W czasie tym, każdy bit klucza K (od LSB do MSB) jest wprowadzany równolegle , za pomocą xor do trzech rejestrów, w miejsce LSB.

2.) W kolejnych 22 cyklach zegara (również z pominięciem reguły start/stop) wprowadzane są 22 bity licznika ramki Fn, w sposób identyczny jak powyżej. Na końcu 22 cyklu zawartość trzech rejestrów tworzy wektor inicjujący ramki,

3.) Rejestry są taktowane 100 cyklami zegara z uwzględnieniem reguły start/stop. W czasie tym dane wyjściowe rejestrów są ignorowane.

4.) Rejestry są taktowane przez kolejne 228 cykli z uwzględnieniem reguły start/stop w celu wygenerowania 228 bitów wyjściowych. W każdym cyklu zegara generowany jest jeden bit, będący sumą xor trzech najstarszych bitów (MSB) rejestrów.

A5/2



Ataki i obserwacje A5

Implementacje A5 stanowiła tajemnicę operatorów GSM, jednak na początku 1998 Smartcard Developer Association opublikowała ich kody źródłowe zdobyte w wyniku *reverse-engineeringu*. Kryptoanaliza A5/2 została dokonana w 1998 roku przez Slobodana Pertovica,

Przeprowadził on atak na A5/2 opierający się na algebrze, o złożoności 2^17.

S.Petrovic, A.Fuster-Sabater, Cryptoanalysis of the A5/2 Algorithm, Crypto 98, Springer-Verlag 1998

Ataki i obserwacje A5

Na wstępie zakłada się, że atakujący kryptoanalityk zna wartości kilku pseudolosowych bitów generowanych przez A5/1 w różnych ramkach. Powyższa założenie stanowi fundamentalne założenie wstępne kryptoanalizy szyfrów strumieniowych.

Ataki i obserwacje A5

Zakłada się również, że atakujący ma do dyspozycji fragment sekwencji wyjściowej A5 - szyfrogram (będący fragmentem rozmowy) i jego celem jest znalezienie klucza sesji umożliwiającego odszyfrowanie pozostałej części rozmowy.

Ataki i obserwacje A5

Ponieważ w GSM każda ramka ma czas trwania równy 4,6ms zatem w jednej sekundzie przesyłanych jest około 220 ramek.

Bezpieczeństwo A5 zostało przeanalizowane w różnych publikacjach. Wyniki tej analizy można podsumować następująco:

Ataki i obserwacje A5

1. R.Anderson i M.Roe[1] zaproponowali atak bazujący na zgadywaniu 41 bitów z krótszych rejestrów - R1 oraz R2 oraz wyliczeniu 23 bitów R3 poprzez analizę wartości wyjściowej. Należy jednak wziąć pod uwagę wartości bitów funkcji większościowej, co powoduje, że łączna złożoność ataku wynosi 2^54. Zakładając, że standardowy komputer PC potrafi przetestować 10 mln bitów w sekundzie, atak ten wymaga ponad miesiąca czasu w celu odnalezienia klucza, co czyni jego bezużytecznym.

   [1] Zob. R. Anderson, M. Roe, A5, http://jya.com/crack-a5.htm, 1994.

Ataki i obserwacje A5

2. M.Briceno[1] analizując A5, odkrył, że w każdej z jego implementacji 10 najmłodszych bitów 64-bitowego klucza ma wartość zerową. Dzięki temu złożoność ataku z wyczerpaniem wszystkich możliwości klucza redukuje się z 2^64 do 2^54.

   [1] Zob. M. Briceno, I. Goldberg, D. Wagner, A pedagogical implementation of A5/1, http://www.scard.org/, May 1999.

Ataki i obserwacje A5

3. Podczas Eurocrypt 97. J.Golic przedstawił atak[1] typu „dziel i zwyciężaj”, mający na celu uzyskanie nieznanej wartości wektora inicjującego rejestrów poprzez wykorzystanie specyficznej reguły taktowania rejestrów oraz rozwiązanie systemu równań. Do przeprowadzenia ataku Golic potrzebował jedną parę tekstu jawnego i szyfrogramu. Atak charakteryzuje się złożonością ok. 2^40.

   [1] J.D. Golic, Cryptanalysis of Alleged A5 Stream Cipher, Eurocypt 97, Springer Verlag 1998

Ataki i obserwacje A5

4. Kolejnym atakiem przedstawionym przez J.Golica był atak zwany Time Memory Trade Off Attack. bazujący na probabilistycznym problemie urodzin. Golic pokazuje, że możliwe jest znalezienie klucza gdy TM>2^63, gdzie T jest czasem obliczeń, zaś M wielkością pamięci mierzoną w 128 bitowych słowach. Biorąc jednak pod uwagę możliwości współczesnych komputerów atak ten również jest bezużyteczny.

Ataki i obserwacje A5

5. W 1999 roku A.Biryukov, A.Shamir oraz R.Wagner[1] na podstawie powyższych obserwacji, oraz poprzez zauważenie, że A5/1 może zostać łatwo zaimplementowany na domowym PC, oraz poprzez zastosowanie algorytmu redukcji odczytów z dysku w kryptoanalizie J.Golica zaproponowali dwa nowe ataki, które mogą zostać przeprowadzone z użyciem komputera klasy PC.

[1] A.Biryukov, A.Shamir, R. Wagner, Real Time Cryptanalysis of the Alleged A5/1 on a PC, Proceedings of fast Software Encryption Workshop, NewYork 2000, Lecture Notes on Computer Science, Berlin, in press

Ataki i obserwacje A5

Celem tych ataków jest wydobycie klucza sesji. Pierwszy z nich, zwany *Biased Birthday Attack*, mając do dyspozycji 2 min rozmowy oraz komputer z macierzą dyskową o pojemności 150Gb potrafi wyciągnąć klucz w czasie 1 sekundy.

Drugi atak - *Random Subgraph Attack*, dokonuje ekstrakcji klucza w czasie jednej minuty, mając do dyspozycji PC z pojemnością dyskową 300Gb oraz 2 sekundy rozmowy. Obydwa ataki wymagają jednak obliczeń wstępnych, wykonywanych jednorazowo, w liczbie 2^48 kroków.

Szyfrowanie w GPRS

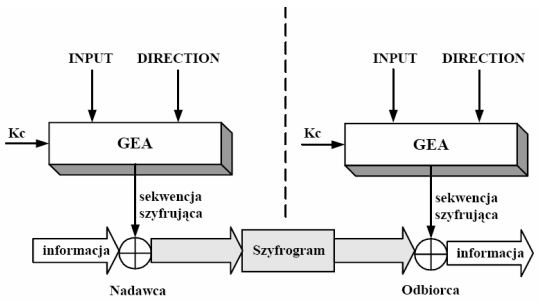
Proces szyfrowania w GPRS rozwiązany jest lepiej niż w systemie GSM. Standardowo ten proces odbywa się pomiędzy telefonem i węzłem **SGSN** (ang. *Serving GPRS Support Node*).

Algorytmem szyfrującym jest **GEA** (ang. *GPRS Encryption Algorithm*) jest podobny do A5 i zawiera pewne poprawki. Siła algorytmu jest nie znana, ponieważ nie został opublikowany, dlatego nie należy mu zbytnio ufać.

Szyfrowany strumień danych z jednej strony oraz deszyfrowany strumień z drugiej strony muszą być synchronizowane. Synchronizacja jest zapewniona przez zmienne INPUT oraz DIRECTION.

Wartości:

- INPUT – jest sekwencyjnym numerem pakietu i jego wartość początkowa jest ustalana przez sieć.
- DIRECTION – jest wartością wskazującą na kierunek transmisji.



- Dla systemu GSM zdefiniowano 7 algorytmów A5, w tym:
  - A5/1 – dla członków CEPT
  - A5/2 – dla operatorów spoza CEPT
  - A5/3 – najnowsza procedura (2001)
- Dla GPRS zdefiniowani również 7 algorytmów A5, w tym:
  - GEA (GPRS Encription Algorithm)
  - GEA-2 – najnowsza procedura (2001)
- Algorytmy są zestandaryzowane, jednak nie są ogólnie dostępne
- Przed rozpoczęciem szyfrowania negocjowany jest rodzaj stosowanego algorytmu szyfrowania:
  - jeżeli brak wspólnego algorytmu:
    - transmisja nieszyfrowana –jeżeli sieć obsługuje połączenia nieszyfrowane,
    - rozłączenie – jeżeli sieć nie obsługuje połączenia nieszyfrowane,
  - jeżeli jest choć jeden wspólny algorytm A5 to jest on stosowany

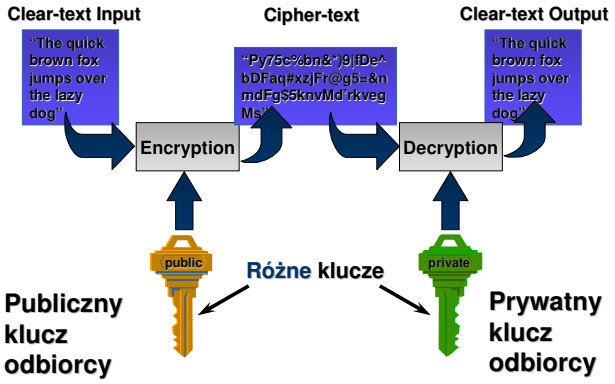
Wykład 7

Systemy kryptograficzne z kluczem publicznym

Kryptografia publiczna (asymetryczna)

- Wiedza o kluczu szyfrującym nie pozwala odgadnąć klucza deszyfrującego
- Odbiorca informacji generuje parę kluczy
  - **I publikuje swój klucz publiczny w jakimś katalogu**
- Zatem każdy może jej posyłać zaszyfrowane dane

Szyfrowanie publiczne (asym)



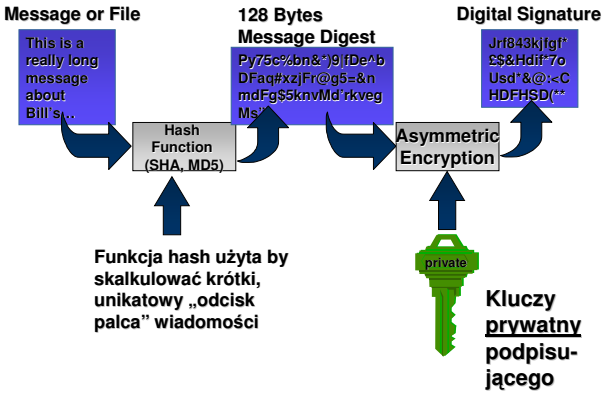
Problem klucza publicznego

- Rozwiązaliśmy problem przekazania klucza
- Ale…
- Scott tworzy parę kluczy i daje wszystkim swój klucz publiczny mówiąc, że należy do Billa
- Ludzie szyfrują tajne dane dla Billa
- Bill nie ma klucza prywatnego by to odszyfrować…
- Scott czyta wiadomości posłane do Billa ☹

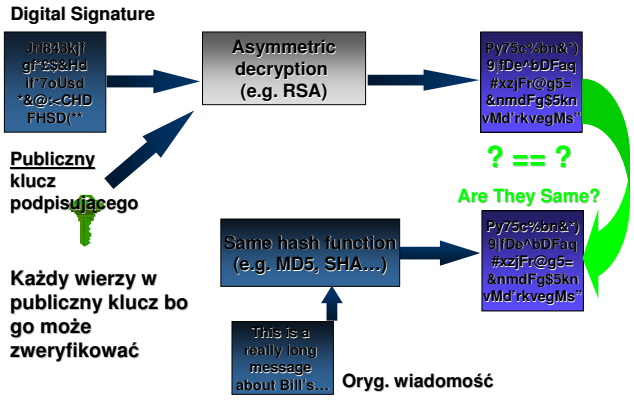
Jak zweryfikować klucz publiczny?

- Dwa podejścia:
  1. **Zadzwoń do Billa zanim użyjesz jego klucza i sprawdź**
     - **Fingerprint lub jakaś suma kontrolna**
  2. **Poproś kogoś komu już ufasz by zcertyfikował prawdziwość klucza Billa**
     - **Niech się oni cyfrowo podpiszą pod kluczem Billa**
     - **Ale musisz komuś ufać…**

Podpisujemy cyfrowo



Weryfikacja podpisu cyfrowego



Certyfikaty

- Najprostszy to:
  - **Informacja o właścicielu i**
  - **Jego klucz publiczny**
- To wszystko jest podpisane przez CA, któremu ufamy

Weryfikacja autentyczności

1. Melinda dostaje certyfikat Billa
2. Sprawdza podpis na certyfikacie
   - **Więc wierzy w certyfikat**
   - **Ale czy przed nią stoi Bill czy Scott**
3. Melinda prosi by Bill zaszyfrował losowo wybraną frazę (*"I really need more shoes"*)
4. Bill ma swój prywatny klucz więc odpowiada (*"*&$^%£$&£fhsdf*&EHFDhd62^&£"*)
5. Melinda odszyfrowuje i porównuje odpowiedź – jako że się zgadza, wie że przed nią jest jedyny właściciel klucza prywatnego czyli Bill ☺

- Tak działa SSL...

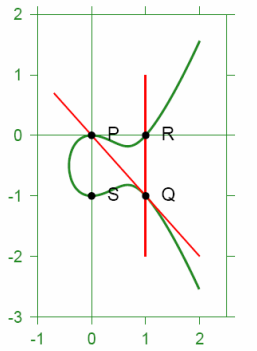
Typowe rekomendacje mocy (May 2003)

	Minimum	Lepiej
Symmetric Key	96 bits (avoid DES as it can do only 56, instead use AES-Rijndael or RC5)	256 bits (Rijndael, RC5 128bits, not DES)
Asymmetric Key	1024 (RSA)	4096 (RSA)
ECC Key	192 bits	256 bits
Hash: SHA/MD5	128 bits (absolutely not 64 bits)	256 bits or more

- **Szyfr na krzywych eliptycznych** (ang. Elliptic Curve Cryptography, **ECC**) - algorytm wykorzystujący krzywe eliptyczne w celu szyfrowania informacji

- Krzywe eliptyczne są obiektem matematycznym pozwalającym w łatwy sposób na generację dużej liczby nietrywialnych grup abelowych. Elementami takich grup są punkty krzywej, zaś jako działanie służy pewna raczej złożona operacja arytmetyczna wymierna na ich współrzędnych. W tak zdefiniowanych grupach może występować wielka, choć skończona ilość elementów (rząd grupy może być skończony) jeśli zawęzić operacje do np. punktów o współrzędnych wymiernych, lub jeśli całą krzywą rozpatrywać wyłącznie w ramach arytmetyki modulo skończona liczba pierwsza p.

krzywa eliptyczna $y^2+y = x^3-x^2$



- Znaczenie kryptografii opartej o krzywe eliptyczne wzrasta gdyż przy tej samej długości klucza, z powodu złożoności arytmetyki grupowej, oferuje ona większe bezpieczeństwo (jej złamanie wymaga większej ilości obliczeń) w porównaniu z np. RSA.

- Odwrotnie, to samo bezpieczeństwo osiągamy przy mniejszej długości klucza niż w innych przypadkach szyfrowania asymetrycznego. Tym samym implementacja algorytmów szyfrujących w oparciu o krzywe eliptyczne dobrze nadaje się do użycia w np. kartach chipowych.

- algorytmy starsze, takie jak RSA są lepiej przebadane oraz znacznie wydajniejsze (szybsze). Rzeczywiście porównanie szybkości szyfrowania wskazuje, ze DES (a więc szyfr symetryczny) jest ok. 100 razy szybszy niż RSA (asymetryczny), który jest ok. 10 razy szybszy niż kryptografia eliptyczna.

- Prowadzone są prace nad standaryzacją szyfrowania opartego na krzywych eliptycznych oraz prace rozwojowe nad rozwojem uogólnienia tego typu szyfrów na krzywe hipereliptyczne.

- Szczególne miejsce w tych pracach zajmuje określenie jakie krzywe eliptyczne nadają się do stosowania w ramach algorytmów kryptograficznych znane są bowiem pewne ograniczenia czyniące szyfrowanie eliptyczne nieprzydatnym (łatwym do złamania) jeśli zostanie ono zbudowane w oparciu o niewłaściwą krzywą.

Przykład

- Alicja chce przesłać do Boba wiadomość wraz z potwierdzeniem autentyczności. W tym celu na podstawie funkcji haszującej oblicza podpis wiadomości, na jego podstawie i przy użyciu swojego klucza ECC wyznacza dwie liczby r i s.
- Wysyła do Boba wiadomość i te dwie liczby.

- Bob na podstawie r i s oraz klucza publicznego Alicji i wyznaczonej przez siebie wartości funkcji haszującej oblicza wartość v.
- Porównuje r i v, gdy są równe ma pewność, że otrzymał nienaruszoną wiadomość od Alicji.

- **Grupa** – jedna z prostszych struktur algebraicznych: niepusty zbiór, na którym określono tylko jedno działanie dwuargumentowe.

- **Grupą** nazywamy parę uporządkowaną, $(G, \star)$
- gdzie *G* jest dowolnym niepustym zbiorem, zaś * działaniem dwuargumentowym spełniającym następujące warunki:

1. $\forall_{a,b,c \in G}\ (a \star b) \star c = a \star (b \star c)$ zapewniający łączność działania
2. $\exists_{e \in G}\ \forall_{a \in G}\ e \star a = a \star e = a$, gdzie e nazywamy **elementem neutralnym** działania,
3. $\forall_{a \in G}\ \exists_{b \in G}\ a \star b = b \star a = e$, gdzie b nazywamy **elementem odwrotnym** do elementu a.

- **Grupa przemienna (abelowa)** – grupa, w której działanie jest przemienne.
- Nazwa *abelowa* pochodzi od nazwiska Nielsa Abela, norweskiego ***matematyka***, w którego pracach *implicite* pojawiło się to pojęcie.

- Grupę $(G, +)$ nazywamy **abelową**, gdy dla każdych $a, b \in G$:

  a + b = b + a.

- **Grupa cykliczna** – grupa, której wszystkie elementy są potęgami pewnego elementu grupy.
- Równoważnie, jest to grupa generowana przez jeden z jej elementów (elementów które generują tę grupę może być wiele).

Generator

- Jeśli rząd elementu (*a*) jest równy rzędowi grupy, to mówimy, ze grupa jest cykliczna, a element *a* nazywamy **generatorem**.
- Rzędem grupy nazywamy liczbę elementów zbioru

Przykład I

$G = \mathbb{Z}_{10}$, działaniem jest dodawanie

$a = 3$

k	0	1	2	3	4	5	6	7	8	9	10	11
$k \cdot a$	0	3	6	9	2	5	8	1	4	7	0	3

$(3) = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} = \mathbb{Z}_{10}$

$\mathbb{Z}_{10}$ jest grupą cykliczną i element 3 jest jej **generatorem**.

$a = 2$

k	0	1	2	3	4	5	6	7
$k \cdot a$	0	2	4	6	8	0	2	4

$(2) = \{0, 2, 4, 6, 8\} \neq \mathbb{Z}_{10}$

$\text{Rząd}(2) = 5$

Przykład II

$G = \mathbb{Z}_{13}^{*} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$, Działaniem jest mnożenie

$a = 3$

k	0	1	2	3	4	5	6	...
a^k	1	3	9	1	3	9	1	...

$(3) = \{1, 3, 9\}$

$\text{Rząd}(3) = 3$

Algorytm uzgadniania klucza Diffiego-Hellmana

- Adam i Beata uzgadniają wspólny klucz nie korzystając z bezpiecznego kanału.

Adam:

1. Adam wybiera dużą liczbę pierwszą p, wybiera generator g grupy $\mathbb{Z}_p$ oraz pewną liczbę $k \in \{2, \ldots, p-2\}$
2. Adam oblicza: $K = g^k \bmod p$ i przesyła do Beaty trójkę liczb: (p, g, K)

Beata:

1. Beata odbiera p, g, K
2. Beata wybiera $l \in \{2, \ldots, p-2\}$, oblicza $L = g^l \bmod p$ i wysyła do Adama liczbę L

Adam:

3. Adam oblicza $L^k = W$

Beata:

3. Beata oblicza $K^l = V$

$W = V$ jest wspólnym kluczem Adama i Beaty.

Dowód

$W = L^k = \left(g^l\right)^k = g^{lk} = g^{kl} = K^l = V$

Bezpieczeństwo tego algorytmu opiera się na założeniu, że problem logarytmu dyskretnego w grupie $\mathbb{Z}_p^{*}$ jest (obliczeniowo) trudny.

Oskar - osoba próbująca złamać szyfr, haker.

Oskar widzi p, g, K, L. Gdy Oskar wyznaczy l lub k to potrafi wyznaczyć V (lub W). Wyznaczenie k lub l wymaga obliczenia logarytmu dyskretnego przy podstawie g z K $\left(\mathrm{D}\log_g K\right)$ lub logarytmu dyskretnego przy podstawie g z L $\left(\mathrm{D}\log_g L\right)$.

Ronald L. Rivest, Adi Shamir i Leonard Adleman

1977 trzej profesorowie z MIT w USA, algorytm RSA



Twórcy algorytmu RSA

Bezpieczeństwo oparte jest na trudności rozkładu dużych liczb na czynniki pierwsze

Ronald L. Rivest, Adi Shamir i Leonard Adleman

Algorytm do 20 września 2000 roku chroniony był patentem numer 4405892 ?
Obecnie może być dowolnie wykorzystywany.

Poważną wadą algorytmu RSA jest jego wolne działanie.
100 razy wolniejszy od DES.
Z tego powodu stosuje się go zazwyczaj w połączeniu z innymi algorytmami

Jest szeroko stosowany w komunikacji internetowej: poufnej poczcie elektronicznej i sygnowaniu dokumentów elektronicznymi podpisami, systemie PGP i protokołach SET, S/MIME, SSL oraz HTTPS.

Opis algorytmu:
1. wybierane są dwie duże liczby pierwsze p i q ?
np. o długości 512 bitów
2. obliczana jest liczba n będąca iloczynem p i q.
n=p*q
3. klucz publiczny e wybieramy ze zbioru
[max(p,q)+1, n-1] będący względnie pierwszą liczbą z funkcją Eulera dla n => f(n)=(p-1)*(q-1)
4. klucz prywatny d obliczamy z równania
d=inv(e,f(n)), czyli jako odwrotność e modulo f(n),
tj. (e*d) mod (p-1)*(q-1)=1
5. Szyfrowanie polega na C=M^e mod n
6. Deszyfrowanie M=C^d mod n

Wspomniany problem faktoryzacji odnosi się do liczby n. znając jedynie liczbę n (która jest publiczna) nie jest się w stanie określić liczb p i q ? będących podstawą do obliczeń kluczy szyfrowania (oczywiście zakładając, że pracujemy na dużych liczbach).

Przykład

Załóżmy, iż dysponujemy superszybkim komputerem, który jest w stanie sprawdzić podzielność miliarda dużych liczb w ciągu jednej sekundy.
Aby złamać szyfr RSA należy rozbić klucz publiczny na dwie liczby pierwsze będące jego dzielnikami.
Znajomość tych liczb pozwala rozszyfrować każdą informację zakodowaną kluczem prywatnym i publicznym.

Jednakże nie ma prostej metody rozbijania dużych liczb na czynniki pierwsze.
Nie istnieje żaden wzór, do którego podstawiamy daną liczbę i w wyniku otrzymujemy wartości jej czynników pierwszych.
Należy je znaleźć testując podzielność kolejnych liczb.

Przykład

Z rozważań o liczbach pierwszych wynika, iż w przypadku dwóch różnych dzielników pierwszych jeden musi leżeć poniżej wartości pierwiastka z danej liczby, a drugi powyżej. Zatem, aby go znaleźć musimy wyliczyć pierwiastek z rozkładanej liczby, a następnie testować podzielność przez liczby nieparzyste leżące poniżej tego pierwiastka.

Statystycznie poszukiwany czynnik pierwszy powinien znajdować się w górnej połówce zakresu od 2 do pierwiastka z *n*. Ile działań musimy wykonać? Policzmy.

Przykład

Klucz 128 bitowy. Pierwiastek jest liczbą 64 bitową. W zakresie od 2 do 2^64 co druga liczba jest nieparzysta, zatem jest ich około 2^64 / 2 = 2^63.
Ponieważ interesuje nas tylko górna połówka, to ilość liczb do sprawdzenia jest dwa razy mniejsza, czyli wynosi 2^63 / 2 = 2^62.
Ile czasu zajmie naszemu superkomputerowi sprawdzenie podzielności przez około 2^62 liczb, jeśli w ciągu 1 sekundy wykonuje on miliard sprawdzeń? Odpowiedź brzmi:
zajmie to około:
2^62 / 10^9 = 4611686018 sekund = 76861433 minut = 1281023 godzin = 53375 dni = 146 lat

Fazy algorytmu RSA

Algorytm RSA składa się z trzech podstawowych kroków:

Faza I

Generacja klucza publicznego i tajnego. Klucz publiczny jest przekazywany wszystkim zainteresowanym i umożliwia zaszyfrowanie danych. Klucz tajny umożliwia rozszyfrowanie danych zakodowanych kluczem publicznym. Jest trzymany w ścisłej tajemnicy

Faza II

Użytkownik po otrzymaniu klucza publicznego, np. poprzez sieć Internet, koduje za jego pomocą swoje dane i przesyła je w postaci szyfru RSA do adresata dysponującego kluczem tajnym, np. do banku, firmy komercyjnej, tajnych służb. Klucz publiczny nie musi być chroniony, ponieważ nie umożliwia on rozszyfrowania informacji - proces szyfrowania nie jest odwracalny przy pomocy tego klucza. Zatem nie ma potrzeby jego ochrony i może on być powierzany wszystkim zainteresowanym bez ryzyka złamania kodu.

Faza III

Adresat po otrzymaniu zaszyfrowanej wiadomości odczytuje ją za pomocą klucza tajnego.

Tworzenie kluczy RSA

Znajdź dwie duże liczby pierwsze (mające np. po 1024 bity). Oznacz je jako p i q. Istnieją specjalne algorytmy generujące duże liczby pierwsze.

Tworzenie kluczy RSA

Oblicz:
Ø = (p - 1) • (q - 1)
oraz
n = p • q
Wygenerowane liczby pierwsze usuń, aby nie wpadły w niepowołane ręce. Ø to tzw. funkcja Eulera, n jest modułem.

Tworzenie kluczy RSA

Wykorzystując odpowiednio algorytm Euklidesa znajdź liczbę e, która jest względnie pierwsza z wyliczoną wartością funkcji Eulera Ø (tzn. NWD(e, Ø) = 1) Liczba ta powinna również spełniać nierówność 1 < e < n . Nie musi ona być pierwsza lecz nieparzysta.

Najprostszą koncepcyjnie (choć kłopotliwą technicznie) metodą na znalezienie *NWD* jest rozkład obu liczb na czynniki pierwsze i znalezienie w rozkładach części wspólnej – jest to *NWD*.

Przykład:

Dla $a=234$ i $b=186$ mamy:

$a = 2 \cdot 3 \cdot 3 \cdot 13$
$b = 2 \cdot 3 \cdot 31$
$c = 2 \cdot 3$

NWD wynosi zatem 6.

Algorytm Euklidesa prześledzimy na wybranym wcześniej przykładzie:

$234 = 1 \cdot 186 + 48$
$186 = 3 \cdot 48 + 42$
$48 = 1 \cdot 42 + 6$
$42 = 7 \cdot 6 + 0$

Tworzenie kluczy RSA

Oblicz liczbę odwrotną modulo Ø do liczby e, czyli spełniającą równanie d • e **mod** Ø = 1. Można to zrobić przy pomocy rozszerzonego algorytmu Euklidesa,

Tworzenie kluczy RSA

Klucz publiczny jest parą liczb (e, n), gdzie e nazywa się publicznym wykładnikiem. Można go przekazać wszystkim zainteresowanym.

Tworzenie kluczy RSA

Klucz tajny to (d, n), gdzie d nazywa się prywatnym wykładnikiem. Klucz ten należy przechowywać pod ścisłym nadzorem.

Przykład

Wybieramy dwie dowolne liczby pierwsze. W rzeczywistości liczby te powinny być ogromne.

p = 13
q = 11

Przykład

Obliczamy Ø = (p - 1) • (q - 1) , czyli tzw. funkcję Eulera:
Ø = (13 - 1) • (11 - 1) = 12 • 10 = 120

Ø = 120

Przykład

Obliczamy moduł n:
n = p • q = 13 • 11 = 143

n = 143

Przykład

Wyznaczamy wykładnik publiczny e. Ma on być względnie pierwszy z Ø czyli z liczbą 120. Warunek ten spełnia, np. liczba 7.

e = 7

Przykład

Wyznaczamy następnie wykładnik prywatny, który ma być odwrotnością modulo Ø liczby e, czyli
d • 7 **mod** 120 = 1.
Liczbą spełniającą ten warunek jest 103

d = 103

Przykład

Klucz publiczny (e, n)

(7,143)

Przykład

Klucz tajny (d, n)

(103,143)

Szyfrowanie kluczem publicznym RSA Faza II

Otrzymujemy od adresata klucz publiczny w postaci pary liczb (e, n).

Szyfrowanie kluczem publicznym RSA Faza II

Wiadomość do zaszyfrowania zamieniamy na liczby naturalne t, które muszą spełniać nierówność

$0 < t < n$

Można tutaj skorzystać np. z łączenia kodów znaków. Oczywiście adresat musi znać używany przez nas sposób przekształcenia tekstu w liczbę, aby mógł on później odtworzyć otrzymaną wiadomość. Zwykle nie ma z tym problemu, ponieważ nadawca i odbiorca stosują wspólne oprogramowanie, które troszczy się o takie szczegóły techniczne.

Szyfrowanie kluczem publicznym RSA Faza II

Na tak otrzymanych liczbach wykonywana jest operacja szyfrowania, uzyskiwane są liczby

$c = t^e \bmod n$.

Szyfrowanie kluczem publicznym RSA Faza II

Liczby c są zaszyfrowaną postacią liczb t i przekazuje się je adresatowi wiadomości. Klucz (e, n) umożliwił ich zaszyfrowanie, lecz nie pozwala ich rozszyfrować.

Przykład

Otrzymaliśmy klucz publiczny (e, n). Przy jego pomocy możemy zakodować liczby od 0 do 142.

e = 7
n = 143

Przykład

Załóżmy, iż chcemy przesłać adresatowi zaszyfrowaną liczbę t = 123. W tym celu musimy obliczyć wartość wyrażenia:
c = 123^7 **mod** 143 = 425927596977747 **mod** 143 = 7
Wynik jest zaszyfrowaną liczbą 123. Przesyłamy go do adresata.

c = 7

c = t^e **mod** n

Rozszyfrowywanie kluczem prywatnym RSA

Jesteś adresatem zaszyfrowanych wiadomości. Wcześniej wszystkim korespondentom przesłałeś wygenerowany klucz publiczny (e,n), za pomocą którego mogą oni szyfrować i przesyłać ci swoje dane. Otrzymujesz więc zaszyfrowaną wiadomość w postaci liczb naturalnych c, które muszą spełniać warunek:
0 < c < n

Rozszyfrowywanie kluczem prywatnym RSA

Liczbę c przekształcasz na pierwotną wartość t stosując wzór:
t = c^d **mod** n

Rozszyfrowywanie kluczem prywatnym RSA

Z otrzymanej liczby t odtwarzasz wg ustalonego systemu znaki tekstu. Teraz możesz odczytać przesłaną wiadomość.

Przykład

Otrzymaliśmy zakodowaną wiadomość o wartości 7. Jesteśmy w posiadaniu klucza prywatnego, który służy do rozszyfrowywania wiadomości zakodowanych kluczem publicznym.

d = 103
n = 143
c = 7

Przykład

Wykonujemy następujące operacje:
t = 7^103 **mod** 143

d = 103
n = 143
c = 7

Przykład

Potęga jest zbyt duża, aby można ją było w normalny sposób obliczyć (języki programowania mają zwykle ograniczenia co do wielkości liczb całkowitych, np. w Pascalu liczby te nie mogą przekraczać wartości 4294967295). Jednakże nas nie interesuje wartość liczbowa potęgi, a jedynie reszta z dzielenia jej przez 143.

Przykład

Możemy więc rozłożyć potęgę na iloczyn składników o wykładnikach równych kolejnym potęgom liczby dwa:

7^103 **mod** 143 = 7^(64 + 32 + 4 + 2 + 1) **mod** 143 =

Przykład

(7^64 **mod** 143) • (7^32 **mod** 143) • (7^4 **mod** 143) • (7^2 **mod** 143) • 7^1 **mod** 143

t = 7^103 **mod** 143 = 113 • 16 • 113 • 49 • 7 **mod** 143 = 123

Wspomniany wcześniej problem faktoryzacji odnosi się do liczby n. Znając jedynie liczbę n (która jest publiczna) nie jest się w stanie określić liczb p i q ? będących podstawą do obliczeń kluczy szyfrowania (oczywiście zakładając, że pracujemy na dużych liczbach).

Wyzwanie faktoryzowania RSA

Jest konkurs finansowany przez Laboratoria RSA, w którym nagrody przyznawane są za sfaktoryzowanie bardzo dużych liczb. Dzięki temu stan bezpieczeństwa RSA jest stale monitorowany.

- W roku 1994 użyto 1600 szybkich stacji roboczych do faktoryzacjiliczby **RSA129**, tzn. znalezienia dwóch dużych czynników pierwszych liczby **129**-cyfrowej –obliczenia zajęły 8 miesięcy (5000 MIPS lat).

Wyzwanie faktoryzowania RSA

Na początku 2007 roku rekord faktoryzowania wynosił RSA-640. Jest to liczba o 193 cyfrach odkryta 2 listopada 2005 roku przez F. Bahra

Teraz nagroda wynosi 30.000$

Długość bezpiecznego klucza

1024 bity na potrzeby własne
2048
4096 bitów długość klucza
zwiększone bezpieczeństwo

Ataki rozproszone

Komputery kwantowe zagrożeniem dla RSA.

Opracowany np.. Algorytm Shora rozwiązuje problem faktoryzacji przy pomocy złożonych wielomianów. Jego implementacja na komputerach kwantowych może pozwolić na łamanie szyfru RSA w rozsądnym czasie.

- Np. do faktoryzacji liczby **RSA129**komputer kwantowy z zegarem 100 MHz potrzebowałby tylko **kilka sekund**!
  - **... a dla liczby** 400**-cyfrowej** niewiele ponad 1 minutę**!**
  - Ze względu na wielomianową zależność czasu faktoryzacji od rozmiaru (liczby bitów) liczby naturalnej kryptosystem RSA nie jest bezpieczny względem algorytmu Shora

- Roku 2001: grupa 6 eksperymentatorów z IBM i UniwersytetuStanford dokonała implementacji algorytmu Shorana komputerze kwantowym opartym o jądrowy rezonans magnetyczny. Dokonano rozkładu na czynniki pierwsze liczby $15 = 3 \cdot 5$.

algorytm Euklidesa

algorytm Euklidesa wyznaczania NWD dwóch liczb naturalnych można znacznie usprawnić zastępując odejmowanie operacją modulo (reszta z dzielenia). Faktycznie, od większej liczby możemy odjąć mniejszą tyle razy, ile się ona w niej mieści. To, co zostanie, jest resztą z dzielenia.

algorytm Euklidesa

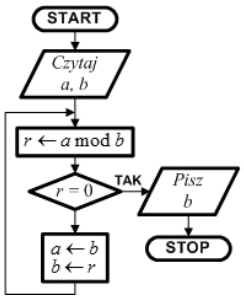
Jeśli reszta z dzielenia osiągnie wartość 0, to znaczy, iż dzielnik dzieli dzielną. W przypadku naszego algorytmu dzielnik jest NWD dwóch początkowych liczb. Algorytm kończy się zwrotem wartości dzielnika jako NWD. Jeśli nie otrzymaliśmy reszty 0, to dzielnik (jako większy) staje się w następnym kroku dzielną, a reszta dzielnikiem (jako mniejsza) i cykl powtarzamy.

algorytm Euklidesa

1 - Wyznaczenie reszty z dzielenia

2 - Sprawdzenie, czy otrzymana reszta jest równa 0

3 - Wymiana dzielnej z dzielnikiem i dzielnika z resztą



Kryptoanaliza RSA ze wstrzykiwaniem błędów

Kryptoanaliza algorytmu RSA wykorzystująca CRT bazuje na wprowadzeniu błędu do urządzenia w czasie wykonywania przez niego szyfrowania. Co więcej o błędzie tym zakładamy jedynie, że zmieni on tylko jeden z szyfrogramów *cp* albo *cq*, nie zakładając nic na temat typu wprowadzonego błędu.

Sposób ataku podany w 1997 roku

Kryptoanaliza RSA ze wstrzykiwaniem błędów

Wyliczamy różnicę między kryptogramem a kryptogramem uzyskanym po wprowadzeniu błędu.
Różnica ta jest wielokrotnością tajnej liczby q.

Stąd istnieje możliwość wyznaczenia q.

Algorytm ElGamala

- (1984, T. Elgamal)
- Opiera się na problemie algorytmów dyskretnych:
  - poszukiwanie liczby *i* dla dużego *p*

$$\alpha^i = \beta \bmod p$$

Algorytm ElGamala

- Szyfrowanie (liczby x)
  - Wybieramy losowo k
  - Obliczamy $y_1 = \alpha^k \bmod p,\ y_2 = x\beta^k \bmod p,$
  - Para liczb (y1, y2) to kryptogram
- Klucz publiczny: p, α, β
- Klucz prywatny: t, gdzie $t < p-1$
  $\beta = \alpha^t$

Algorytm ElGamala

- Elementy algorytmu:
  - duża liczba pierwsza *p*
  - liczba α
    - generator w pierścieniu Z_p
  - liczby β, t
    - gdzie $\beta = \alpha^t$

Algorytm ElGamala

- Deszyfrowanie
  - obliczamy wartość:
    $$D(y_1, y_2) = y_2 \cdot (y_1^t)^{-1}$$
  - dowód:

$$y_2\ (y_1^t)^{-1} = x\beta^k(\alpha^{kt})^{-1} = x\alpha^{tk}(\alpha^{tk})^{-1} = x \bmod p$$

Algorytm ElGamala

- Każdorazowe szyfrowanie jest inne, wykorzystuje losową liczbę *k*
- Asymetria kluczy: klucz publiczny jest do szyfrowania, klucz prywatny do deszyfrowania
- Asymetria algorytmu: inaczej szyfrujemy niż deszyfrujemy
- Kryptogram 2 razy dłuższy niż tekst

Wykład

Podpis elektroniczny

Podpis elektroniczny to:

Technika uwiarygodniania informacji utrwalonej w sposób elektroniczny, np. zapisanej lub przesyłanej w postaci danych komputerowych.

Pozwala ona na określenie źródła pochodzenia zapisu, oraz na wykrycie każdej zmiany (nawet pojedynczego bitu) w treści zapisu.

Definicja wg PN-I-02000

Podpis elektroniczny to przekształcenie kryptograficzne danych, umożliwiające odbiorcy danych sprawdzenie autentyczności i integralności danych, oraz zapewniające nadawcy ochronę przed sfałszowaniem.

Podpis cyfrowy, a tradycyjny

Jedną z istotnych różnic między dokumentem elektronicznym, a tradycyjnym jest to, że podpis na papierze składa własnoręcznie osoba fizyczna, a podpis cyfrowy w postaci dołączenia odpowiednio zakodowanego ciągu bitów – urządzenie techniczne, którego przypisanie do osoby fizycznej nie koniecznie jest jednoznaczne.

Cechy podpisu cyfrowego:

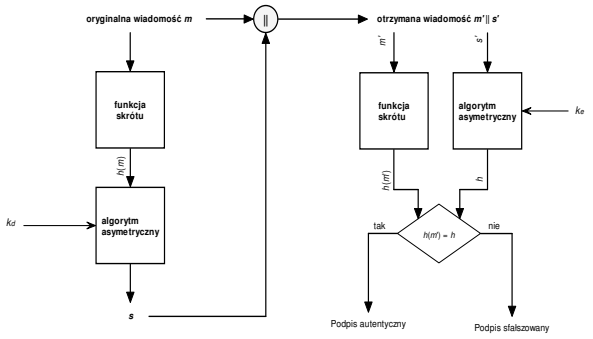
- Gwarantuje wykrycie każdej zmiany w już oznakowanym dokumencie
- Gwarantuje jednoznaczną identyfikację autora
- Wyklucza możliwość fałszowania
- Brak możliwości powtórnego użycia (przeniesienia na inny dokument)
- Pozwala określić czas znakowania

Zagadnienia związane z podpisem elektronicznym

- Kryptografia
- Szyfrowanie danych
- Algorytmy szyfrujące
- Funkcje skrótu (funkcje hashu)

Podział schematów podpisów cyfrowych

- z załącznikiem (z dodatkiem) – stosowana dla wiadomości o dowolnej długości; do weryfikacji podpisu niezbędna jest wiadomość oryginalna (schemat ElGamala),
- z odtwarzaniem – stosowana dla wiadomości o ustalonej długości, przy czym wykorzystuje się funkcje redundancji (w szeregu przypadkach może to być funkcja identycznościowa); przy weryfikacji podpisu część wiadomości można uzyskać z podpisu (schematy RSA, Rabina).



Ogólny schemat podpisywania i weryfikacji dla podpisu z załącznikiem

Funkcja hashująca to:

Pewna funkcja matematyczna, która pozwala na obliczenie skrótu (wyciągu) wiadomości o odpowiedniej długości. Funkcja skrótu jest wyliczana na podstawie sekwencji kodów znaków, składających się na zapis informacji. Każda zmiana w sekwencji kodów powoduje zmianę wartości wyciągu.

Algorytmy hashujące:

1. MD5 – algorytm wymyślonym w RSA Data Security Inc. Może być użyty do hashowania przypadkowej długości ciągu znaków w wartość 128 bitową. MD5 jest powszechnie używany.
2. SHA (Secure Hash Algorithm) – algorytm kryptograficzny opublikowany przez Rząd USA. Uważany jest za całkiem dobry, gdyż produkuje 160 – bitową hash-wartość z przypadkowej długości ciągu znaków.
3. RIPEMD-160 – jeden z najnowszych algorytmów, który produkuje 160 – bitowy wyciąg z wiadomości. Jego zaletą jest szybkość.

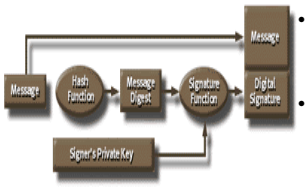
Działanie podpisu cyfrowego

Do podpisu cyfrowego potrzebna jest para kluczy: publiczny/prywatny.

Cały proces składa się z dwóch głównych etapów:

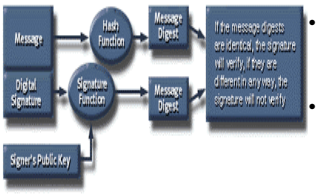
- Generowania podpisu cyfrowego
- Weryfikacji podpisu cyfrowego

Generowanie podpisu cyfrowego



- Na podstawie wiadomości tworzony jest jej skrót (hash)
- Hash jest szyfrowany kluczem prywatnym użytkownika
- Zaszyfrowana wartość hash to właśnie podpis elektroniczny

Weryfikacja podpisu cyfrowego



- Odbiorca wyznacza skrót z otrzymanej wiadomości
- Deszyfruje podpis używając klucza publicznego nadawcy
- Jeśli obie wartości hash są równe to podpis jest prawidłowy, a wiadomość wiarygodna

Podpisem cyfrowym nie jest:

- Kombinacja identyfikatora użytkownika i hasła. Jest to słabe zabezpieczenie przed podszywaniem. Nie ma zapewnionej integralności danych.
- Kombinacja identyfikatora użytkownika, hasła i liczby tokena (token zawiera algorytm symetryczny). Jest to co prawda wysokie zabezpieczenie przed podszywaniem, brak jest jednak integralności danych.

Bezpieczeństwo algorytmu RSA

Proces generowania kluczy w tej metodzie sprowadza się do rozkładania na czynniki pierwsze gigantycznych liczb (min 300 cyfr). Przy zastosowaniu klucza 1024 lub 2048 bitowego faktoryzacja zajęłaby miliony lat (przy obecnej mocy obliczeniowej). Takie klucze będą więc bezpieczne przez wiele lat.

Techniki kryptoanalityczne:

- Metoda na siłę
- Atak na szyfrogram
- Atak na szyfrogram częściowo znany
- Atak na szyfrogram o znanej treści
- Atak człowiek w środku (man-in-the-middle)
- Atak na czas wykonania

Metoda na siłę

- Metoda ta polega na sprawdzaniu wszystkich możliwych kombinacji kluczy.
- Potrzebna moc obliczeniowa wzrasta wykładniczo wraz z długością klucza (np. klucz 32 bitowy wymaga 2^{32} kroków.
- Uważa się, że klucze ze 128 bitami pozostaną prawdopodobnie nie do złamania w przeciągu przewidywanej przyszłości.

Atak na szyfrogram

Taka sytuacja ma miejsce, gdy atakujący nic nie wie o zawartości wiadomości i musi pracować tylko z szyfrogramem. W praktyce jest często możliwe zgadnięcie tekstu jawnego, bo wiele typów wiadomości ma z góry ustalone nagłówki. Nawet zwykłe listy i dokumenty zaczynają się w bardzo przewidywalny sposób.

Atak na szyfrogram częściowo znany

Atakujący zna lub może odgadnąć jawny tekst dla niektórych części szyfrogramu. Zadaniem jest odkodować resztę szyfrogramu używając tej informacji.

Atak na szyfrogram o znanej treści

Atakujący jest w stanie zdobyć jakikolwiek tekst wybrany przez siebie i zaszyfrowany z użyciem nieznanego klucza. Zadaniem jest ustalenie klucza użytego do kodowania. Algorytm RSA jest niezwykle wrażliwy na ten typ ataku. Podczas używania takich algorytmów, największy nacisk kładzie się na to, by osoba postronna nie miała możliwości zaszyfrowania wybranego przez siebie tekstu.

Atak człowiek w środku

- Ten atak jest odpowiedni dla protokołów wymiany klucza.
- Gdy dwie osoby wymieniają się kluczem atakujący ustawia się pomiędzy nimi na linii komunikacyjnej.
- Atakujący przeprowadza oddzielną wymianę kluczy z każdą ze stron.
- Strony kończą wymianę używając innych kluczy, z których każdy jest znany atakującemu.

Atak na czas wykonania

- Ten wymyślony bardzo niedawno atak bazuje na powtarzaniu mierzenia dokładnego czasu wykonania operacji potęgowania.
- Jest odpowiedni co najmniej dla algorytmów RSA, Diffie-Hellmana i krzywych eliptycznych.

Zastosowania

- Podpis cyfrowy z elektronicznie utrwalonej informacji czyni dokument, który ma wartość prawną i można się nim posługiwać w obrocie zarówno handlowym jak i między osobami fizycznymi.
- Może służyć do prowadzenia rachunkowości własnej przedsiębiorstwa.
- Jest niezbędny do komunikowania się z bankiem.
- Może znacząco obniżyć koszty odbierania i weryfikacji deklaracji ubezpieczeniowych i podatkowych.

Certyfikacja kluczy

- Certyfikacja daje gwarancję, że osoba, która użyła klucza, jest tą, za którą się podaje.
- Certyfikat składa się z grupy danych:
  - Dane o podmiocie dla którego został wystawiony,
  - Klucz publiczny posiadacza certyfikatu,
  - Dane urzędu certyfikującego.
- Certyfikat umożliwia sprawdzenie, czy podpis cyfrowy jest autentyczny.

- Certyfikat cyfrowy (x.509v3)

  Certyfikat cyfrowy – elektroniczne zaświadczenie, potwierdzające tożsamość użytkownika danej pary kluczy (osoby fizycznej, serwera, witryny WWW, koncentratora VPN, komputera)

- Zawiera klucz publiczny
- przechowuje informacje o podmiocie (np. imię nazwisko, e-mail, nazwa domenowa, adres IP)
- Może zawierać ograniczenia stosowania certyfikatu (np. podpis cyfrowy, szyfrowanie danych, uwierzytelnienie)
- Numer seryjny
- z góry określony jest termin ważności certyfikatu
- klucz publiczny oraz powyższe dane są podpisane przez Urząd ds. Certyfikatów, który jest gwarantem ich prawdziwości
- Możliwość unieważnienia (odwołania) certyfikatu

Certyfikacja kluczy

- **Zbiór standardów ustanawiających środki techniczne do wykorzystania kryptografii z kluczem publicznym (asymetrycznej) na potrzeby potwierdzania tożsamości, hierarchicznego składania podpisów cyfrowych i wymiany kluczy w protokołach kryptograficznych.**
- **Standardy PKI noszą różne nazwy, w zależności od autorów: np., dokumenty tworzone przez firmę RSA Data Security są publikowane pod nazwa PKCS, podczas gdy publikacje ITU-T używają oznaczenia X.509.**

Certyfikacja kluczy

- • Poufność przesyłanych danych,
- • Kontrolę dostępu,
- • Silną autentykację użytkowników,
- • Zapewnienie integralności danych,
- • Niezaprzeczalność dokonywanych transakcji,
- • Eliminację anonimowości.

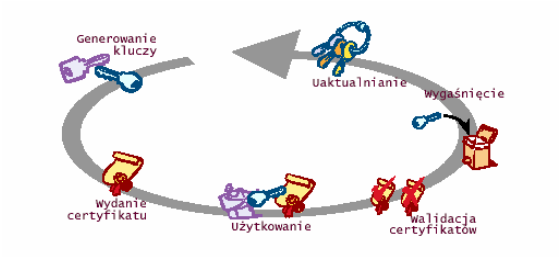
Certyfikacja kluczy

- **Certyfikat PKI** - pakiet danych, zawierający klucz publiczny certyfikowanego podmiotu (serwera, osoby), informacje na temat tego ostatniego (nazwa, adres).
- Certyfikat zawiera także podpis cyfrowy urzędu certyfikującego poświadczający prawdziwość danych oraz autentyczność klucza publicznego.

Czym jest certyfikat

- **Generowane są klucze użytkownika (szyfrujące, podpisujące), a dalszej obróbce poddawane są tylko ich części publiczne by ostatecznie móc certyfikat przekazywać osobom trzecim.**
- **Razem z wybranymi informacjami o użytkowniku/obiekcie (serwer, router, itp.) są one umieszczane np. w pliku, na karcie chipowej, e-tokenie.**
- **Z jego zawartości wyliczana jest wartość funkcji skrótu (np. MD5, SHA1).**
- **Uzyskana wartość jest następnie szyfrowana kluczem prywatnym Certificate Authority.**
- **Obie informacje (dane o użytkowniku/obiekcie i zaszyfrowana wartość skrótu) są scalane i tak uzyskany twór nosi miano certyfikatu.**

Cykl życia certyfikatu



Zastosowania

Wybór schematu podpisu powinien być zdeterminowany:

- okresem czasu zapewniania bezpieczeństwa,
- możliwościami pamięciowymi i obliczeniowymi,
- przeznaczeniem podpisu

Zastosowania podpisów cyfrowych:

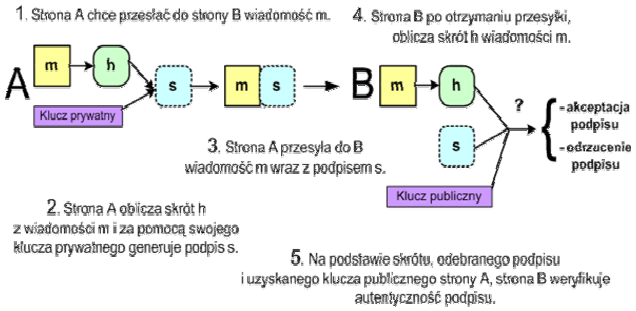
- Przy przesyłaniu informacji (korespondencja, przelewy elektroniczne itp. – w bankowości i handlu).
- W weryfikacji integralności baz danych.
- W głosowaniu komputerowym.
- Przy ustalaniu klucza w kryptosystemach symetrycznych.
- W ochronie programów przed nielegalnym użytkowaniem.

Typy ataków

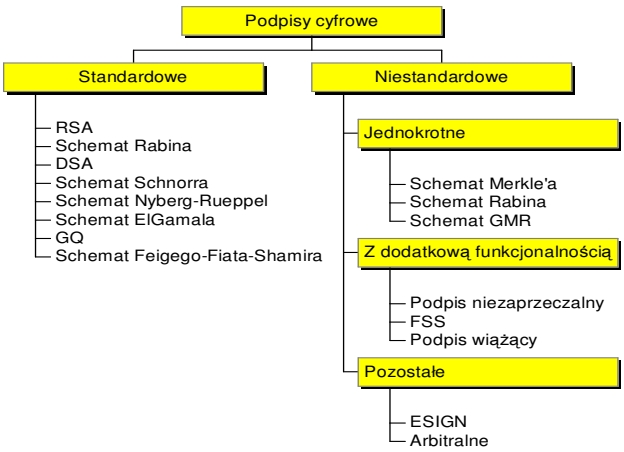
Podstawowe typy ataków na schematy podpisów cyfrowych:

- Atak bez wiadomości *(no-message attack).*
- Atak ze znaną wiadomością *(known-message attack).*
- Atak ogólny wybraną wiadomością *(generic chosen-message attack).*
- Atak zorientowany wybraną wiadomością *(oriented cosen-message attack).*
- Atak adaptacyjny wybraną wiadomością *(adaptive chosen-message attack).*

Schemat ogólny



Podpis z załącznikiem.



Cechy podpisów niestandardowych

- Podpisy z załącznikiem.
- Są odpowiedzią na szczególne wymagania (dodatkowa funkcjonalność) – specyficzne zastosowania.
- Bazują na nietypowych problemach (nie wymagają skomplikowanych obliczeń).
- Skrócony czas oczekiwania na podpis.
- Wzmacniają wymaganie niezaprzeczalności , niepodrabialności lub umożliwiają kontrolę weryfikacji.

ESIGN

- Nazwa: *Efficient digital SIGNature.*
- Powstanie: 1985, japońska firma NTT.
- Bezpieczeństwo: problem faktoryzacji dużych liczb całkowitych.
- Porównywalny z DSA i RSA pod względem szybkości i bezpieczeństwa.
- Przyspieszenie procesu podpisywania można uzyskać przez wykonanie obliczeń wstępnych.
- Podpisywanie wiadomości dowolnej długości.
- Funkcja skrótu.

$$h:\{0,1\}^* \to \mathrm{Z}_n$$

Generowanie kluczy

1. Wylosowanie liczb pierwszych p i q
2. Obliczenie $n = p^2q$
3. Wybranie liczby $k \in \{8, 16, 32, \dots\}$

Klucz prywatny: (p, q)
Klucz publiczny: (n, k)

1. $p = 5 \quad q = 7$
2. $n = 175$
3. $k = 4$

Klucz prywatny: $(5, 7)$
Klucz publiczny: $(175, 4)$

Generowanie podpisu

1. Obliczenie skrótu
2. Wylosowanie tajnej liczby $x \in \mathrm{Z} \quad 0 \geq x \geq pq$
3. Obliczenie
$$w = \left\lceil \frac{((h_0 - x^k) \bmod n)}{pq} \right\rceil$$
$$y = w \cdot \left(kx^{k-1}\right)^{-1} \bmod p$$
4. Obliczenie podpisu s:
$$s = x + ypq$$

Wiadomość: $m = 123$

1. $h_0 = h(123) = 123$
$h_0 = h(m)$
2. $x = 2$
3. $w = \left\lceil \frac{(123 - 2^4) \bmod 175}{35} \right\rceil = 4$
$y = 4 \cdot \left(4 \cdot 2^3\right)^{-1} \bmod 5 = 2$
4. Podpis:
$s = 2 + 2 \cdot 5 \cdot 7 = 72$

Weryfikacja podpisu

1. Uzyskanie autentycznego klucza publicznego autora podpisu (n, k)
2. Obliczenie
$u = s^k \bmod n$,
$h_1 = h(m)$
3. Zaakceptowanie podpisu, jeżeli prawdziwa jest nierówność:
$$h_1 \leq u \leq h_1 + 2^{\left\lceil \frac{2}{3} \log_2 n \right\rceil}$$

Podpis: $s = 72$
Wiadomość: $m = 123$
Klucz publiczny: $(175, 4)$

2. $u = 72^4 \bmod 175$
$h_1 = h(123) = 123$
3. Ponieważ
$123 \leq 156 \leq 123 + 2^6$
uznajemy podpis za aktualny

Jednokrotne schematy podpisu

- Para kluczy wykorzystywana jest do podpisania jednej wiadomości.
- Weryfikacja danej wiadomości może wystąpić tylko raz bez narażenia podpisu na podrobienie.
- Wielokrotne podpisywanie uzyskuje się w połączeniu z drzewami uwierzytelniania.
- Generowanie podpisu i weryfikacja są procesami szybkimi.
- Odporność na ataki z wybraną wiadomością.
- Schematy proste, szybsze od standardowych schematów, mogą być efektywnie implementowane.

Schemat Merkle'a jednokrotnego podpisu cyfrowego

- Nazwa pochodzi od autora – Ralpha Merkle.
- Brak konwersacji z podpisującym.
- Wymagany udział TTP.
- Schemat rozważa jednocześnie tylko jeden bit wiadomości.
- Możliwość zwiększenia efektywności.
- Możliwość wykorzystania drzew uwierzytelniania.
- Podpisywanie wiadomości dowolnej długości.
- Funkcja skrótu trudno-odwracalna. $h:\{0,1\}^* \to \{0,1\}^l$

Generowanie klucza

1. Wybranie *l*-bitowych ciągów $k_1, k_2, ..., k_t$
   $t = n + \lfloor \log_2 n \rfloor + 1$
2. Obliczenie
   $y_i = h(k_i), 1 \le i \le t$

Wiadomość: $m = 101001$

$t = 6 + \lfloor \log_2 6 \rfloor + 1 = 9$

Klucz prywatny: $k_1, k_2, ..., k_t$
Klucz publiczny: $y_1, y_2, ..., y_t$

Klucz prywatny: $k_1, k_2, ..., k_9$
Klucz publiczny: $y_1, y_2, ..., y_9$

Generowanie podpisu

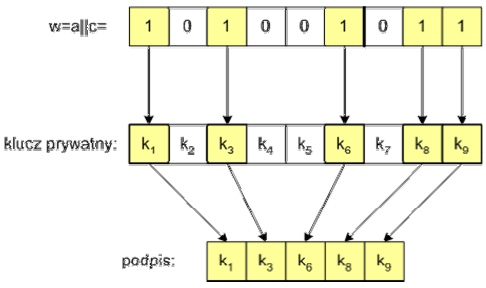
1. Obliczenie *c* – ilości zer w wiadomości *m*
2. Dołączenie do *m* liczby *c*
   $w = m \| c = (a_1 a_2 ... a_t)_2$
3. Znalezienie takich
   $i_1 \le i_2 \le ... \le i_u$, że
   $a_{i_j} = 1$, $1 \le j \le u$
4. Pobranie z klucza prywatnego elementów
   $k_{i_j}, 1 \le j \le u$

Podpis: $s_j = k_{i_j}, 1 \le j \le u$

Klucz prywatny: $k_1, k_2, ..., k_9$
Wiadomość: $m = 101001$

1. $c = 3 = (11)_2$
2. $w = 101001 \| 011$
3. $i_1 = 1, i_2 = 3, i_3 = 6$
   $i_4 = 8, i_5 = 9$
4. Podpis:
   k_1, k_3, k_6, k_8, k_9

$m = 101001$ $c = 11$



Weryfikacja podpisu

1. Uzyskanie autentycznego klucza publicznego autora podpisu – $y_1, y_2, ..., y_t$
2. Obliczenie *c*, utworzenie *w* oraz znalezienie indeksów analogicznie jak podczas generowania podpisu $i_j, 1 \le j \le u$
3. Zaakceptowanie podpisu, jeśli
   $y_{i_j} = h(s_j)$
   dla wszystkich
   $1 \le j \le u$

Klucz publiczny: $y_1, y_2, ..., y_9$
Wiadomość: $m = 101001$

2. $c = 3 = (11)_2$
   $w = 101001 \| 011$
   $i_1 = 1, i_2 = 3,$
   $i_3 = 6, i_4 = 8, i_5 = 9$
3. Jeżeli
   $y_1 = k_1$, $y_3 = k_3$, $y_6 = k_6$,
   $y_8 = k_8$, $y_9 = k_9$
   podpis uznajemy za autentyczny

Pośrednio-bezpośredni podpis cyfrowy

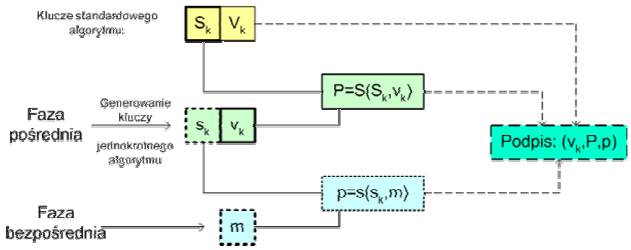
- Nazwa: On-line/Off-line Digital Signature.
- Łączy efektywność jednokrotnych schematów oraz funkcjonalność schematów standardowych.
- Wydłużenie kluczy w schemacie standardowym powoduje wydłużenie fazy pośredniej – schemat zwiększa bezpieczeństwo, zachowuje użyteczność.
- Odporny na ataki z wybraną wiadomością.
- Funkcja skrótu: szybka i odporna na kolizje.

Oznaczenia:

G – standardowy algorytm generujący klucze S_k, V_k
S – standardowy algorytm podpisujący
V – standardowy algorytm weryfikujący
$S(S_k,m)$ – podpis standardowy pod wiadomością *m*

g – jednokrotny algorytm generujący klucze s_k, v_k
s – jednokrotny algorytm podpisujący
v – jednokrotny algorytm weryfikujący
$s(s_k,m)$ – podpis jednokrotny pod wiadomością *m*

Faza pośrednia i bezpośrednia



Schematy podpisu z dodatkową funkcjonalnością

- Weryfikacja może przebiegać tylko za zgodą sygnatariusza (schemat podpisu niezaprzeczalnego).
- Fałszerstwo można udowodnić.
- Można wykazać faktyczne autorstwo podpisu, tj. sygnatariusz nie może zaprzeczyć podpisania.
- Znalezienie klucza prywatnego nie zależy od możliwości obliczeniowych przeciwnika (schemat FSS).

FSS – niepodrabialny podpis cyfrowy

- Nazwa: *Fail-Stop Signature.*
- Autorzy: Birgit Pfitzmann, Michael Waidner
- Pozwala udowodnić fałszerstwo – wyprowadzenie tzw. „dowodu fałszerstwa".
- Generowanie kluczy odbywa się przy współudziale TTP.
- Podpisywane są wiadomości z przedziału $[0,q]$, Z_q^* takiego, że problem logarytmowania dyskretnego w jest trudny obliczeniowo.

Generowanie kluczy

(TTP)

1. Wybranie liczb pierwszych *p* i *q*, $q \mid (p-1)$
2. Wybranie α - generatora podgrupy cyklicznej w Z_p^*
3. Wylosowanie liczby całkowitej $d \in [1, q-1]$
4. Obliczenie $\beta = \alpha^d \bmod p$

1. *p=7*, *q=3*
2. $\alpha = 2$
3. *d=2*
4. $\beta = 2^2 \bmod 7 = 4$

TTP wysyła do sygnatariusza (p,q,α,β) $(7,3,2,4)$

TTP trzyma *d* w tajemnicy.

(sygnatariusz)

- Wylosowanie z przedziału $[0,q-1]$ liczb całkowitych x_1, x_2, y_1, y_2.
- Obliczenie
  $\beta_1 = \alpha^{x_1}\beta^{x_2} \bmod p$
  $\beta_2 = \alpha^{y_1}\beta^{y_2} \bmod p$

5. $x_1 = 0,\ x_2 = 1,$
   $y_1 = 2,\ y_2 = 2$

6. $\beta_1 = 2^0 4^1 \bmod 7 = 4$
   $\beta_2 = 2^2 4^2 \bmod 7 = 1$

Klucz prywatny: (x_1, x_2, y_1, y_2)
Klucz publiczny: $(\beta_1, \beta_2, p, q, \alpha, \beta)$

Klucz prywatny: $(0,1,2,2)$
Klucz publiczny: $(4,1,7,3,2,4)$

Generowanie podpisu

1. Obliczenie

$s_1 = x_1 + m y_1 \bmod q$

$s_2 = x_2 + m y_2 \bmod q$

Podpis: (s_1, s_2)

Wiadomość: $m = 1$

1.

$s_1 = 0 + 1 \cdot 2 \bmod 3 = 2$

$s_2 = 1 + 1 \cdot 2 \bmod 3 = 0$

Podpis: $(2,0)$

Weryfikacja podpisu

1. Uzyskanie autentycznego klucza publicznego autora podpisu $(\beta_1, \beta_2, p, q, \alpha, \beta)$
2. Obliczenie

$v_1 = \beta_1 \beta_2^m \bmod p$

$v_2 = \alpha^{s_1} \beta^{s_2} \bmod p$

3. Zaakceptowanie podpisu, jeżeli zachodzi równość

$v_1 = v_2$

Podpis: $(2,0)$

Wiadomość: $m = 1$

Klucz publiczny: $(4,1,7,3,2,4)$

2. $v_1 = 4 \cdot 1 \bmod 7 = 4$

$v_2 = 2^2 \cdot 4^0 \bmod 7 = 4$

3. Podpis autentyczny:

$v_1 = 4 = v_2$

Dowodzenie fałszerstwa

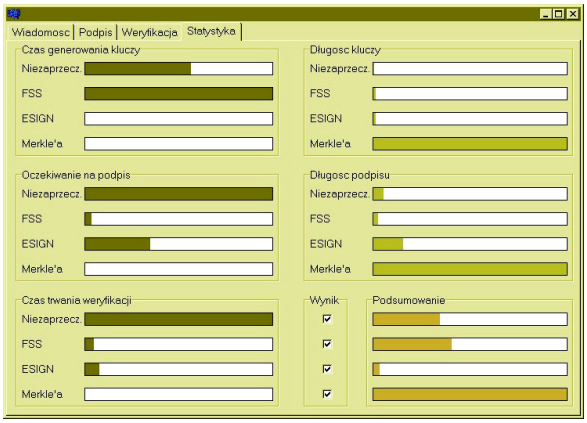
Autentyczny podpis: (s_1, s_2)

Podrobiony podpis: (s_1', s_2')

Jeżeli $(s_1, s_2) \neq (s_1', s_2')$, sygnatariusz wyprowadza dowód fałszerstwa:

$d = (s_1 - s_1')(s'_2 - s_2)^{-1} \bmod q$

(Liczbę $d = \log_\alpha \beta$ TTP przechowuje w tajemnicy.)



Zestawienie

	Gen. Kluczy (ms)	Gen. Podpisu (ms)	Weryfikacja (ms)	Długość kluczy (b)	Długość podpisu (b)
Niezaprz.	1 431	240	520	74	18
FSS	2 553	9	29	92	10
ESIGN	0	70	40	91	50
Merkle	9	0	0	4 137	292

Literatura

- I.B. Damgård, T.P. Pedersen, B. Pfitzmann, “On the Existence of Statistically Hiding Bit Commitment Schemes and Fail-Stop Signatures”, *Journal of Cryptology*.
- S. Even, O. Goldreic, S. Micali, “On-Line/Off-Line Digital Signatures”, *Journal of Cryptology*.
- Alfred J. Menezes, Paul C. van Oorshot, Scott A. Vanstone: „Handbook of Applied Cryptography”. Dostępna na stronie: http://www.cacr.math.uwaterloo.ca/hac/
- B. Schneier, “Kryptografia dla praktyków. Protokoły, algorytmy i programy źródłowe w języku C.”, *WNT*, 1995.

FUNKCJE JEDNOKIERUNKOWE

właściwości

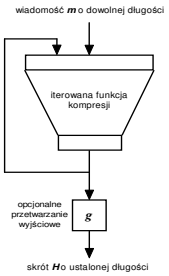
- Dla każdego X łatwo jest obliczyć H(X)
- H(X) ma taka samą długość dla wszystkich tekstów X
- Dla zadanego Y znalezienie takiego X, że H(X) = Y jest praktycznie niemożliwe; **funkcja jednokierunkowa**
- Dla danego X trudno znaleźć X' takie, że H(X) = H(X'); **funkcja słabo bezkonfliktowa**
- Nie jest praktycznie możliwe znalezienie X i X' takich, że X != X' oraz H(X) = H(X'); **funkcja silnie bezkonfliktowa**

Własności funkcji skrótu

- własność kompresji
- własność łatwości obliczeń, mając dane M łatwo jest obliczyć h
- własność jednokierunkowości, mając dane h, trudno jest obliczyć M
- własność słabej odporności na kolizje, mając dane M, trudno jest znaleźć inną wiadomość M' taką, że H(M)=H(M')
- własność odporności na kolizje jest obliczeniowo trudne znalezienie dwóch dowolnych argumentów $M \neq M'$, dla których $H(M) = H(M')$.

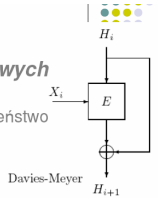
zastosowania

- Zabezpieczanie haseł w komputerach
- Ochrona integralności danych
- Ochrona przed wirusami
- Podpis cyfrowy
- Generatory ciągów pseudolosowych (kluczy), tworzonej na podstawie tajnej danej.
- MIC(mesage integrity checks)
- MAC (message authentication code) – funkcje odwracalne ale tylko w przypadku posiadania klucza



Konstruowanie funkcji skrótu

- ***Funkcje skrótu oparte na szyfrach blokowych***
  - wykorzystują strukturę szyfrów blokowych
  - gotowe rozwiązania, redukcja kosztów, bezpieczeństwo
  - popularne w latach 80-tych
  - zbyt wolne



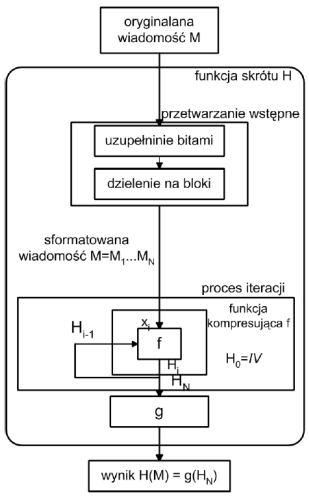
- ***Funkcje oparte na arytmetyce modularnej***
  - najczęściej wolne
  - łatwo skalowalne

$$H_{i+1} = ((((H_i \oplus \tilde{X}_i) \vee A)^2 \bmod M) \dashv c) \oplus H_i$$

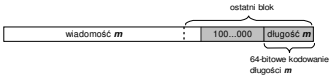
- ***Dedykowane funkcje skrótu***
  - wykorzystują własne funkcje kompresujące
  - obecnie najczęściej stosowane
  - znacznie szybsze

Dwie klasy funkcji skrótu

- funkcje skrótu bez klucza kryptograficznego
- funkcje skrótu z kluczem (MAC – ang. *message authentication code*)



- W większości zastosowań funkcjom mieszającym stawia się dodatkowe wymaganie, nazywane odpornością na kolizje. Kolizję definiujemy jako znalezienie dwóch wiadomości losowych M i M', takich, że H(M) = H(M'). Jest to zadanie bardzo trudne, można tu jednak zastosować tzw. paradoks urodzin.



różnice w budowie i działaniu funkcji kompresji *f*
- długość bitowa wyznaczanego skrótu,
- liczba iteracji,
- liczba wykorzystywanych elementarnych funkcji logicznych,
- liczba wykorzystywanych predefiniowanych stałych,
- liczba wykorzystywanych operacji logicznych.

Atak na funkcje skrótu

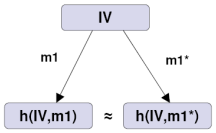
Brutalny, słownikowy
Paradoks dnia urodzin
Atak pseudokolizyjny
Atak strukturalny
Atak różnicowy

- Paradoks urodzin jest typowym problemem statystycznym. Ile osób musi znaleźć się w jednym pokoju, aby były znaczące szanse, że znajdzie się tam osoba urodzona konkretnego dnia (np. 1 stycznia)? **Odpowiedź wynosi: 253**.
- Zadajemy więc kolejne pytanie: ile osób musi być w tym pokoju, aby uzyskać znaczącą szansę, że są tam co najmniej dwie osoby urodzone tego samego dnia? **Odpowiedź jest zaskakująca: wystarczą 23 osoby** (mając 23 osoby mamy nadal 253 rożne pary ludzi).
- Poszukiwanie kogoś z określoną datą urodzin jest analogią do ataku, polegającego na tym, że atakujący posiada wiadomość M i szuka takiej wiadomości M', że H(M) = H(M').

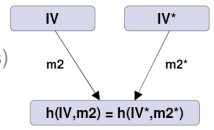
- Wartości funkcji skrótu, będące liczbami 64-bitowymi, są zbyt krótkie, aby wytrzymać atak metodą dnia urodzin. Większość praktycznie stosowanych funkcji skrótu daje wynik co najmniej 128-bitowy (zaleca się używanie funkcji dających znacznie dłuższe wartości), co oznacza, że atakujący musi obliczyć wartości funkcji skrótu dla 264 wiadomości.

Ataki strukturalne

- prawie-kolizje (near-collisions)



- pseudo-kolizje (pseudo-collisions)



- Załóżmy, że *H* jest iteracyjną funkcją skrótu i jej funkcja kompresująca *f* działa na *k* bitowych blokach. Niech *M*1 i *M*2 będą *k* bitowymi wiadomościami. Wtedy

$H(IV,M1M2) = f(f(IV,M1),M2)$

gdzie *M*1*M*2 oznacza konkatenację wiadomości *M*1 i *M*2. Dla

$IV' = f(IV,M1)$

mamy

$H(IV',M2) = H(IV,M1M2)$.

- Zatem można uzyskać dwie różne wiadomości, dające ten sam skrót ale przy dwóch różnych wartościach inicjujących.

Ataki różnicowe

- współczesne ataki Bihama i Chena, Wanga są różnicowe
- poprzez zmianę kilku bitów w wiadomości prawdopodobne jest zniwelowanie „różnicy” po kilku rundach
- różnica jest zazwyczaj definiowana jako funkcja logiczna *xor*

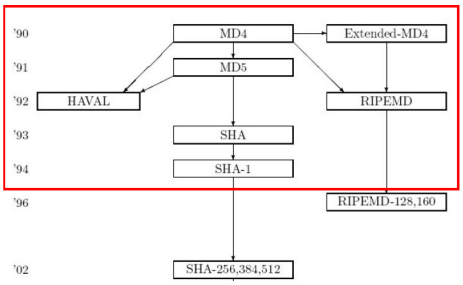
Funkcje skrótu

- SNERFU – 1990 rok, wolna i mało bezpieczna
- N-Hash – potomek alg. FEAL, równie niebezpieczna jak on
- GOST – standard rosyjski
- MD2 – Ron Rivest, RFC1319, podatna na wymuszenie konfliktu
- MD4 – Ron Rivest, 1990 rok, 128-bitowa wartość
- MD5 – Ron Rivest, 1991, „poprawiona” MD4

Funkcje skrótu

- SHA – opracowana przez NIST&NSA, opublikowana w 1994 roku, wynik ma 160 bitów, zawiera kanały podprogowe, element składowy DSA
- RIPE-MD160 – efekt europejskiego projektu RIPE, bazuje na MD4, zwraca wartości 160-bitowe

Rodzina funkcji MD/SHA



Rodzina funkcji MD/SHA

Wady

- Powolna dyfuzja
  - większość rejestrów jest tylko kopiowana w kolejnych rundach
  - wolny efekt lawinowy z powodu używanych funkcji (xor, add)
- Obecność neutralnych bitów
- Podatność na ataki różnicowe

Algorytm MD4 (Message Digest)

- MD4 jest jednokierunkową funkcją skrótu zaprojektowaną przez Rona Rivesta. Skrót MD pochodzi od angielskiego określenia M*essage Digest* oznaczającego skrót wiadomości. Algorytm ten, dla danej wiadomości, wytwarza skrót wiadomości o długości 128 bitów.

Algorytm MD4 (Message Digest)

- Rivest podał następujące cele przy projektowaniu tego algorytmu.
- Bezpieczeństwo. Powinno być obliczeniowo niewykonalne tzn. znalezienie dwóch wiadomości, które po skróceniu dają tę samą wartość. Żaden atak nie powinien być bardziej efektywny niż, atak brutalny.
- Szybkość. Algorytm MD4 powinien być przystosowany do bardzo szybkich implementacji programowych. Powinien bazować na zbiorze prostych operacji bitowych na 32-bitowych argumentach.

Algorytm MD4 (Message Digest)

- Prostota i zwartość. Algorytm MD4 powinien być tak prosty, jak tylko jest to możliwe, bez dużych struktur danych lub skomplikowanego programu.
- Algorytm MD4 powinien być zoptymalizowany pod kątem architektur mikroprocesorów (w szczególności mikroprocesorów firmy Intel); większe i szybsze komputery dokonają niezbędnych translacji.

Ataki różnicowe na funkcje MD/SHA

- Dwie rundy MD4 – kilka publikacji w latach 1990-1994
- MD4 – prawiekolizja, para wiadomości różniących się 3 bitami (Dobbertin,1995)
- MD5 – kolizja ze zmienioną wartością wektora inicjującego IV
- (Dobbertin, 1996)
- SHA-0 – atak o złożoności 2^{61} (1998)

- SHA-0 - reduced/extended versions, near-collisions (Biham, Chen, 2004)
- Multi-block, reduced versions of SHA-0 and SHA-1 (Biham, Chen, 2004)
- SHA-0 - a multi-block collision (Joux et al., 2004)
- SHA-1 - reduced versions (Rijmen, Oswald, 2004)
- MD5, MD4, and others — differential attacks (Wang et al., 2004)
- SHA-0 - a fast collision (Wang et al., 2005)
- SHA-1 – atak o złożoności 2^{69} (Wang et al., 2005)

Algorytm MD4 (Message Digest)

- Bert den Boer i Antoon Bosselaers przeprowadzili skuteczną kryptoanalizę dwóch z trzech cykli tego algorytmu. Ralph Merkle skutecznie zaatakował pierwsze dwa cykle .
- Eli Biham przeprowadził dyskusję możliwych ataków, za pomocą kryptoanalizy różnicowej, przeciwko dwóm z trzech cykli MD4.
- Pomimo że ataki te nie mogły być rozszerzone na cały algorytm, Rivest wzmocnił swój algorytm. Wynikiem tego był algorytm MD5.

Algorytm MD5 (Message Digest)

- Algorytm, zaprojektowany przez Rivesta, jest modyfikacją wcześniejszego algorytmu MD4.

Wiadomość dowolnej długości jest przekształcona w jej 128 bitowy „odcisk palca” (sumę kontrolną, skrót wiadomości). Zasadnicza procedura algorytmu działa na blokach 512 bitowych podzielonych na 16 podbloków o długości 32 bity każdy.

Na wyjściu algorytmu otrzymujemy zbiór czterech bloków 32-bitowych, które po konkatenacji tworzą skrót **128-bitowy**.

Różnice MD5 względem MD4

- 1. Dodany został czwarty cykl.
- 2. W każdym kroku algorytmu występuje obecnie inna wartość stałej addytywnej.
- 3. Funkcja g występująca w drugim cyklu została zmieniona *z (XY* OR *XZ.* OR *YZ)* na *(XZ* OR *Y* NOT(Z)), aby uczynić funkcję *g* mniej symetryczną.
- 4. Każdy krok wykorzystuje teraz wynik kroku poprzedniego. Sprzyja to szybszemu występowaniu „efektu lawinowego”.
- 5. Zmieniona została kolejność, w jakiej bloki wejściowe są włączane w cyklu 2 i 3, aby uczynić otrzymywane ciągi mniej podobnymi do siebie.
- 6. Wielkości przesunięć w każdym cyklu zostały w przybliżeniu zoptymalizowane tak, aby uzyskać szybszy „efekt lawinowy". Wielkości przesunięć w różnych cyklach są różne.

Różnice MD5 względem MD4

- Tom Berson próbował zastosować kryptoanalizę różnicową przeciw pojedynczemu cyklowi algorytmu MD5, lecz jego atak nie okazał się skuteczny.
- Dużo bardziej skuteczny atak Berta den Boera i Antoona Bosselaersa może wyznaczać kolizje, wykorzystując funkcję kompresującą w MD5. Nie jest to zbyt użyteczne do ataków przeciw MD5 w praktycznych zastosowaniach, lecz oznacza to, że jeden z podstawowych wymogów projektowych algorytmu MD5 - zaprojektowanie funkcji kompresującej wolnej od kolizji - został naruszony.

Etapy MD5

- Krok 1

Wiadomość dowolnej długości jest uzupełniana w taki sposób, że na końcu dodawany jest bit „1” i odpowiednia ilość zer, tak aby ostatni blok miał długość 448 bitów.

Etapy MD5

- Krok 2

Do ostatniego bloku dodawana jest 64 bitowa liczba reprezentująca długość wiadomości (w bitach) i w ten sposób przygotowana wiadomość ma długość będąca całkowitą wielokrotnością 512 bitów. Tym samym wiadomość jest wielokrotnością 16 słów 32-bitowych. Niech M(0),M(1), . . .M(N–1) oznaczają kolejne słowa wiadomości, gdzie N jest wielokrotnoscią 16.

Etapy MD5

- Krok 3

Algorytm operuje na 32-bitowych zmiennych a, b, c, d, których wartości początkowe A,B,C,D w zapisie szesnastkowym są następujące:

- A = 0x67452301
- B = 0xefcdab89
- C = 0x98badcfe
- D = 0x10325476

Etapy MD5

- Krok 4

Główna pętla algorytmu składa się z 4 rund, w każdej rundzie 16 razy wykonywane są operacje na 32 bitowych słowach. Operacje te zdefiniowane są przez 4 funkcje

- F(X, Y,Z) = (X ^ Y) v (¬X) ^ Z
- G(X, Y,Z) = (X ^ Z) v Y ^ (¬X)
- H(X, Y,Z) = X xor Y xor Z
- I(X, Y,Z) = Y xor (X v (¬Z))

gdzie xor oznacza operację mod2, ^ operację and, v operację or, zaś ¬ operację not.

Runda 1

Niech $[abcd\ j\ s\ k]$ oznacza operację $a = b + ((a + F(b,c,d) + X_j + T_k) \hookleftarrow s)$, wtedy 16 operacji tej rundy to:

$[abcd\ \ 0\ 7\ \ 1]$ $[dabc\ \ 1\ 12\ \ 2]$ $[cdab\ \ 2\ 17\ \ 3]$ $[bcda\ \ 3\ 22\ \ 4]$
$[abcd\ \ 4\ 7\ \ 5]$ $[dabc\ \ 5\ 12\ \ 6]$ $[cdab\ \ 6\ 17\ \ 7]$ $[bcda\ \ 7\ 22\ \ 8]$
$[abcd\ \ 8\ 7\ \ 9]$ $[dabc\ \ 9\ 12\ 10]$ $[cdab\ 10\ 17\ 11]$ $[bcda\ 11\ 22\ 12]$
$[abcd\ 12\ 7\ 13]$ $[dabc\ 13\ 12\ 14]$ $[cdab\ 14\ 17\ 15]$ $[bcda\ 15\ 22\ 16]$

Niech $X_j = M_{i*16+j}$ oznacza j-te słowo w i-tym bloku wiadomości M $(j = 0, \ldots, 15, i = 0, \ldots, N/16 - 1)$, $\hookleftarrow s$ niech oznacza cykliczne przesunięcie w lewo o s bitów oraz zdefiniujmy liczby T_k $(k = 1, \ldots, 64)$ tak, że $T_k = \lfloor 2^{32} \cdot |\sin k| \rfloor$, gdzie k jest w radianach. Mamy wtedy:

Runda 4

Niech $[abcd\ j\ s\ k]$ oznacza operację $a = b + ((a + I(b,c,d) + X_j + T_k) \hookleftarrow s)$, wtedy 16 operacji tej rundy to:

$[abcd\ \ 0\ 6\ 49]$ $[dabc\ \ 7\ 10\ 50]$ $[cdab\ 14\ 15\ 51]$ $[bcda\ \ 5\ 21\ 52]$
$[abcd\ 12\ 6\ 53]$ $[dabc\ \ 3\ 10\ 54]$ $[cdab\ 10\ 15\ 55]$ $[bcda\ \ 1\ 21\ 56]$
$[abcd\ \ 8\ 6\ 57]$ $[dabc\ 15\ 10\ 58]$ $[cdab\ \ 6\ 15\ 59]$ $[bcda\ 13\ 21\ 60]$
$[abcd\ \ 4\ 6\ 61]$ $[dabc\ 11\ 10\ 62]$ $[cdab\ \ 2\ 15\ 63]$ $[bcda\ \ 9\ 21\ 64]$

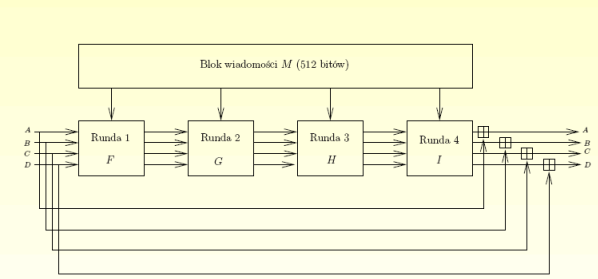
Etapy MD5

- Krok 5

Po przejściu wszystkich 512-bitowych bloków wiadomości M algorytm łączy rejestry a, b, c, d dając 128 bitową liczbę będącą wartością funkcji hashującej.

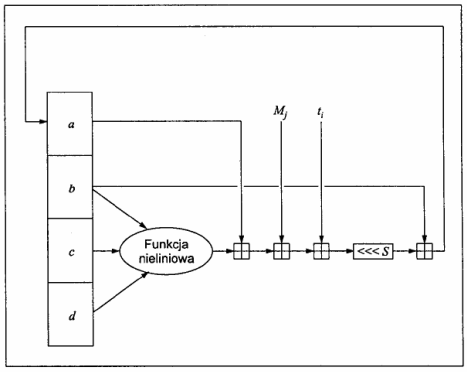
Etapy MD5

- Główna pętla algorytmu realizowana jest dla tylu 512- bitowych bloków, ile zawiera ich wiadomość. Cztery zmienne są kopiowane na inne cztery zmienne: A na a, B na b, C na c i D na d. Pętla główna składa się z czterech cykli, które są bardzo podobne.
- Każdy cykl składa się z 16 operacji, a w każdej operacji jest obliczana funkcja nieliniowa trzech z czterech zmiennych a, b, c i d. Następnie do wyniku dodawana jest wartość pozostałej, czwartej zmiennej, pewien podblok wiadomości i pewna stała. Wynik jest przesuwany cyklicznie w prawo o zmienną liczbę bitów, a potem sumowany z jedną ze zmiennych a, b, c lub d. Ostatecznie wyniki jest przypisywany jednej ze zmiennych a, b, c lub d.



Główna pętla algorytmu MD5

Jedna operacja MD5



SHA (Secure Hash Algorithm)

- Algorytm opracowany przez NIST przy udziale NSA opublikowany w 1993. Nowsza wersja SHA-1 opublikowana w 1995 r.
- Idea tego algorytmu oparta jest na MD4 i MD5.
- Wartość funkcji hashującej to liczba 160 bitowa i w związku z tym algorytm wymaga 5 rejestrów zamiast 4.
- Używa też w nieco inny sposób nieliniowych funkcji transformujących, dokonuje dodatkowych operacji na poszczególnych słowach wiadomości, w każdej rundzie wykonuje 20 operacji zamiast 16.

SHA (Secure Hash Algorithm)

- Zasada działania jest jednak bardzo podobna do MD5.
- Ogólnie uważa się, że jest bezpieczniejszy niż MD5 ze względu na „dłuższą” wartość funkcji hashującej i pewne ulepszenia samego algorytmu.

SHA (Secure Hash Algorithm)

- Długość wiadomości musi być wielokrotnością 512 – uzupełnienie jest identyczne, jak w przypadku MD5
- Pięć 32-bitowych zmiennych jest inicjowanych następująco:
- A = 0x67452301
- B = 0xefcdab89
- C = 0x98badcfe
- D = 0x10325476
- E = 0xc3d2e1f0

SHA (Secure Hash Algorithm)

- Następnie rozpoczyna się główna pętla algorytmu. W pętli tej przetwarzany jest każdorazowo 512-bitowy blok wiadomości i proces ten jest powtarzany tyle razy, ile bloków ma wiadomość.
- Najpierw wartości zmiennych A, B, C, D i E są kopiowane na wartości innych zmiennych, oznaczonych odpowiednio a, b, c, d i e.

SHA (Secure Hash Algorithm)

- Główna pętla składa się z czterech cykli, z których każdy zawiera 20 operacji (MD5 miał cztery cykle po 16 operacji). Każda operacja składa się z nieliniowej operacji na trzech spośród czterech zmiennych a, b, c i d oraz przesunięcia i sumowania, podobnie jak w przypadku MD5.

SHA (Secure Hash Algorithm)

Zbiór nieliniowych funkcji SHA jest następujący:

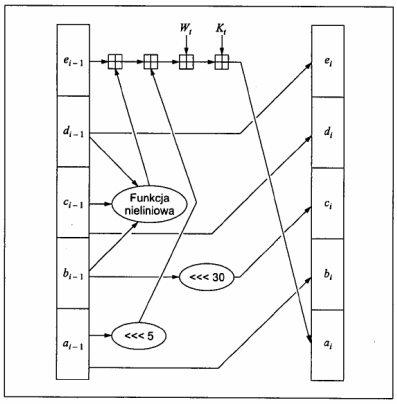
$f_t(X,Y,Z) = (X \wedge Y) \vee ((\neg X) \wedge Z)$ dla $t \in < 0,19 >$

$f_t(X,Y,Z) = X \oplus Y \oplus Z$ dla $t \in < 20,39 >$

$f_t(X,Y,Z) = (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z)$ dla $t \in < 40,59 >$

$f_t(X,Y,Z) = X \oplus Y \oplus Z$ dla $t \in < 60,79 >$

Pojedynczy cykl operacji SHA



SHA

- Funkcje mieszające (MD5 i SHA-1) niestety nie zapewniają już odpowiedniego poziomu bezpieczeństwa ze względu na zbyt krótką wartość skrótu (zwłaszcza, gdy weźmie się pod uwagę atak urodzinowy). Ma to znaczenie zwłaszcza przy używaniu nowych algorytmów szyfrów blokowych, takich, jak AES (z kluczem 256-bitowym). Wychodząc naprzeciw oczekiwaniom, zostały opublikowane specyfikacje nowych funkcji z rodziny SHA: SHA-256, SHA-384 i SHA-512, przeznaczone do współpracy z algorytmami blokowymi z kluczem o długości odpowiednio 128, 192 i 256 bitów.

HAVAL

- Zaproponowana w 1992
- W latach 2002-2003 przedstawiono kilka ataków na zredukowaną wersję algorytmu
- W następnych latach ataki na pełną wersję algorytmu

RIPEMD

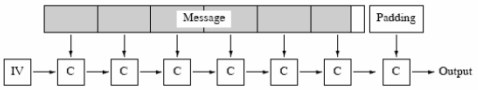
- Powstała w ramach projektu Unii Europejskiej RIPE (RACE Integrity Primitives Evaluation)

Rodzina Funkcji Petra

- rodzina funkcji Petra składającą się z 4 funkcji
- każda funkcja należąca do rodziny jest oznaczana jako Petra-r/v, gdzie $r \in \{192,256\}$ jest długością skrótu w bitach, natomiast $v \in \{1,2\}$ oznacza jedną z dwóch wersji funkcji kompresującej.

Rodzina Funkcji Petra - struktura

- blok długości 512 bitów
- ostatni blok jest uzupełniany ciągiem 100…0
- na ostatnich 64 bitach zapisywana jest długość oryginalnej wiadomości



Wszystkie funkcje z rodziny Petra bazują na tych samych zmiennych inicjujących:

IV[0]=$DAA4361D IV[1]=$B16EC431 IV[2]=$615BE562 IV[3]=$98256F2E
IV[4]=$E565B322 IV[5]=$98D8EB18 IV[6]=$53A0EF98 IV[7]=$6EA9D519

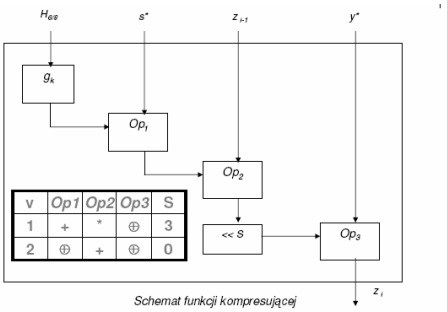
oraz tych samych stałych y*:

y[0] =$4CC59886 y[1] =$33166219 y[2] =$F31941D9 y[3] =$E9053E34
y[4] =$155DD64D y[5] =$9809045A y[6] =$44E8ECDD y[7] =$B310C998
y[8] =$CA0ECF98 y[9] =$29F1A748 y[10]=$EEB268AA y[11]=$4822D4C0
y[12]=$4766EA27 y[13]=$864CC598 y[14]=$767CC650 y[15]=$8D3A414F

a także wykorzystują funkcje:

$g_{1/192} = H[0] \wedge H[2] \oplus H[0] \wedge H[5] \oplus H[1] \wedge H[3] \oplus H[1] \wedge H[5] \oplus H[4] \wedge H[5]$
$g_{2/192} = H[0] \wedge H[1] \oplus H[0] \wedge H[4] \oplus H[2] \wedge H[3] \oplus H[4] \wedge H[5]$
$g_{3/192} = H[0] \wedge H[3] \oplus H[0] \wedge H[5] \oplus H[1] \wedge H[4] \oplus H[2] \wedge H[4] \oplus H[2] \wedge H[5] \oplus H[3] \wedge H[4]$
…

Blok kompresujący



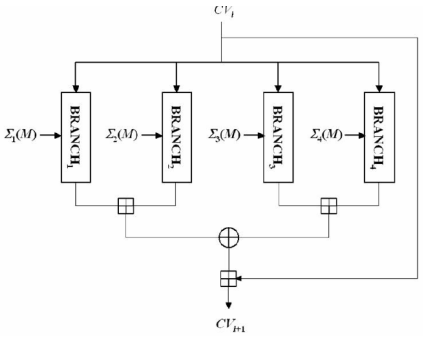
v	Op1	Op2	Op3	S
1	+	*	⊕	3
2	⊕	+	⊕	0

Schemat funkcji kompresującej

Wykorzystywane operacje

⊕ – xor logiczny
∧ – and logiczny
+ – dodawanie mod 2^{32}
* – mnożenie mod 2^{32}
S – wielkość przesunięcia bitowego
L – liczba rund

Algorytm FORK-256

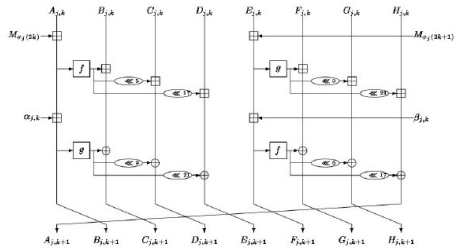


- Message Block Size : 512 bits (16 words)
- Output Size : 256 bits (8 words)
- Use 4 Branches
- 8 steps in Each Branch

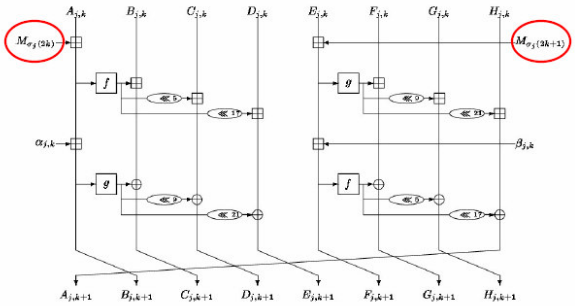
Wykorzystywane operacje

$\boxplus$: addition mod 2^{32}
$\oplus$: XOR (eXclusive OR)
$A^{\lll s}$: s-bit left rotation for a 32-bit string A

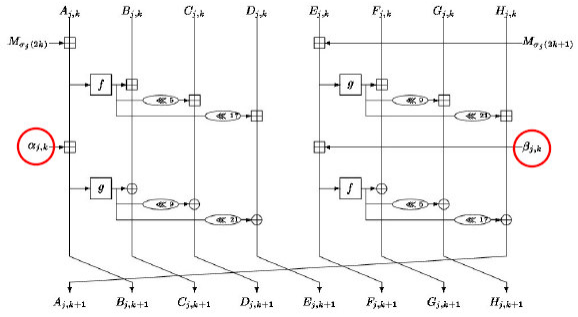
- Each Branch consists of 8 steps



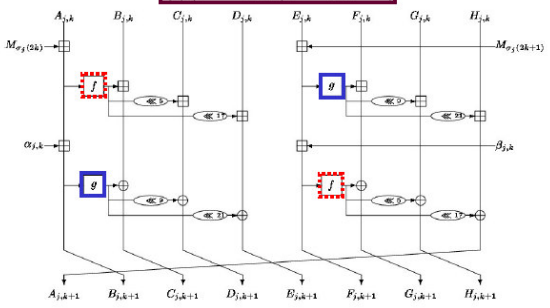
- Two message words in each step



- Two constants in each step



- f, g function : $f(x) = x \boxplus (x^{\lll 7} \oplus x^{\lll 22})$, $g(x) = x \oplus (x^{\lll 13} \boxplus x^{\lll 27})$



Bezpieczeństwo w GSM

- Dla systemu GSM odpowiednie długości parametrów stosowanych w procedurach bezpieczeństwa wynoszą:
- klucz identyfikacyjny abonenta Ki: 128 bitów;
- zapytanie autentykacyjne RAND: 128 bitów;
- odpowiedź autentykacyjna SRES: 32 bity;
- klucz szyfrujący Kc: 64 bity.

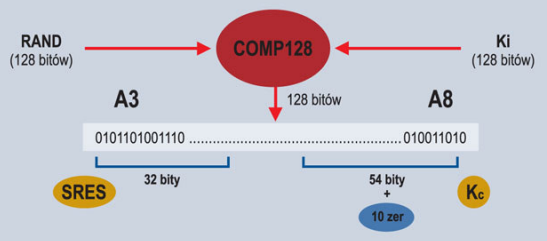
- cały model bezpieczeństwa GSM bazuje na tajności klucza identyfikacyjnego Ki. Ujawnienie tego klucza oznacza dla konkretnego abonenta złamanie podstawowych zabezpieczeń systemowych (a więc możliwość nieuprawnionego dostępu do przypisanych mu usług, jak również zniesienie poufności informacji). Znajomość klucza Ki daje potencjalnemu włamywaczowi nie tylko pełny dostęp do połączeń (włączając w to deszyfrowanie informacji), ale pozwala też podszywać się pod legalnego abonenta, np. przeprowadzając rozmowy na jego koszt.

- Algorytmy bezpieczeństwa związane z autentykacją (A3) i generowaniem klucza szyfrującego (A8) zostały w specyfikacjach GSM pozostawione do uznania operatorów poszczególnych sieci komórkowych.

- Teoretycznie każdy z operatorów mógłby używać innej pary A3 i A8. Niestety, przytłaczająca większość operatorów wybrała wspólny dla wszystkich "podwójny" algorytm A3/A8, znany pod nazwą COMP128, obliczający zarówno SRES, jak i Kc w jednym przebiegu

DZIAŁANIE ALGORYTMU COMP128

Algorytm COMP128 wytwarza odpowiedź autentykacyjną SRES i klucz szyfrujący Kc w tym samym przebiegu, według schematu przedstawionego poniżej.

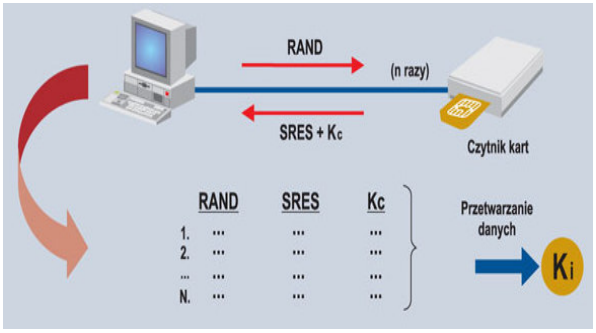


Niestety, wartość klucza Kc jest w algorytmie COMP128 brana jako ostatnie 54 (a nie 64!) bity wygenerowanej 128-bitowej odpowiedzi. Ponieważ algorytm szyfrujący A5 wymaga 64-bitowego klucza Kc, do 54-bitowej części właściwej dodaje się jeszcze dziesięć bitów zerowych, co wydatnie obniża poziom bezpieczeństwa operacji szyfrowania.

- W 1998 r. Stowarzyszenie Wytwórców Kart Inteligentnych oraz ISAAC2 Security Research Group uniwersytetu Berkeley w Kalifornii ogłosiły odtworzenie pełnego algorytmu COMP128 i znalezienie w nim słabego punktu. Pokazali oni, że osoby lub organizacje posiadające fizyczny dostęp do karty SIM mogą odtworzyć (na podstawie analizy kryptograficznej) odpowiedni klucz identyfikacyjny Ki i sklonować kartę

- Umożliwiłoby to oszustwa polegające na przeprowadzaniu rozmów na koszt właściciela karty poprzez wprowadzenie wyprodukowanego klonu do dowolnego terminala GSM i "udawanie" w ten sposób legalnego użytkownika.

- Odtworzenie klucza Ki z karty SIM implikuje również całkowity dostęp do informacji przekazywanej przez właściciela karty drogą radiową, z uwagi na łatwość obliczenia klucza Kc i deszyfrowania połączeń.



- Teoretycznie istnieje metoda ataku, polegająca na odtworzeniu klucza Ki metodą bezprzewodową.
- Atak tego rodzaju wykorzystuje fakt, że zgodnie z normami GSM karta SIM musi odpowiedzieć na każde zapytanie autentykacyjne skierowane do niej drogą radiową przez sieć.

- Jeśli więc sygnał wysłany z legalnej stacji bazowej zostanie zdominowany przez silniejszy sygnał pochodzący z fałszywej stacji bazowej, to taka fałszywa stacja może "bombardować" kartę seriami zapytań autentykacyjnych, a w rezultacie uzyskane zostaną dane potrzebne do odtworzenia klucza Ki.

- Brak procedury odwrotnej autentykacji (stacji bazowej przez telefon) całkowicie uniemożliwia telefonowi GSM wykrycie tego rodzaju sytuacji. Dodatkowo, atakujący nie musi znać PIN-u karty SIM, ponieważ wprowadza go w dobrej wierze właściciel telefonu w momencie logowania do sieci. Uzyskanie danych umożliwiających obliczenie klucza Ki jest tutaj kwestią ok. kilku godzin, dlatego nie jest realne zastosowanie ataku ciągłego.

- tak można prowadzić z przerwami, np. po kilkanaście minut dziennie, rozciągając go na wiele dni - chodzi jedynie o zebranie wystarczającej liczby danych. Z powyższego opisu widać też jasno, że atak bezprzewodowy jest bardzo trudny do wykrycia.

- W maju 2002 r. naukowcy z koncernu IBM odkryli nowy sposób uzyskania klucza Ki, przy zastosowaniu ataku poprzez tzw. kanały boczne (obserwację emisji ujawniających, zużycia mocy, przebiegów czasowych oraz błędów w trakcie przeprowadzanych w karcie SIM obliczeń).

- Metoda ta umożliwia odtworzenie klucza przy użyciu jedynie 1000 przypadkowo wybranych wartości parametru RAND. Stosując tę metodę, możliwe było znaczne zredukowanie ilości czasu potrzebnego do sklonowania karty SIM - był on co najwyżej rzędu minut, a w najlepszym wypadku mógł być rzędu sekund4!

- Algorytm szyfrujący A5 dostarczany jest w dwóch wersjach: A5/1 i A5/2. Pierwsza wersja A5/1, pochodząca z zastosowań wojskowych, uważana była początkowo za zbyt "silną", aby można ją było eksportować do niektórych państw (takich jak kraje Bliskiego Wschodu czy ówczesnej Europy Wschodniej).

- Zaprojektowano więc i wprowadzono do użytku "słabszą" wersję A5/2. Istnieje również wersja A5/0, która oznacza po prostu brak szyfrowania5, więc niektórzy operatorzy nie szyfrują połączeń

- Najprostszą metodą, podsłuchu w systemie GSM niewymagającą nawet złamania algorytmu szyfrującego, jest metoda wykorzystująca zasadę "man in the middle". Posłużyć do tego może tzw. IMSI catcher - urządzenie diagnostyczne, służące do symulowania stacji bazowej przy testowaniu telefonów komórkowych.

- IMSI catcher (użyty przez włamywacza) może udawać przed telefonem stację bazową, a przed inną stacją bazową może udawać telefon (problem braku autentykacji stacji bazowej przez telefon). Za pomocą tego urządzenia możliwe jest wyłączenie szyfrowania (poprzez ustanowienie wersji A5/0) i zastosowanie pasywnego podsłuchiwania rozmów.

- Mimo że szczegóły projektowe obu algorytmów A5/1 i A5/2 były utrzymywane w ścisłej tajemnicy przez władze GSM Association, zostały one ujawnione w formie przecieków po roku 1994, a ostatecznie cały algorytm udało się odtworzyć w roku 1999 z pamięci terminala GSM metodami tzw. inżynierii odwrotnej (reverse engineering).

- W późnych latach 90. podjęto kilka prób analizy i złamania algorytmów A5. Zaowocowały one lepszym zrozumieniem podstaw kryptograficznych tej grupy algorytmów i znalezieniem konkretnych słabych punktów w ich architekturze. Jednak nadal próby oszacowania czasu potrzebnego do odtworzenia klucza Kc przy użyciu zwykłego komputera PC dawały wyniki rzędu kilkuset godzin (a więc kilkunastu dni), co nie mogło zagrozić rozmowom prowadzonym w czasie rzeczywistym.

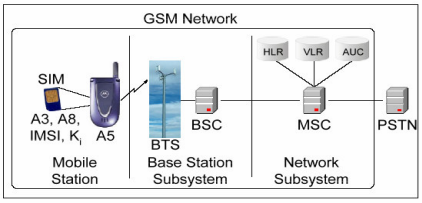
- Ostatecznie w roku 1999 zademonstrowano nowy rodzaj ataku, oparty na subtelnych słabościach w strukturze i zasadzie działania tzw. rejestrów przesuwnych używanych w implementacji sprzętowej algorytmu A5/1. Zademonstrowany atak został przeprowadzony z użyciem komputera PC wyposażonego w olbrzymią (jak na owe lata) pamięć dyskową ok. 300 GB, potrzebną do zapisu opracowanej uprzednio macierzy stanów algorytmu.

- Rezultatem ataku było odtworzenie klucza Kc w czasie rzeczywistym na podstawie niewielkiej liczby danych uzyskanych z krótkiego fragmentu rozmowy8. Otworzyło to drogę do deszyfrowania połączeń w sieci GSM i ich podsłuchu drogą radiową.

- Po udowodnieniu słabych punktów GSM Association stworzyła jego nową wersję COMP128-2. Stara wersja nosi od tego momentu oznaczenie COMP128-1. Nowy algorytm został już zaimplementowany w nowo produkowanych kartach SIM i jest stosowany przez niektórych operatorów. Na razie wydaje się, że nie posiada on wewnętrznych wad poprzedniej wersji, a z informacji wymienianych między sobą przez hakerów na forach internetowych wynika, iż - jak do tej pory - nie został on jeszcze poznany (kart SIM opartych na COMP128-2 nie da się więc na razie klonować).

GSM architecture

GSM System Architecture



GSM - establishing communication

Immediate assignment procedure:

(a) The phone sends a CHANNEL REQUEST message on the random access channel (RACH). The CHANNEL REQUEST message includes a very small amount of information — only 8 bits. It does not contain an identification of the mobile, rather it includes a random discriminator (5 bits). The remaining three bits contain the establishment cause.

(b) The network broadcasts an IMMEDIATE ASSIGNMENT message on the PAGCH. This message contains the random discriminator (and also the TDMA frame number in which the CHANNEL REQUEST was received), and the details of the channel that is allocated to the mobile (including frequency hopping information, if needed). The messages also includes other technical information such as timing advance. The mobile immediately tunes to the the assigned traffic channel.[11]

[From Barkan et al]

Service Request and Contention Resolution:

(a) The mobile sends a service request message (e.g., paging response, service request, etc.), this message includes the TMSI of the mobile. The message also includes the mobile classmark (including the A5 versions that are supported), and a ciphering key sequence number $(0, \ldots, 6)$.

(b) The network acknowledges the service request message, and repeats the TMSI. The reason for repeating the TMSI is contention resolution: It is possible that two mobiles used the same random discriminator on the same TDMA frame, and therefore, both "think" that they are assigned to the same channel. The mobile that his TMSI is acknowledged by the network, stays on the channel, and the other mobile quits.

GSM - establishing

Authentication:

(a) The network sends authentication request (AUTHREQ). The authentication request includes a random 128-bit value RAND, and a ciphering key sequence number, in which the resulting Kc should be stored.

(b) The mobile answers the authentication with the computed signed response (SRES), in an authentication response message (AUTHRES).

(c) The network asks the mobile to start encryption using a cipher mode command (CIPHMODCMD). The network can specify the encryption algorithm to be used, and it specifies the encryption key by a ciphering key sequence number $(0, \ldots, 6)$. The network starts to decipher incoming communication. This message can also be used to ask the mobile to send its international mobile equipment identity, and software version (IMEISV).

(d) The mobile starts to encrypt and decrypt, and responds with (encrypted) cipher mod complete message (CIPHMODCOM). If requested, the mobile sends its IMEISV.

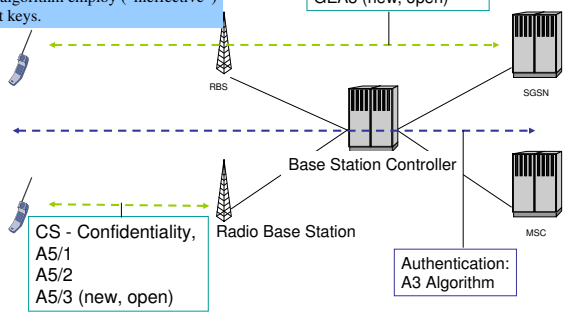
The network and the mobile "talk" on the channel. It might well be that the network changes the channel. For example, if it is a voice conversation the channel might need to be changed to suit a voice conversation, etc. In case a channel is changed or a handover is needed, the new channel information is sent by the network (including the frequency hopping information). Note that if the conversation is encrypted, then the new channel information is encrypted as well.

[From Barkan et al]

GSM security

54 bits is the effective key length of the A5/1 algorithm.
40 bits is the effective key length of the GEA algorithm.
Both algorithm employ ("ineffective") 64-bit keys.

GPRS - Confidentiality:
GEA1
GEA2
GEA3 (new, open)

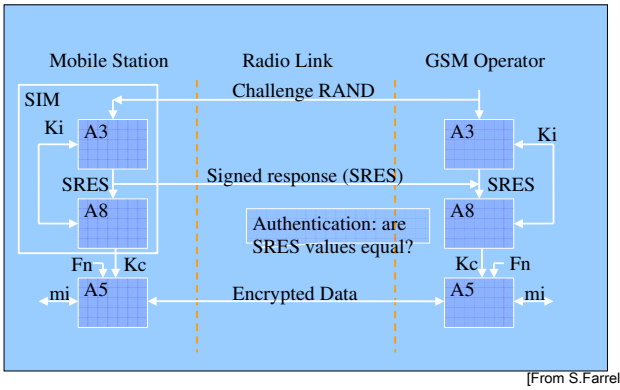


CS - Confidentiality,
A5/1
A5/2
A5/3 (new, open)

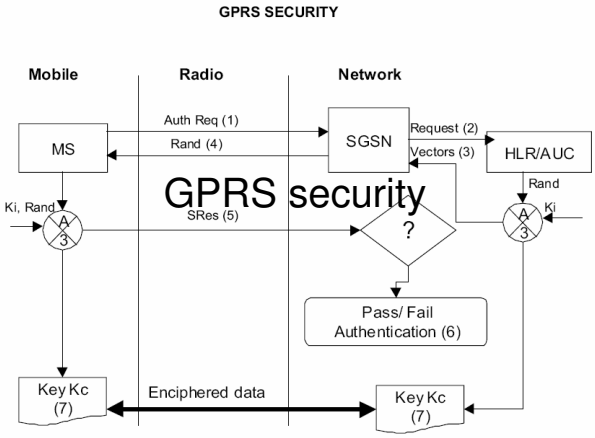
Authentication:
A3 Algorithm

[From M.Näslund]

GSM security

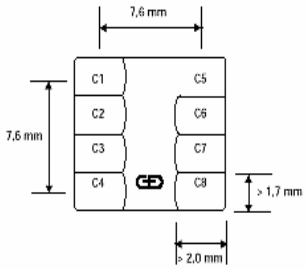


[From S.Farrell]



Subscriber Identity Module

- C1: Supply voltage
  - (4.5 to 5.5 volts DC).
- C2: Reset signal
- C3: Clock signal
  - (1 to 5 MHz, external)
- C4: Reserved
- C5: Ground
- C6: Programming voltage
  - (if available)
- C7: Input/Output
  - Baudrate is (clock frequency) / 372.
- C8: Reserved



[From D.Veeneman]

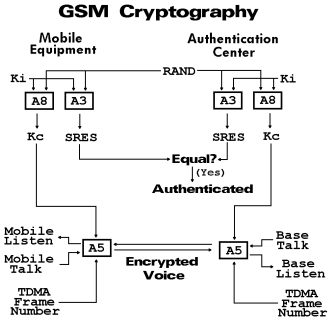
SIM attacks

- Repeated authenticate, leaks Ki
  - (New SIMs have a limit (about 50k) on the number of times the authentication algorithm can be run)
- Side-channel attacks
  - Power consumption
  - Timing
  - Electromagnetic emanations

[From D.Veeneman]

GSM authentication

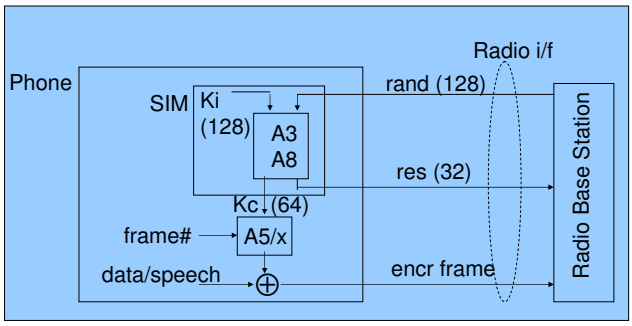
- A random challenge is issued to the mobile
- Mobile encrypts the challenge using the authentication algorithm (A3) and the key assigned to the mobile (Ki)
- Mobile sends response back (SRES)
- Network checks that the response to the challenge is correct.



[From D.Veeneman]

GSM authentication

A3 and A8: Authentication and key derivation (proprietary)
A5: encryption (A5/1-4, standardized)
(No netw auth, no integrity/replay protection)



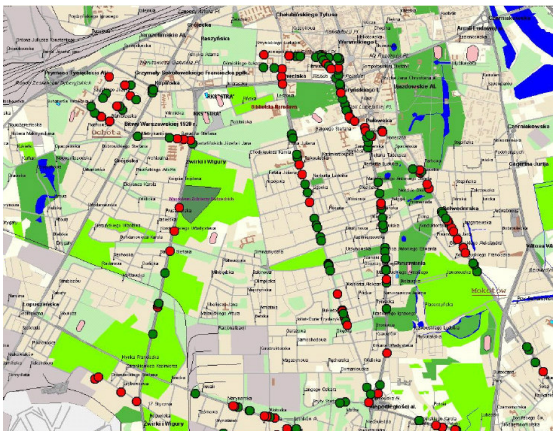
[From M.Näslund]

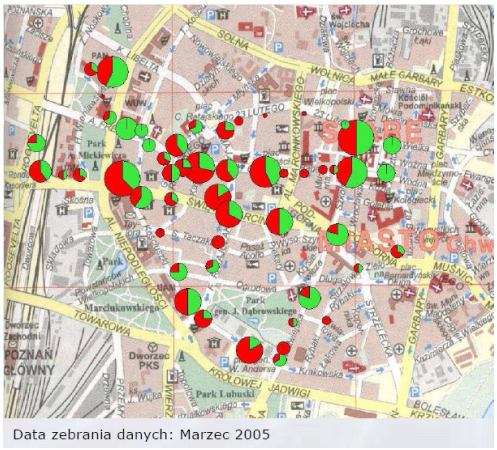
COMP128 updates

- COMP128-2
  - 54-bit Kc
  - Secret algorithm
- COMP128-3
  - 64-bit Kc
  - Secret algorithm
- Proposal for new A3/A8 based on MILENAGE
  - Milenage based on Rijndael (AES)
  - Algorithm will be public
- New A3/A8 requires
  - AuC software upgrade
  - New SIMs

[From D.Veeneman]

Bezpieczeństwo w sieciach WLAN 802.11





Aspekty bezpieczeństwa

- Poufność (słaba)
- Integralność (niekryptograficzna)
- Uwierzytelnienie (słabe)

Brak kontroli dostępu
Brak zarządzania kluczami

wszystkie usługi związane z
WEP – Wired Equivalent Privacy

WEP Wired Equivalent Privacy

Cel:

Przy pomocy szyfrowania zapewnić poziom bezpieczeństwa przewodowych sieci lokalnych.

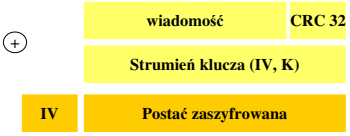
Cechy:

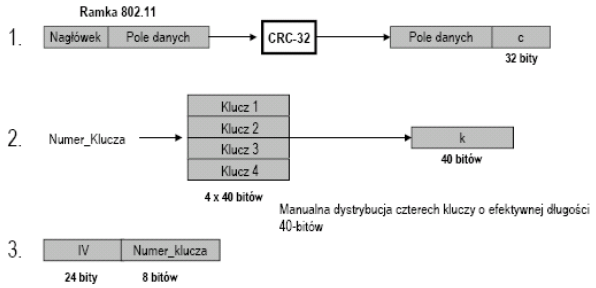
Siła w tajności klucza, eksport poza USA (klucz 40 bitowy), użycie WEP jako opcji.

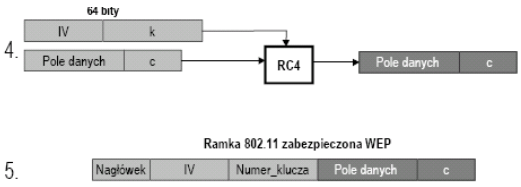
W rezultacie cel nie został osiągnięty na skutek niewłaściwego użycia szyfru RC4 i braku zarządzania kluczami.

WEP szyfrowanie

- Wykorzystuje mechanizm klucza wspólnego z symetrycznym szyfrem RC4







- klucze WEP statycznie skonfigurowane we wszystkich elementach sieci,
- można zdefiniować do 4 kluczy na urządzenie, ale do szyfrowania wychodzącej ramki można użyć tylko jednego,
- szyfrowanie stosowane tylko do ramek danych i podczas uwierzytelniania,
- szyfrowaniu podlegają dane oraz wartość ICV (Integrity Check Value),
- wektory IV nie są szyfrowane

Programy do łamania kluczy WEP

- AirSnort
- Aircrack
- WepLab

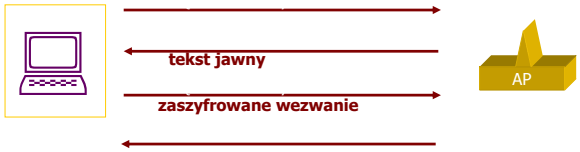
802.11 uwierzytelnienie

- Otwarty typ uwierzytelnienia
  - Punkt dostępowy akceptuje stację bez jakichkolwiek zastrzeżeń
- Uwierzytelnienia typu współdzielonego klucza
  - oparte na WEP – polega na wykazaniu faktu posiadania tajnego klucza
  - mechanizm wymiany uwierzytelniającej: wyzwanie-odpowiedź

Uwierzytelnienie otwarte



Uwierzytelnianie ze współdzielonym kluczem WEP



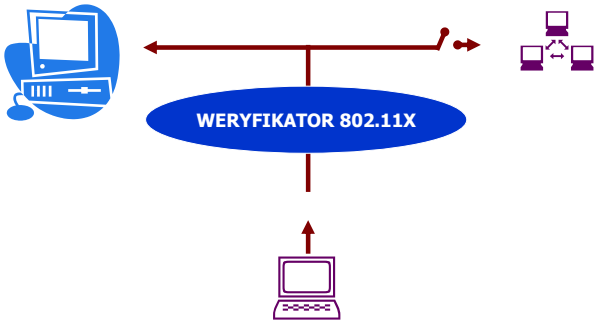
WPA

- Wi-Fi Protected Access (WPA)
  - Temporal Key Integrity Protocol (TKIP)
  - Message Integrity Check (MIC)
  - EAP i 802.1x

Uwierzytelnienie przez EAP i 802.1x

- Protokół EAP (Extensible Authentication Protocol) polega na sekwencji pytań i odpowiedzi między jednostką uwierzytelniającą a stacją roboczą. W zależności od otrzymanych odpowiedzi przydziela dostęp lub nie.
- EAP znalazł zastosowanie w specyfikacji 802.1x nazywanej kontrolą dostępu do sieci na podstawie uwierzytelniania portów.

Szkielet uwierzytelniania



Uwierzytelnienie przez EAP i 802.1x



1. Punkt dostępowy żąda od klienta podania tożsamości
2. Klient podaje do AP swoją tożsamość
3. AP przekazuje informację o otwartym porcie do serwera RADIUS.
4. Serwer RADIUS żąda uwierzytelnienia od klienta
5. AP przesyła żądanie 4. do klienta.
6. Odpowiedź klienta na 5.zawiera np. hasło lub poprawne szyfrowanie zawartego w żądaniu 4. ciągu znaków.
7. Przekazanie odpowiedzi do serwera RADIUS
8. Serwer RADIUS sprawdza odpowiedź i przekazuje odpowiednią informację do AP
9. AP udostępnia kontrolowany port i przesyła z niego komunikat do klienta

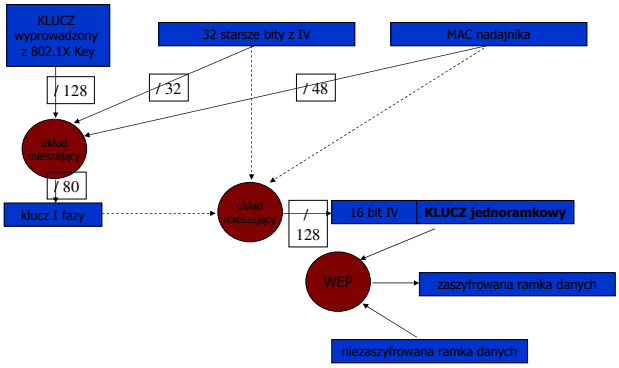
Algorytm uwierzytelniania

- WPA oraz 802.11i nie narzucają konkretnych algorytmów uwierzytelniania,
- zalecane są takie algorytmy, które obsługują:
  - wzajemne uwierzytelnianie na bazie użytkowników,
  - dynamiczne generowanie kluczy szyfrowania,

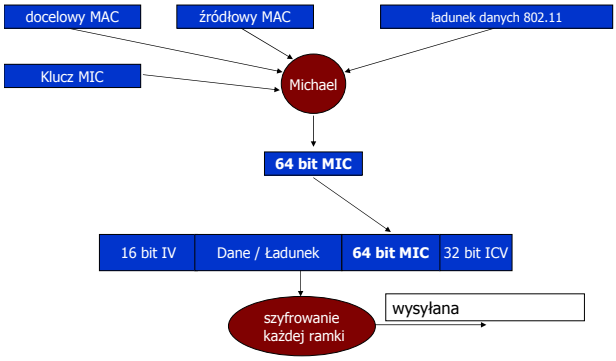
TKIP właściwości

- Nowy algorytm zapewniania integralności komunikatów, Michael
- Mechanizm chroniący przed atakami wykorzystującymi powtórzenia nadawanych ramek
- Wykorzystuje ten sam hardware co algorytm WEP
- Wprowadza mechanizmy informowania o tym, że sieć jest atakowana (w przeciwieństwie do WEP)

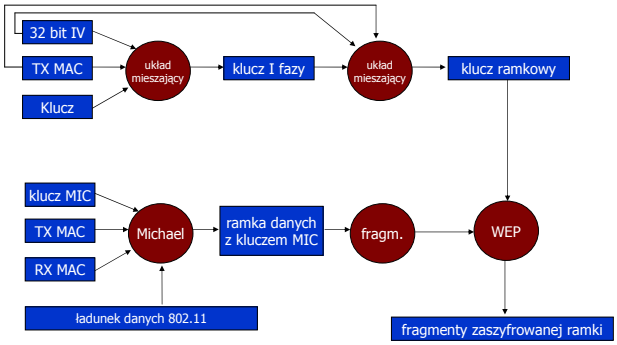
Poufność danych



Integralność danych - MIC



Pełny mechanizm szyfrowania TKIP



Zarządzanie kluczem

- dwa podstawowe klucze: klucz szyfrowania i klucz MIC,
- klucz wyprowadzony w trakcie uwierzytelniania to klucz główny *(master key*), który jest wykorzystywany do wyprowadzania pozostałych kluczy,
- mechanizm generowania kluczy szyfrowania nosi nazwę *czteroetapowego uzgadniania* (*four-way handshake*)
- mechanizm zarządzania kluczami i ich dystrybucji wykorzystuje architekturę 802.1x

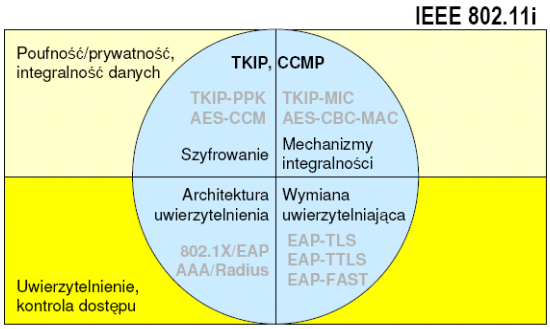
802.11i

Cele przyświecające powstawaniu standardu 802.11i:

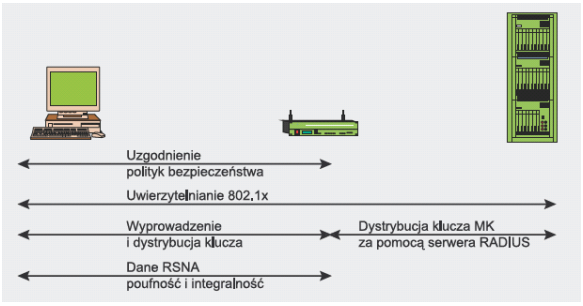
- technologia dostępna dla wszystkich,
- brak ograniczeń patentowych na stosowane algorytmy
- elastyczna, adaptowalna architektura, nadająca się zarówno dla małych jak i dużych wdrożeń
- zewnętrzne recenzje, aby zminimalizować szanse na kolejny WEP

802.11i

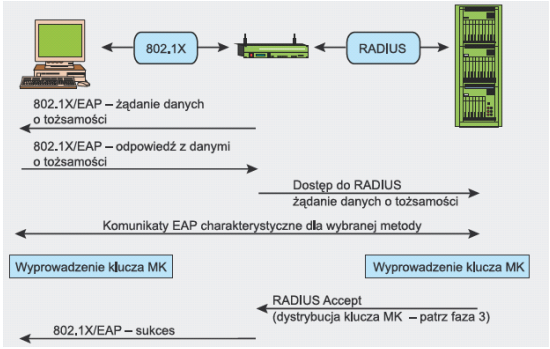
- Rozszerzenie uwierzytelnienia
- Algorytmy zarządzania kluczami, w tym algorytmy generacji kluczy
- Mechanizmy ochrony w warstwie MAC: Kodowanie CCMP i opcjonalnie TKIP



802.11i



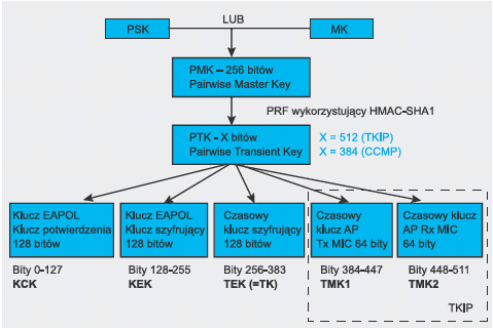
802.11i

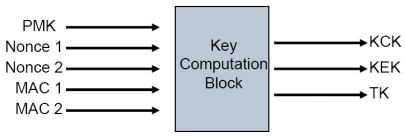


Hierarchia i dystrybucja kluczy

- Bezpieczeństwo połączenia zależy od zbioru tajnych kluczy. Każdy klucz ma ograniczony czas ważności i odpowiada za określony proces.
- Celem fazy trzeciej jest generacja i wymiana kluczy.

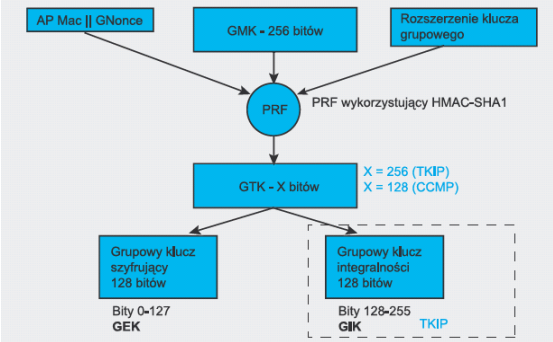
802.11i



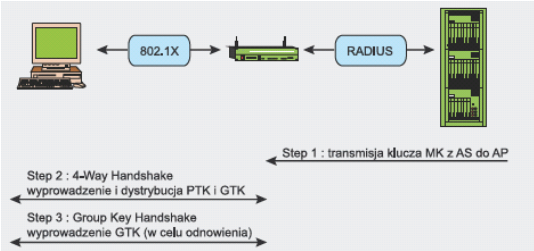


„Nonce” - liczba losowa
Nonce 1 – generowana przez AP
Nonce 2 – generowana przez Klienta

802.11i



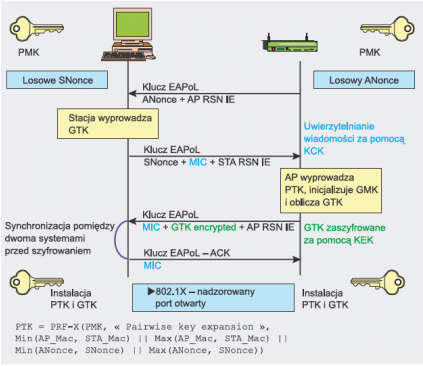
802.11i



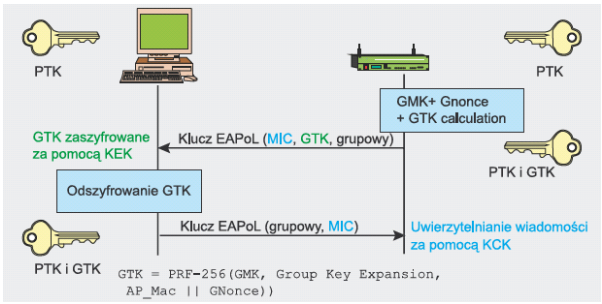
Negocjacja czteroetapowa

- Potwierdzenie, ze klient faktycznie zna klucz PMK
- Wygenerowanie nowego klucza PTK
- Instalacja kluczy szyfrowania i integralności
- Szyfrowanie transportu klucza GTK
- Potwierdzenie wyboru zestawu szyfrów

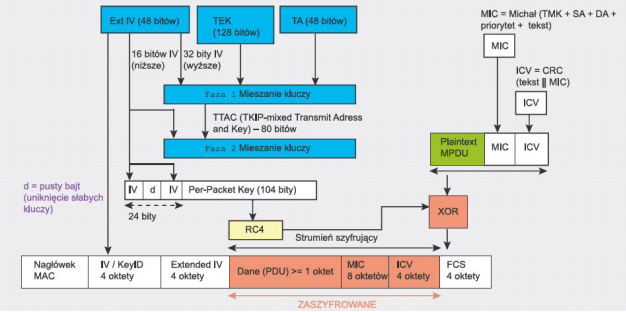
Negocjacja czteroetapowa



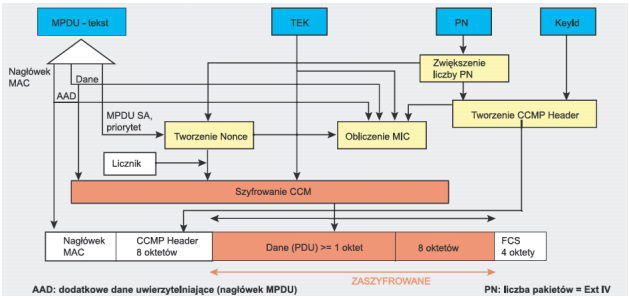
Negocjacja dwuetapowa



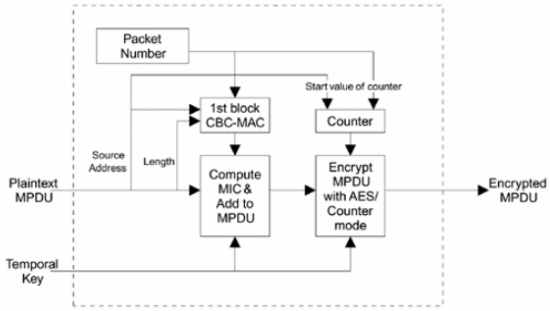
Szyfrowanie TKIP
(Temporal Key Integrity Protocol)



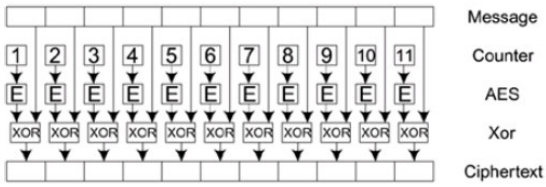
Szyfrowanie CCMP
(Counter-Mod/Cipher Block Chaining Message Authentication Code Protocol)



Blok szyfrowania CCMP



Tryb CCM



- Nie wszystkie problemy związane z ochroną danych zostały rozwiązane w standardzie 802.11i:
- Komunikaty grupowe zabezpieczane są wspólnym kluczem, a więc poufność ograniczona jest do grupy
- Brak zabezpieczenia dla ramek typu Management
- Brak zabezpieczeń w warstwie PHY

Podsumowanie

- Wyłączyć rozgłaszanie SSID przez AP
- Włączyć szyfrowanie WEP z max kluczem
- Włączyć szyfrowanie dla wszystkich klientów w sieci
- Regularnie aktualizować klucz statyczny
- Stosować niestandardowe nazwy identyfikatorów SSID
- Używać długich i trudnych do zgadnięcia haseł WEP (20, 85)
- Sprawdzać regularnie pliki dzienników w poszukiwaniu nieznanych adresów MAC
- Na włączanie do sieci zezwalać tylko klientom o ustalonych adr. MAC
- Wyłączyć w AP serwer DHCP i zdefiniować ręcznie adresy IP komputerów w sieci
- Ustawić odpowiednio anteny nadawczo-odbiorcze
- Izolować AP od strony przewodowej LAN przy pomocy zapór ogniowych
- Jeśli to możliwe do autoryzacji połączeń wykorzystuj serwery RADIUS